

Курсовая работа с нейросетью

User: Anonymous

Created: 5/24/2026 15:19:07

Updated: 5/24/2026 17:31:32

Exported: 5/24/2026 17:53:51

Link: <https://chatgpt.com/g/g-p-6a12eca4912081918edf4cd40ac43449-proekt-ii/c/6a12ecb5-bd04-83eb-827c-ebe99b837239>

Prompt:

24.05.2026, 15:19:02

Привет, смотри, я обучаюсь на 2 курсе в университете на программировании, заданием преподавателя на курсовую работу было придумать какое нибудь приложение, а затем ТОЛЬКО с помощью нейросети реализовать этот проект в реальность, главным условием является то, что я не могу внести ни одну строчку саморучно, все сделать должен только ты, а если ты делаешь что то неправильно я подправлять тебя не могу, только дополнительными промптами я должен реализовать то, что вижу, если ты готов, то я предоставлю тебе проект, который я придумал, поехали?

Response:

24.05.2026, 15:19:07

Да, поехали.

Но сразу честно: **я могу полностью писать код, структуру проекта, команды запуска, исправления и объяснения**, а ты будешь только копировать/запускать и давать мне обратную связь промптами: что сломалось, какой текст ошибки, что не нравится визуально или функционально. Это подходит под условие “ни одной строчки вручную”.

Пришли проект в таком виде:

1. **Идея приложения** — что оно делает.
2. **Для кого приложение** — пользователь/администратор/студент/магазин и т.д.
3. **Основные функции** — хотя бы 5–10 пунктов.
4. **На чем хочешь делать:**
например C#, Java, Python, Web, Android, Windows Forms, WPF, консоль, сайт.
5. **Требования преподавателя** — если есть: база данных, авторизация, отчёт, диаграммы, ООП, паттерны, интерфейс.

6. **Какой уровень сложности нужен** — “простенько, но рабочее” или “выглядит как нормальная курсовая”.

7. **Где будешь запускать** — Windows / macOS / Linux / Visual Studio / VS Code.

После этого я составлю структуру проекта, выберу реалистичный стек и начну выдавать готовые файлы без ручной дописки с твоей стороны.

Prompt:

24.05.2026, 15:19:51

[Attachment: Вставленный текст.txt]

Response:

24.05.2026, 15:19:52

Searching CoinGecko API documentation and limitations

[Crypto Data API: Most Comprehensive & Reliable ...](#) [Coin Historical Chart Data within Time Range by ID](#) [CoinGecko API: The Cryptocurrency Data Powerhouse](#) [CoinGecko CLI - Real Time & Historical Crypto Data](#) [coingecko_api - Dart API docs](#) [CoinAPI vs CoinGecko: Which Crypto API Gives You a ...](#) [Interval=hourly not working on /market_chart endpoints](#)

План проекта для интерфейса

Нужно завершить проект на русском. Для архитектуры: начать с окружения Stage 0, возможно, использовать VS и Qt 6 с CMake и C++17. Skeleton UI на этапе 1 без API с фейковыми данными. Stage 2 — сетевое взаимодействие с API. Stage 3 — исторические данные и графики.

Реализация графиков и подходы к зависимостям

Документы говорят, что `interval=hourly` доступен для всех тарифных планов с 24 марта 2026 года. Для OHLC параметр доступен только для платных тарифов. На первом этапе будут использоваться данные с `market_chart` (линии + объем + RSI). Для построения графиков можно использовать `Qt Charts` или создать собственный виджет с `QPainter`.

Думал на протяжении 21s

Сначала отделию идею от реализации: проект нормальный, но слабое место — не интерфейс, а данные для свечей/RSI/объёмов и ограничения API. Проверю, на каких CoinGecko-методах это реально собрать без ручной подделки данных, и после этого дам план по этапам.

Проект нормальный, но его нельзя начинать с “сразу делаем всё приложение”. Так ты гарантированно утонешь в Qt, API, графиках и ошибках сборки. Делать надо слоями: сначала

интерфейс на фейковых данных, потом API, потом графики, потом аналитика, потом оповещения и конвертор.

Я изучил твоё описание: у тебя утверждён проект **CryptoTrack на C++/Qt**, 7 монет, левая панель выбора, центральный анализ монеты, графики 24ч/7д/30д/90д, свечи/линия, объёмы, RSI, оповещения и конвертор. Это совпадает с тем, что ты отправлял преподавателю и что он уточнял по функциональности. Вставленный текст.txt

Важное слабое место проекта

Главная проблема не в коде, а в данных.

CoinGecko действительно даёт текущие рыночные данные через endpoint `/coins/markets`, а исторические цены, капитализацию и объёмы через `/coins/{id}/market_chart` и `/coins/{id}/market_chart/range`. Также у них есть OHLC endpoint для свечей. (docs.coingecko.com)

Но свечи - потенциально проблемное место. У CoinGecko есть отдельный `/coins/{id}/ohlcv`, но доступность точной дискретности зависит от тарифа и параметров API. Поэтому безопаснее строить проект так:

линейный график, объёмы и RSI делаем через `market_chart`; свечи добавляем отдельным этапом через OHLC или агрегируем из исторических точек, если API не даст нужный формат.

Так мы не застрянем на одном красивом, но рискованном пункте.

Рабочий план реализации CryptoTrack

Этап 0. Подготовка проекта

Цель: создать пустой Qt-проект, который стабильно собирается.

Стек предлагаю такой:

- **C++17**
- **Qt 6**
- **CMake**
- GUI на **Qt Widgets**
- HTTP-запросы через `QNetworkAccessManager`
- JSON через `QJsonDocument`
- графики лучше рисовать вручную через `QPainter`, а не через Qt Charts

Почему ручные графики лучше: меньше зависимостей, проще контролировать свечи, RSI, объёмы, цвета и внешний вид. Для курсовой это выглядит даже сильнее, потому что можно объяснить, что графики строятся собственным виджетом.

Результат этапа: открывается окно `CryptoTrack`.

Этап 1. Каркас интерфейса без API

Цель: собрать внешний вид приложения на тестовых данных.

Окно делим на 3 части:

Слева

Список монет:

- BTC - Bitcoin
- ETH - Ethereum
- SOL - Solana
- DOGE - Dogecoin
- ADA - Cardano
- XRP - Ripple
- DOT - Polkadot

У каждой монеты:

- название
- цена
- изменение за 24 часа
- стрелка вверх/вниз
- цвет: зелёный/красный

Центр

Детальная карточка выбранной монеты:

- название монеты
- крупная текущая цена
- изменение за 24 часа
- кнопки периода: 24ч , 7д , 30д , 90д
- переключатель графика: Линия / Свечи
- основной график
- график объёмов
- RSI-график

Справа

Панель:

- блок оповещений
- выбор монеты
- условие: выше/ниже
- целевая цена
- список активных оповещений
- конвертор:
 - из какой монеты
 - в какую монету
 - количество
 - кнопка рассчитать
 - результат

Снизу

- время последнего обновления
- кнопка `Обновить`

Результат этапа: приложение выглядит как будущий CryptoTrack, но данные пока фейковые.

Этап 2. Модель данных

Цель: не размазывать данные по интерфейсу.

Создадим структуры:

```
CoinInfo
```

Для текущих данных:

- id CoinGecko
- symbol
- name
- currentPrice
- priceChange24h
- priceChangePercent24h
- totalVolume

```
ChartPoint
```

Для исторических точек:

- timestamp
- price
- volume

```
Candle
```

Для свечей:

- timestamp
- open
- high
- low
- close
- volume

```
Alert
```

Для оповещений:

- coinId
- targetPrice
- conditionAbove
- active

Результат этапа: интерфейс получает данные не напрямую из JSON, а из нормальных C++-структур.

Этап 3. Подключение CoinGecko API

Цель: получить реальные текущие цены.

Сначала делаем только текущие данные для 7 монет.

Будет класс:

```
CoinGeckoApi
```

Методы:

```
fetchMarkets()  
fetchMarketChart(coinId, days)  
fetchOhlc(coinId, days)
```

Для текущих цен используем `/coins/markets`, потому что этот endpoint возвращает рыночные данные по монетам: цену, капитализацию, объём и другие показатели. (docs.coingecko.com)

Пример логики:

- приложение запускается
- отправляет запрос на 7 монет
- получает JSON
- парсит данные
- обновляет левую панель
- выбирает Bitcoin по умолчанию
- показывает его в центре

Результат этапа: слева настоящие цены и изменения за 24 часа.

Этап 4. История цены и линейный график

Цель: при выборе монеты загружать историю.

Периоды:

Период	Что показываем
24ч	точки по часам
7д	точки с шагом около 4 часов
30д	точки по дням
90д	точки по дням

У CoinGecko endpoint `/coins/{id}/market_chart` возвращает исторические данные по цене, капитализации и объёму. (docs.coingecko.com)

Сначала делаем только:

- линейный график цены
- столбики объёма

Без свечей. Это правильнее, потому что график цены и объёмы - основа всего остального.

Результат этапа: при выборе периода меняется график цены и объёма.

Этап 5. RSI

Цель: добавить технический индикатор, чтобы проект выглядел аналитическим, а не просто “таблица курсов”.

RSI считаем по ценам закрытия.

Период RSI:

14 точек

Логика:

- берём последовательность цен
- считаем изменения между соседними точками
- отдельно считаем средний рост и среднее падение
- получаем RSI
- рисуем линию RSI
- добавляем уровни 30 и 70

На защите можно говорить так:

RSI показывает относительную силу движения цены. Если значение выше 70, монета считается перекупленной, если ниже 30 - перепроданной. В приложении он используется как дополнительный индикатор анализа, а не как торговая рекомендация.

Результат этапа: под объёмами появляется RSI-график.

Этап 6. Свечи

Цель: добавить candlestick-график.

Вот здесь аккуратно: преподавателю ты заявил конкретную дискретность:

- 24ч - 1 час
- 7д - 4 часа
- 30д - 1 день
- 90д - 1 день

В CoinGecko есть OHLC endpoint `/coins/{id}/ohlcv`, а в документации также указан endpoint `/coins/{id}/ohlcv/range`. (docs.coingecko.com)

Но на практике мы будем действовать так:

1. Сначала пробуем получить OHLC.
2. Если нужная дискретность нормально приходит - используем её.
3. Если нет - строим приближённые свечи из точек `market_chart`.

Это не обман, а нормальная инженерная защита от ограничений API.

Результат этапа: кнопка `Свечи` переключает основной график с линии на свечной вид.

Этап 7. Уровни поддержки и сопротивления

Цель: добавить ещё один аналитический элемент, который просил преподаватель.

Простая и защищаемая реализация:

- уровень поддержки = локальные минимумы цены
- уровень сопротивления = локальные максимумы цены
- берём несколько последних экстремумов
- рисуем горизонтальные линии на графике

Не надо делать “супер-трейдинг”. Для курсовой достаточно объяснить:

Уровни поддержки и сопротивления рассчитываются по локальным минимумам и максимумам выбранного периода. Они помогают визуально оценить зоны, где цена раньше разворачивалась.

Результат этапа: на графике появляются горизонтальные уровни.

Этап 8. Оповещения

Цель: сделать полезное действие, а не просто отображение данных.

Функционал:

- выбрать монету
- выбрать условие:
 - цена выше
 - цена ниже
- ввести целевую цену
- добавить оповещение
- хранить список активных оповещений
- при обновлении цен проверять условия
- если условие выполнено - показывать `QMessageBox`

Пример:

```
Bitcoin выше 70000 USD
Ethereum ниже 3000 USD
```

Оповещения можно сначала хранить в памяти, потом добавить сохранение в JSON-файл.

Результат этапа: пользователь может создать уведомление, и приложение работает при достижении цены.

Этап 9. Конвертор криптовалют

Цель: реализовать предложение преподавателя про обмен одного актива в другой.

Формула простая:

```
result = amount * priceFromUsd / priceToUsd
```

Пример:

```
1 BTC = priceBTC / priceETH ETH
```

То есть если BTC стоит 70000 USD, а ETH стоит 3500 USD:

```
1 BTC = 70000 / 3500 = 20 ETH
```

Интерфейс:

```
[Bitcoin] → [Ethereum]
Количество: [1]
[Рассчитать]
= 20 ETH
```

Результат этапа: справа работает конвертация между всеми 7 монетами.

Этап 10. Сохранение настроек

Цель: чтобы приложение выглядело законченным.

Сохраняем в JSON:

- активные оповещения
- последнюю выбранную монету
- последний выбранный период
- последний тип графика

Файлы:

```
alerts.json
settings.json
```

Результат этапа: после перезапуска приложение помнит настройки.

Этап 11. Обработка ошибок

Цель: не развалиться на защите, если API не отвечает.

Нужно обработать:

- нет интернета

- CoinGecko не отвечает
- пустой JSON
- превышен лимит запросов
- монета не загрузилась
- история цены не пришла

В интерфейсе:

Не удалось загрузить данные. Проверьте подключение к интернету или повторите обновление позже

CoinGecko сам указывает, что для реального времени и исторических цен можно использовать REST endpoints вроде `/simple/price`, `/coins/markets` и исторические методы; при этом API-тарифы и лимиты зависят от плана. ([CoinGecko](#))

Результат этапа: приложение не падает, а показывает понятную ошибку.

Предлагаемая структура файлов

```
CryptoTrack/
|
├── CMakeLists.txt
|
├── main.cpp
|
├── MainWindow.h
├── MainWindow.cpp
|
├── models/
|   ├── CoinInfo.h
|   ├── ChartPoint.h
|   ├── Candle.h
|   └── Alert.h
|
├── api/
|   ├── CoinGeckoApi.h
|   └── CoinGeckoApi.cpp
|
├── widgets/
|   ├── CoinListWidget.h
|   ├── CoinListWidget.cpp
|   ├── PriceChartWidget.h
|   ├── PriceChartWidget.cpp
|   ├── VolumeChartWidget.h
|   ├── VolumeChartWidget.cpp
|   ├── RsiChartWidget.h
|   ├── RsiChartWidget.cpp
|   ├── AlertPanelWidget.h
|   └── AlertPanelWidget.cpp
```

```
| | ConverterWidget.h
| | ConverterWidget.cpp
|
| services/
| | RsiCalculator.h
| | RsiCalculator.cpp
| | SupportResistanceCalculator.h
| | SupportResistanceCalculator.cpp
|
| storage/
| | AppStorage.h
| | AppStorage.cpp
```

Правильный порядок работы

Первая рабочая версия

Минимум:

1. окно приложения
2. левая панель монет
3. центральная карточка выбранной монеты
4. правая панель-заглушка
5. фейковые цены
6. переключение монет

Это будет **версия 0.1**.

Вторая рабочая версия

Добавляем API:

1. реальные цены
2. изменение за 24 часа
3. кнопку обновления
4. время последнего обновления
5. обработку ошибки сети

Это будет **версия 0.2**.

Третья рабочая версия

Добавляем графики:

1. история цены
2. периоды 24ч/7д/30д/90д
3. линейный график
4. объёмы

Это будет **версия 0.3**.

Четвёртая рабочая версия

Добавляем аналитику:

1. RSI
2. уровни 30/70
3. поддержка/сопротивление
4. свечной график

Это будет **версия 0.4**.

Пятая рабочая версия

Добавляем пользовательские действия:

1. оповещения
2. конвертор
3. сохранение настроек
4. финальная обработка ошибок

Это будет **версия 1.0**.

Что будет сложнее всего

1. Свечной график

Не потому что он сложный визуально, а потому что данные OHLC могут не совпасть с твоей заявленной дискретностью. Поэтому его делаем не первым.

2. API-лимиты

Нельзя обновлять каждую монету каждую секунду. Это глупо и может быстро упереться в ограничения. Нормально:

- автообновление раз в 60-120 секунд
- ручная кнопка `Обновить`

- историю загружать только для выбранной монеты

3. RSI

Сама формула несложная, но надо аккуратно обработать малое количество точек. Если точек меньше 14, RSI не считаем и показываем “недостаточно данных”.

4. Qt-интерфейс

Qt может быстро превратиться в кашу, если весь код писать в `MainWindow.cpp`. Поэтому сразу разделяем на виджеты и сервисы.

Как будем делать дальше

Начинать надо с **версии 0.1**: каркас приложения на Qt без API. Это самый безопасный старт.

Следующий шаг: я дам тебе полностью готовый набор файлов для первого запуска:

```
CMakeLists.txt
main.cpp
MainWindow.h
MainWindow.cpp
CoinInfo.h
```

После этого ты просто создашь проект, вставишь файлы, соберёшь и покажешь ошибку, если она будет. Я буду исправлять только через новые готовые версии файлов.

Sources:

- [Coins List with Market Data](#)
- [Crypto Data API: Most Comprehensive & Reliable ...](#)

Prompt:

24.05.2026, 15:53:37

```
CryptoTrack/
|
|— CMakeLists.txt
|
|— main.cpp
|
```

```
├── MainWindow.h
├── MainWindow.cpp
|
├── models/
|   ├── CoinInfo.h
|   ├── ChartPoint.h
|   ├── Candle.h
|   └── Alert.h
|
├── api/
|   ├── CoinGeckoApi.h
|   └── CoinGeckoApi.cpp
|
├── widgets/
|   ├── CoinListWidget.h
|   ├── CoinListWidget.cpp
|   ├── PriceChartWidget.h
|   ├── PriceChartWidget.cpp
|   ├── VolumeChartWidget.h
|   ├── VolumeChartWidget.cpp
|   ├── RsiChartWidget.h
|   ├── RsiChartWidget.cpp
|   ├── AlertPanelWidget.h
|   ├── AlertPanelWidget.cpp
|   ├── ConverterWidget.h
|   └── ConverterWidget.cpp
|
├── services/
|   ├── RsiCalculator.h
|   ├── RsiCalculator.cpp
|   ├── SupportResistanceCalculator.h
|   └── SupportResistanceCalculator.cpp
|
└── storage/
    ├── AppStorage.h
    └── AppStorage.cpp
```

я делаю через Qt Creator, на данный момент мой проект выглядит так: project, project.pro, Заголовочные, Исходники, Формы, (дальше 3 уровень) ai_project.h в загол, ai_project.cpp & main.cpp в исходниках, ai_project.ui в формах, давай сейчас начнем делать поэтапно, не создавая весь проект даже просто файлами, а вот прям кусочек за кусочком, + буду очень рад, если визуал приложения, само окно будет очень красивым современным, а не просто прямоугольная кнопка добавить и все

Response:

24.05.2026, 15:53:43

Начнём правильно: не трогаем API и графики, сначала делаем красивый рабочий каркас в твоей текущей структуре Qt Creator. Сейчас задача — чтобы проект собрался и выглядел как приложение CryptoTrack, пусть пока на фейковых данных.

Да, так лучше. **Пока не создаём новые папки и классы.** Работаем строго в твоей текущей структуре:

```
project
├─ project.pro
├─ Заголовочные
│   └─ ai_project.h
├─ Исходники
│   ├── ai_project.cpp
│   └─ main.cpp
└─ Формы
    └─ ai_project.ui
```

`ai_project.ui` пока **не трогаем**. Интерфейс сделаем кодом, потому что так проще получить современный вид и не мучиться с Qt Designer.

Сейчас делаем **версию 0.1**:

- красивое главное окно;
- левая панель с 7 монетами;
- центральная область с ценой, периодами, типом графика;
- фейковый график цены;
- объёмы;
- RSI;
- правая панель оповещений;
- конвертор;
- данные пока тестовые, без CoinGecko API.

Это соответствует твоему утверждённому описанию проекта: слева список монет, центр с графиками и анализом, справа оповещения и конвертор. `Вставленный текст.txt`

Шаг 1. Замени `project.pro`

Открой `project.pro`, выдели всё и вставь:

```
QT += core gui widgets
```



```
CONFIG += c++17

greaterThan(QT_MAJOR_VERSION, 4): QT += widgets

TARGET = CryptoTrack
TEMPLATE = app

SOURCES += \
    main.cpp \
    ai_project.cpp

HEADERS += \
    ai_project.h

FORMS += \
    ai_project.ui
```

Шаг 2. Замени ai_project.h

Открой ai_project.h , выдели всё и вставь:

```
#ifndef AI_PROJECT_H
#define AI_PROJECT_H

#include <QMainWindow>
#include <QVector>
#include <QString>

class QLabel;
class QListWidget;
class QPushButton;
class QComboBox;
class QLineEdit;
class QFrame;
class MiniChartWidget;

struct CoinUiData
{
    QString id;
    QString symbol;
    QString name;

    double price;
    double changePercent;
    double volume;

    QVector<double> history;
    QVector<double> volumes;
    QVector<double> rsi;
};
```

```

class ai_project : public QMainWindow
{
    Q_OBJECT

public:
    explicit ai_project(QWidget *parent = nullptr);
    ~ai_project();

private:
    QVector<CoinUiData> coins;

    QListWidget *coinList;

    QLabel *coinNameLabel;
    QLabel *coinPriceLabel;
    QLabel *coinChangeLabel;
    QLabel *periodInfoLabel;
    QLabel *lastUpdateLabel;

    MiniChartWidget *priceChart;
    MiniChartWidget *volumeChart;
    MiniChartWidget *rsiChart;

    QPushButton *period24Button;
    QPushButton *period7Button;
    QPushButton *period30Button;
    QPushButton *period90Button;

    QPushButton *lineButton;
    QPushButton *candleButton;
    QPushButton *refreshButton;

    QComboBox *alertCoinCombo;
    QComboBox *alertConditionCombo;
    QLineEdit *alertPriceEdit;
    QListWidget *alertsList;

    QComboBox *fromCoinCombo;
    QComboBox *toCoinCombo;
    QLineEdit *amountEdit;
    QLabel *converterResultLabel;

    QString selectedCoinId;
    QString selectedPeriod;
    bool candleMode;

private:
    void setupInterface();
    void setupStyle();
    void fillDemoData();
    void buildCoinList();

```

```
QFrame *createCard();
QPushButton *createSmallButton(const QString &text);

void selectCoin(int row);
void updateCenterPanel(const CoinUiData &coin);
void updateConverter();
void addAlert();
void refreshDemoData();

int findCoinIndexById(const QString &id) const;
QString formatPrice(double value) const;
QString formatVolume(double value) const;
};

#endif
```

Шаг 3. Замени main.cpp

Открой main.cpp , выдели всё и вставь:

```
#include "ai_project.h"

#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication app(argc, argv);

    ai_project window;
    window.show();

    return app.exec();
}
```

Шаг 4. Замени ai_project.cpp

Открой ai_project.cpp , выдели всё и вставь:

```
#include "ai_project.h"

#include <QApplication>
#include <QButtonGroup>
#include <QComboBox>
#include <QDateTime>
#include <QFrame>
#include <QHBoxLayout>
```

```

#include <QLabel>
#include <QLineEdit>
#include <QListWidget>
#include <QMessageBox>
#include <QPainter>
#include <QPushButton>
#include <QScrollBar>
#include <QVBoxLayout>
#include <QtMath>

class MiniChartWidget : public QWidget
{
public:
    enum ChartType
    {
        LineChart,
        CandleChart,
        BarChart,
        RsiChart
    };

    explicit MiniChartWidget(QWidget *parent = nullptr)
        : QWidget(parent),
          chartType(LineChart)
    {
        setMinimumHeight(150);
        setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Expanding);
    }

    void setChartType(ChartType type)
    {
        chartType = type;
        update();
    }

    void setValues(const QVector<double> &newValues)
    {
        values = newValues;
        update();
    }

protected:
    void paintEvent(QPaintEvent *) override
    {
        QPainter painter(this);
        painter.setRenderHint(QPainter::Antialiasing, true);

        QRect area = rect().adjusted(18, 18, -18, -18);

        painter.setPen(Qt::NoPen);
        painter.setBrush(QColor("#121a2e"));
        painter.drawRoundedRect(rect(), 18, 18);
    }
};

```

```

if (values.size() < 2)
{
    painter.setPen(QColor("#8f9bb3"));
    painter.drawText(area, Qt::AlignCenter, "Недостаточно данных");
    return;
}

double minValue = values[0];
double maxValue = values[0];

for (double value : values)
{
    minValue = qMin(minValue, value);
    maxValue = qMax(maxValue, value);
}

if (qFuzzyCompare(minValue, maxValue))
{
    maxValue += 1.0;
    minValue -= 1.0;
}

drawGrid(painter, area, minValue, maxValue);

if (chartType == BarChart)
    drawBars(painter, area, minValue, maxValue);
else if (chartType == CandleChart)
    drawCandles(painter, area, minValue, maxValue);
else if (chartType == RsiChart)
    drawRsi(painter, area);
else
    drawLine(painter, area, minValue, maxValue);
}

private:
    QVector<double> values;
    ChartType chartType;

void drawGrid(QPainter &painter, const QRect &area, double minValue, double maxValue)
{
    Q_UNUSED(minValue);
    Q_UNUSED(maxValue);

    painter.setPen(QPen(QColor("#26314f"), 1));

    for (int i = 0; i <= 4; ++i)
    {
        int y = area.top() + area.height() * i / 4;
        painter.drawLine(area.left(), y, area.right(), y);
    }

    for (int i = 0; i <= 6; ++i)
    {

```

```

        int x = area.left() + area.width() * i / 6;
        painter.drawLine(x, area.top(), x, area.bottom());
    }
}

QPointF pointAt(int index, const QRect &area, double minValue, double maxValue)
{
    double x = area.left() + static_cast<double>(area.width()) * index / (values.size() - 1);
    double normalized = (values[index] - minValue) / (maxValue - minValue);
    double y = area.bottom() - normalized * area.height();

    return QPointF(x, y);
}

void drawLine(QPainter &painter, const QRect &area, double minValue, double maxValue)
{
    QPainterPath path;
    path.moveTo(pointAt(0, area, minValue, maxValue));

    for (int i = 1; i < values.size(); ++i)
        path.lineTo(pointAt(i, area, minValue, maxValue));

    QLinearGradient gradient(area.topLeft(), area.bottomLeft());
    gradient.setColorAt(0.0, QColor(34, 197, 94, 90));
    gradient.setColorAt(1.0, QColor(34, 197, 94, 0));

    QPainterPath fillPath = path;
    fillPath.lineTo(area.right(), area.bottom());
    fillPath.lineTo(area.left(), area.bottom());
    fillPath.closeSubpath();

    painter.fillPath(fillPath, gradient);

    painter.setPen(QPen(QColor("#22c55e"), 3));
    painter.drawPath(path);
}

void drawBars(QPainter &painter, const QRect &area, double minValue, double maxValue)
{
    Q_UNUSED(minValue);

    int count = values.size();
    double step = static_cast<double>(area.width()) / count;
    double barWidth = qMax(3.0, step * 0.55);

    painter.setPen(Qt::NoPen);
    painter.setBrush(QColor("#38bdf8"));

    for (int i = 0; i < count; ++i)
    {
        double normalized = values[i] / maxValue;
        double barHeight = normalized * area.height();
    }
}

```

```

        QRectF bar(
            area.left() + i * step + (step - barWidth) / 2.0,
            area.bottom() - barHeight,
            barWidth,
            barHeight
        );

        painter.drawRoundedRect(bar, 3, 3);
    }
}

void drawCandles(QPainter &painter, const QRect &area, double minValue, double maxValue)
{
    int count = values.size();
    double step = static_cast<double>(area.width()) / count;
    double candleWidth = qMax(4.0, step * 0.48);

    for (int i = 1; i < count; ++i)
    {
        double open = values[i - 1];
        double close = values[i];

        double high = qMax(open, close) * 1.004;
        double low = qMin(open, close) * 0.996;

        auto mapY = [&](double value)
        {
            double normalized = (value - minValue) / (maxValue - minValue);
            return area.bottom() - normalized * area.height();
        };

        double x = area.left() + i * step;
        QColor color = close >= open ? QColor("#22c55e") : QColor("#ef4444");

        painter.setPen(QPen(color, 2));
        painter.drawLine(QPointF(x, mapY(high)), QPointF(x, mapY(low)));

        QRectF body(
            x - candleWidth / 2.0,
            qMin(mapY(open), mapY(close)),
            candleWidth,
            qAbs(mapY(close) - mapY(open))
        );

        if (body.height() < 3)
            body.setHeight(3);

        painter.setPen(Qt::NoPen);
        painter.setBrush(color);
        painter.drawRoundedRect(body, 3, 3);
    }
}

```

```

void drawRsi(QPainter &painter, const QRect &area)
{
    painter.setPen(QPen(QColor("#ef4444"), 1, Qt::DashLine));
    int level70 = area.bottom() - area.height() * 70 / 100;
    painter.drawLine(area.left(), level70, area.right(), level70);

    painter.setPen(QPen(QColor("#22c55e"), 1, Qt::DashLine));
    int level30 = area.bottom() - area.height() * 30 / 100;
    painter.drawLine(area.left(), level30, area.right(), level30);

    painter.setPen(QColor("#8f9bb3"));
    painter.drawText(area.left() + 4, level70 - 4, "70");
    painter.drawText(area.left() + 4, level30 - 4, "30");

    QPainterPath path;

    auto rsiPoint = [&](int index)
    {
        double x = area.left() + static_cast<double>(area.width()) * index / (values.size() - 1);
        double y = area.bottom() - values[index] / 100.0 * area.height();
        return QPointF(x, y);
    };

    path.moveTo(rsiPoint(0));

    for (int i = 1; i < values.size(); ++i)
        path.lineTo(rsiPoint(i));

    painter.setPen(QPen(QColor("#a855f7"), 3));
    painter.drawPath(path);
}

};

ai_project::ai_project(QWidget *parent)
: QMainWindow(parent),
  coinList(nullptr),
  coinNameLabel(nullptr),
  coinPriceLabel(nullptr),
  coinChangeLabel(nullptr),
  periodInfoLabel(nullptr),
  lastUpdateLabel(nullptr),
  priceChart(nullptr),
  volumeChart(nullptr),
  rsiChart(nullptr),
  period24Button(nullptr),
  period7Button(nullptr),
  period30Button(nullptr),
  period90Button(nullptr),
  lineButton(nullptr),
  candleButton(nullptr),
  refreshButton(nullptr),
  alertCoinCombo(nullptr),
  alertConditionCombo(nullptr),

```



```

        alertPriceEdit(nullptr),
        alertsList(nullptr),
        fromCoinCombo(nullptr),
        toCoinCombo(nullptr),
        amountEdit(nullptr),
        converterResultLabel(nullptr),
        selectedPeriod("24ч"),
        candleMode(false)
{
    setWindowTitle("CryptoTrack");
    resize(1320, 820);

    fillDemoData();
    setupInterface();
    setupStyle();

    buildCoinList();

    if (!coins.isEmpty())
    {
        coinList->setCurrentRow(0);
        selectCoin(0);
    }
}

ai_project::~ai_project()
{
}

void ai_project::setupInterface()
{
    QWidget *central = new QWidget(this);
    setCentralWidget(central);

    QHBoxLayout *rootLayout = new QHBoxLayout(central);
    rootLayout->setContentsMargins(18, 18, 18, 18);
    rootLayout->setSpacing(16);

    QFrame *leftPanel = createCard();
    leftPanel->setFixedWidth(270);

    QVBoxLayout *leftLayout = new QVBoxLayout(leftPanel);
    leftLayout->setContentsMargins(16, 16, 16, 16);
    leftLayout->setSpacing(12);

    QLabel *appTitle = new QLabel("CryptoTrack");
    appTitle->setObjectName("AppTitle");

    QLabel *appSubtitle = new QLabel("Курсы и аналитика криптовалют");
    appSubtitle->setObjectName("MutedText");

    coinList = new QListWidget();
    coinList->setObjectName("CoinList");

```

```
coinList->setSpacing(8);
coinList->setHorizontalScrollBarPolicy(Qt::ScrollBarAlwaysOff);

leftLayout->addWidget(appTitle);
leftLayout->addWidget(appSubtitle);
leftLayout->addSpacing(10);
leftLayout->addWidget(coinList);

QFrame *centerPanel = createCard();

QVBoxLayout *centerLayout = new QVBoxLayout(centerPanel);
centerLayout->setContentsMargins(22, 22, 22, 22);
centerLayout->setSpacing(14);

QHBoxLayout *coinHeaderLayout = new QHBoxLayout();

QVBoxLayout *coinTextLayout = new QVBoxLayout();
coinTextLayout->setSpacing(4);

coinNameLabel = new QLabel("Bitcoin");
coinNameLabel->setObjectName("CoinName");

coinPriceLabel = new QLabel("$0");
coinPriceLabel->setObjectName("CoinPrice");

coinChangeLabel = new QLabel("+0.00%");
coinChangeLabel->setObjectName("PositiveText");

coinTextLayout->addWidget(coinNameLabel);
coinTextLayout->addWidget(coinPriceLabel);
coinTextLayout->addWidget(coinChangeLabel);

QVBoxLayout *updateLayout = new QVBoxLayout();
updateLayout->setAlignment(Qt::AlignRight | Qt::AlignTop);

lastUpdateLabel = new QLabel("Последнее обновление: -");
lastUpdateLabel->setObjectName("MutedText");

refreshButton = createSmallButton("Обновить");

updateLayout->addWidget(lastUpdateLabel);
updateLayout->addWidget(refreshButton, 0, Qt::AlignRight);

coinHeaderLayout->addLayout(coinTextLayout);
coinHeaderLayout->addStretch();
coinHeaderLayout->addLayout(updateLayout);

QHBoxLayout *controlLayout = new QHBoxLayout();
controlLayout->setSpacing(10);

period24Button = createSmallButton("24ч");
period7Button = createSmallButton("7д");
period30Button = createSmallButton("30д");
```

```
period90Button = createSmallButton("90д");

period24Button->setCheckable(true);
period7Button->setCheckable(true);
period30Button->setCheckable(true);
period90Button->setCheckable(true);
period24Button->setChecked(true);

QButtonGroup *periodGroup = new QButtonGroup(this);
periodGroup->addButton(period24Button);
periodGroup->addButton(period7Button);
periodGroup->addButton(period30Button);
periodGroup->addButton(period90Button);
periodGroup->setExclusive(true);

lineButton = createSmallButton("Линия");
candleButton = createSmallButton("Свечи");

lineButton->setCheckable(true);
candleButton->setCheckable(true);
lineButton->setChecked(true);

QButtonGroup *chartGroup = new QButtonGroup(this);
chartGroup->addButton(lineButton);
chartGroup->addButton(candleButton);
chartGroup->setExclusive(true);

periodInfoLabel = new QLabel("Период: 24 часа, шаг: 1 час");
periodInfoLabel->setObjectName("MutedText");

controlLayout->addWidget(period24Button);
controlLayout->addWidget(period7Button);
controlLayout->addWidget(period30Button);
controlLayout->addWidget(period90Button);
controlLayout->addSpacing(20);
controlLayout->addWidget(lineButton);
controlLayout->addWidget(candleButton);
controlLayout->addStretch();
controlLayout->addWidget(periodInfoLabel);

QLabel *mainChartTitle = new QLabel("График цены");
mainChartTitle->setObjectName("BlockTitle");

priceChart = new MiniChartWidget();
priceChart->setMinimumHeight(280);

QHBoxLayout *smallChartsLayout = new QHBoxLayout();
smallChartsLayout->setSpacing(14);

QFrame *volumeCard = createCard();
QVBoxLayout *volumeLayout = new QVBoxLayout(volumeCard);
volumeLayout->setContentsMargins(14, 14, 14, 14);
```

```
QLabel *volumeTitle = new QLabel("Объём торгов");
volumeTitle->setObjectName("BlockTitle");

volumeChart = new MiniChartWidget();
volumeChart->setChartType(MiniChartWidget::BarChart);
volumeChart->setMinimumHeight(160);

volumeLayout->addWidget(volumeTitle);
volumeLayout->addWidget(volumeChart);

QFrame *rsiCard = createCard();
QVBoxLayout *rsiLayout = new QVBoxLayout(rsiCard);
rsiLayout->setContentsMargins(14, 14, 14, 14);

QLabel *rsiTitle = new QLabel("RSI");
rsiTitle->setObjectName("BlockTitle");

rsiChart = new MiniChartWidget();
rsiChart->setChartType(MiniChartWidget::RsiChart);
rsiChart->setMinimumHeight(160);

rsiLayout->addWidget(rsiTitle);
rsiLayout->addWidget(rsiChart);

smallChartsLayout->addWidget(volumeCard);
smallChartsLayout->addWidget(rsiCard);

centerLayout->addLayout(coinHeaderLayout);
centerLayout->addLayout(controlsLayout);
centerLayout->addWidget(mainChartTitle);
centerLayout->addWidget(priceChart, 1);
centerLayout->addLayout(smallChartsLayout);

QFrame *rightPanel = createCard();
rightPanel->setFixedWidth(310);

QVBoxLayout *rightLayout = new QVBoxLayout(rightPanel);
rightLayout->setContentsMargins(16, 16, 16, 16);
rightLayout->setSpacing(12);

QLabel *alertsTitle = new QLabel("Оповещения");
alertsTitle->setObjectName("BlockTitle");

alertCoinCombo = new QComboBox();
alertConditionCombo = new QComboBox();
alertPriceEdit = new QLineEdit();
QPushButton *addAlertButton = createSmallButton("Добавить оповещение");

alertConditionCombo->addItem("Цена выше");
alertConditionCombo->addItem("Цена ниже");
alertPriceEdit->setPlaceholderText("Целевая цена, USD");

alertsList = new QListWidget();
```

```
alertsList->setObjectName("SimpleList");
alertsList->setMinimumHeight(125);

QLabel *converterTitle = new QLabel("Конвертор");
converterTitle->setObjectName("BlockTitle");

fromCoinCombo = new QComboBox();
toCoinCombo = new QComboBox();
amountEdit = new QLineEdit();
QPushButton *calculateButton = createSmallButton("Рассчитать");

amountEdit->setPlaceholderText("Количество");
amountEdit->setText("1");

converterResultLabel = new QLabel("Результат: -");
converterResultLabel->setObjectName("ConverterResult");
converterResultLabel->setWordWrap(true);

rightLayout->addWidget(alertsTitle);
rightLayout->addWidget(alertCoinCombo);
rightLayout->addWidget(alertConditionCombo);
rightLayout->addWidget(alertPriceEdit);
rightLayout->addWidget(addAlertButton);
rightLayout->addWidget(alertsList);
rightLayout->addSpacing(14);
rightLayout->addWidget(converterTitle);
rightLayout->addWidget(fromCoinCombo);
rightLayout->addWidget(toCoinCombo);
rightLayout->addWidget(amountEdit);
rightLayout->addWidget(calculateButton);
rightLayout->addWidget(converterResultLabel);
rightLayout->addStretch();

rootLayout->addWidget(leftPanel);
rootLayout->addWidget(centerPanel, 1);
rootLayout->addWidget(rightPanel);

connect(coinList, &QListWidget::currentRowChanged, this, &ai_project::selectCoin);

connect(refreshButton, &QPushButton::clicked, this, &ai_project::refreshDemoData);

connect(period24Button, &QPushButton::clicked, this, [this]()
{
    selectedPeriod = "24ч";
    periodInfoLabel->setText("Период: 24 часа, шаг: 1 час");
    selectCoin(coinList->currentRow());
});

connect(period7Button, &QPushButton::clicked, this, [this]()
{
    selectedPeriod = "7д";
    periodInfoLabel->setText("Период: 7 дней, шаг: 4 часа");
    selectCoin(coinList->currentRow());
});
```

```

});

connect(period30Button, &QPushButton::clicked, this, [this]()
{
    selectedPeriod = "30д";
    periodInfoLabel->setText("Период: 30 дней, шаг: 1 день");
    selectCoin(coinList->currentRow());
});

connect(period90Button, &QPushButton::clicked, this, [this]()
{
    selectedPeriod = "90д";
    periodInfoLabel->setText("Период: 90 дней, шаг: 1 день");
    selectCoin(coinList->currentRow());
});

connect(lineButton, &QPushButton::clicked, this, [this]()
{
    candleMode = false;
    priceChart->setChartType(MiniChartWidget::LineChart);
});

connect(candleButton, &QPushButton::clicked, this, [this]()
{
    candleMode = true;
    priceChart->setChartType(MiniChartWidget::CandleChart);
});

connect(addAlertButton, &QPushButton::clicked, this, &ai_project::addAlert);
connect(calculateButton, &QPushButton::clicked, this, &ai_project::updateConverter);

connect(fromCoinCombo, &QComboBox::currentTextChanged, this, &ai_project::updateConverter);
connect(toCoinCombo, &QComboBox::currentTextChanged, this, &ai_project::updateConverter);
connect(amountEdit, &QLineEdit::textChanged, this, &ai_project::updateConverter);
}

void ai_project::setupStyle()
{
    qApp->setStyleSheet(
        "QMainWindow {"
        "    background: #070b16;"
        "}"

        "QWidget {"
        "    font-family: Segoe UI, Arial;"
        "    color: #eef2ff;"
        "    font-size: 14px;"
        "}"

        "QFrame#Card {"
        "    background: #0f172a;"
        "    border: 1px solid #1e293b;"
        "    border-radius: 22px;"
    );
}

```

```
"}"
```

```
"QLabel#AppTitle {"  
"    font-size: 28px;"  
"    font-weight: 800;"  
"    color: #ffffff;"  
"}"
```

```
"QLabel#CoinName {"  
"    font-size: 24px;"  
"    font-weight: 700;"  
"    color: #ffffff;"  
"}"
```

```
"QLabel#CoinPrice {"  
"    font-size: 42px;"  
"    font-weight: 800;"  
"    color: #ffffff;"  
"}"
```

```
"QLabel#BlockTitle {"  
"    font-size: 17px;"  
"    font-weight: 700;"  
"    color: #f8fafc;"  
"}"
```

```
"QLabel#MutedText {"  
"    color: #94a3b8;"  
"    font-size: 13px;"  
"}"
```

```
"QLabel#PositiveText {"  
"    color: #22c55e;"  
"    font-size: 16px;"  
"    font-weight: 700;"  
"}"
```

```
"QLabel#NegativeText {"  
"    color: #ef4444;"  
"    font-size: 16px;"  
"    font-weight: 700;"  
"}"
```

```
"QLabel#ConverterResult {"  
"    background: #111c33;"  
"    border: 1px solid #27344f;"  
"    border-radius: 14px;"  
"    padding: 12px;"  
"    color: #dbeafe;"  
"    font-weight: 700;"  
"}"
```

```
"QPushButton {"
```

```
"    background: #1e293b;"
"    color: #e5e7eb;"
"    border: 1px solid #334155;"
"    border-radius: 13px;"
"    padding: 9px 14px;"
"    font-weight: 600;"
"}"

"QPushButton:hover {"
"    background: #273449;"
"    border: 1px solid #475569;"
"}"

"QPushButton:checked {"
"    background: #2563eb;"
"    color: white;"
"    border: 1px solid #3b82f6;"
"}"

"QListWidget#CoinList, QListWidget#SimpleList {"
"    background: transparent;"
"    border: none;"
"    outline: none;"
"}"

"QListWidget::item {"
"    border-radius: 16px;"
"    margin: 3px;"
"    padding: 4px;"
"}"

"QListWidget::item:selected {"
"    background: #1d4ed8;"
"}"

"QComboBox, QLineEdit {"
"    background: #111827;"
"    border: 1px solid #334155;"
"    border-radius: 13px;"
"    padding: 9px 11px;"
"    color: #f8fafc;"
"}"

"QComboBox:hover, QLineEdit:hover {"
"    border: 1px solid #475569;"
"}"

"QComboBox::drop-down {"
"    border: none;"
"    width: 28px;"
"}"

"QScrollBar:vertical {"
```



```

"    background: transparent;"
"    width: 8px;"
"}"

"QScrollBar::handle:vertical {"
"    background: #334155;"
"    border-radius: 4px;"
"}"

"QScrollBar::add-line:vertical, QScrollBar::sub-line:vertical {"
"    height: 0px;"
"}"

);
}

void ai_project::fillDemoData()
{
    coins.clear();

    struct BaseCoin
    {
        QString id;
        QString symbol;
        QString name;
        double price;
        double change;
        double volume;
    };

    QVector<BaseCoin> baseCoins = {
        {"bitcoin", "BTC", "Bitcoin", 68420.0, 2.41, 28500000000.0},
        {"ethereum", "ETH", "Ethereum", 3570.0, 1.76, 15900000000.0},
        {"solana", "SOL", "Solana", 168.4, -0.82, 4200000000.0},
        {"dogecoin", "DOGE", "Dogecoin", 0.147, 4.15, 2100000000.0},
        {"cardano", "ADA", "Cardano", 0.46, -1.24, 860000000.0},
        {"ripple", "XRP", "Ripple", 0.53, 0.64, 1300000000.0},
        {"polkadot", "DOT", "Polkadot", 7.12, -2.35, 510000000.0}
    };

    for (int coinIndex = 0; coinIndex < baseCoins.size(); ++coinIndex)
    {
        const BaseCoin &base = baseCoins[coinIndex];

        CoinUiData coin;
        coin.id = base.id;
        coin.symbol = base.symbol;
        coin.name = base.name;
        coin.price = base.price;
        coin.changePercent = base.change;
        coin.volume = base.volume;

        double start = base.price * (1.0 - base.change / 100.0);

```

```

for (int i = 0; i < 60; ++i)
{
    double t = static_cast<double>(i) / 59.0;
    double wave = qSin(i * 0.35 + coinIndex) * base.price * 0.018;
    double trend = (base.price - start) * t;
    double localNoise = qSin(i * 0.91 + coinIndex * 2.0) * base.price * 0.006;

    coin.history.append(start + trend + wave + localNoise);

    double volumeWave = qAbs(qSin(i * 0.27 + coinIndex));
    coin.volumes.append(base.volume * (0.45 + volumeWave * 0.55));

    double rsiValue = 50.0 + qSin(i * 0.22 + coinIndex) * 22.0 + base.change * 1.5;
    rsiValue = qBound(12.0, rsiValue, 88.0);
    coin.rsi.append(rsiValue);
}

coins.append(coin);
}
}

void ai_project::buildCoinList()
{
    coinList->clear();

    alertCoinCombo->clear();
    fromCoinCombo->clear();
    toCoinCombo->clear();

    for (const CoinUiData &coin : coins)
    {
        QListWidgetItem *item = new QListWidgetItem();
        item->setData(Qt::UserRole, coin.id);
        item->setSizeHint(QSize(220, 72));

        QWidget *rowWidget = new QWidget();
        QHBoxLayout *rowLayout = new QHBoxLayout(rowWidget);
        rowLayout->setContentsMargins(10, 8, 10, 8);
        rowLayout->setSpacing(8);

        QLabel *symbolLabel = new QLabel(coin.symbol);
        symbolLabel->setFixedWidth(48);
        symbolLabel->setAlignment(Qt::AlignCenter);
        symbolLabel->setStyleSheet(
            "background:#172554;"
            "border-radius:14px;"
            "padding:10px 6px;"
            "font-weight:800;"
            "color:#dbeafe;"
        );

        QVBoxLayout *textLayout = new QVBoxLayout();
        textLayout->setSpacing(2);

```

```

QLabel *nameLabel = new QLabel(coin.name);
nameLabel->setStyleSheet("font-weight:700; color:#f8fafc;");

QLabel *priceLabel = new QLabel(formatPrice(coin.price));
priceLabel->setStyleSheet("color:#94a3b8; font-size:12px;");

textLayout->addWidget(nameLabel);
textLayout->addWidget(priceLabel);

QLabel *changeLabel = new QLabel(QString("%1%2%")
                                   .arg(coin.changePercent >= 0 ? "▲ +" : "▼ ")
                                   .arg(coin.changePercent, 0, 'f', 2));

changeLabel->setAlignment(Qt::AlignRight | Qt::AlignVCenter);
changeLabel->setStyleSheet(coin.changePercent >= 0
                           ? "color:#22c55e; font-weight:800;"
                           : "color:#ef4444; font-weight:800;");

rowLayout->addWidget(symbolLabel);
rowLayout->addLayout(textLayout);
rowLayout->addStretch();
rowLayout->addWidget(changeLabel);

coinList->addItem(item);
coinList->setItemWidget(item, rowWidget);

alertCoinCombo->addItem(coin.symbol, coin.id);
fromCoinCombo->addItem(coin.symbol, coin.id);
toCoinCombo->addItem(coin.symbol, coin.id);
}

if (toCoinCombo->count() > 1)
    toCoinCombo->setCurrentIndex(1);
}

QFrame *ai_project::createCard()
{
    QFrame *frame = new QFrame();
    frame->setObjectName("Card");
    return frame;
}

QPushButton *ai_project::createSmallButton(const QString &text)
{
    QPushButton *button = new QPushButton(text);
    button->setCursor(Qt::PointingHandCursor);
    return button;
}

void ai_project::selectCoin(int row)
{
    if (row < 0 || row >= coins.size())

```

```
return;
```

```
selectedCoinId = coins[row].id;  
updateCenterPanel(coins[row]);  
updateConverter();  
}
```

```
void ai_project::updateCenterPanel(const CoinUiData &coin)
```

```
{  
    coinNameLabel->setText(coin.name + " (" + coin.symbol + ")");  
    coinPriceLabel->setText(formatPrice(coin.price));  
  
    QString changeText = QString("%1%2% за 24 часа")  
        .arg(coin.changePercent >= 0 ? "▲ +" : "▼ ")  
        .arg(coin.changePercent, 0, 'f', 2);  
  
    coinChangeLabel->setText(changeText);  
    coinChangeLabel->setObjectName(coin.changePercent >= 0 ? "PositiveText" : "NegativeText");  
    coinChangeLabel->style()->unpolish(coinChangeLabel);  
    coinChangeLabel->style()->polish(coinChangeLabel);  
  
    QVector<double> chartValues = coin.history;  
    QVector<double> volumeValues = coin.volumes;  
    QVector<double> rsiValues = coin.rsi;  
  
    if (selectedPeriod == "24ч")  
    {  
        chartValues = chartValues.mid(chartValues.size() - 24);  
        volumeValues = volumeValues.mid(volumeValues.size() - 24);  
        rsiValues = rsiValues.mid(rsiValues.size() - 24);  
    }  
    else if (selectedPeriod == "7д")  
    {  
        chartValues = chartValues.mid(chartValues.size() - 42);  
        volumeValues = volumeValues.mid(volumeValues.size() - 42);  
        rsiValues = rsiValues.mid(rsiValues.size() - 42);  
    }  
    else if (selectedPeriod == "30д")  
    {  
        chartValues = chartValues.mid(chartValues.size() - 30);  
        volumeValues = volumeValues.mid(volumeValues.size() - 30);  
        rsiValues = rsiValues.mid(rsiValues.size() - 30);  
    }  
    else  
    {  
        chartValues = coin.history;  
        volumeValues = coin.volumes;  
        rsiValues = coin.rsi;  
    }  
  
    priceChart->setValues(chartValues);  
    volumeChart->setValues(volumeValues);  
    rsiChart->setValues(rsiValues);  
}
```

```

if (candleMode)
    priceChart->setChartType(MiniChartWidget::CandleChart);
else
    priceChart->setChartType(MiniChartWidget::LineChart);

lastUpdateLabel->setText("Последнее обновление: " +
                        QDateTime::currentDateTime().toString("dd.MM.yyyy HH:mm:ss"));
}

void ai_project::updateConverter()
{
    if (!fromCoinCombo || !toCoinCombo || !amountEdit || !converterResultLabel)
        return;

    QString fromId = fromCoinCombo->currentData().toString();
    QString toId = toCoinCombo->currentData().toString();

    int fromIndex = findCoinIndexById(fromId);
    int toIndex = findCoinIndexById(toId);

    if (fromIndex < 0 || toIndex < 0)
        return;

    bool ok = false;
    double amount = amountEdit->text().replace(",", ".").toDouble(&ok);

    if (!ok || amount <= 0.0)
    {
        converterResultLabel->setText("Результат: введите корректное количество");
        return;
    }

    double result = amount * coins[fromIndex].price / coins[toIndex].price;

    converterResultLabel->setText(
        QString("Результат: %1 %2 = %3 %4")
            .arg(amount, 0, 'f', 4)
            .arg(coins[fromIndex].symbol)
            .arg(result, 0, 'f', 6)
            .arg(coins[toIndex].symbol)
    );
}

void ai_project::addAlert()
{
    QString coinSymbol = alertCoinCombo->currentText();
    QString condition = alertConditionCombo->currentText();

    bool ok = false;
    double targetPrice = alertPriceEdit->text().replace(",", ".").toDouble(&ok);

    if (!ok || targetPrice <= 0.0)

```

```

{
    QMessageBox::warning(this, "Ошибка", "Введите корректную целевую цену.");
    return;
}

QString text = QString("%1: %2 %3 USD")
                .arg(coinSymbol)
                .arg(condition)
                .arg(targetPrice, 0, 'f', 2);

alertsList->addItem(text);
alertPriceEdit->clear();
}

void ai_project::refreshDemoData()
{
    for (int i = 0; i < coins.size(); ++i)
    {
        double direction = qSin(QDateTime::currentMSecsSinceEpoch() / 7000.0 + i);

        coins[i].price *= (1.0 + direction * 0.0015);
        coins[i].changePercent += direction * 0.08;

        coins[i].history.removeFirst();
        coins[i].history.append(coins[i].price);

        coins[i].volumes.removeFirst();
        coins[i].volumes.append(coins[i].volume * (0.6 + qAbs(direction) * 0.5));

        coins[i].rsi.removeFirst();
        double newRsi = qBound(10.0, coins[i].rsi.last() + direction * 2.0, 90.0);
        coins[i].rsi.append(newRsi);
    }

    buildCoinList();

    int selectedIndex = findCoinIndexById(selectedCoinId);
    if (selectedIndex < 0)
        selectedIndex = 0;

    coinList->setCurrentRow(selectedIndex);
    selectCoin(selectedIndex);
}

int ai_project::findCoinIndexById(const QString &id) const
{
    for (int i = 0; i < coins.size(); ++i)
    {
        if (coins[i].id == id)
            return i;
    }

    return -1;
}

```

```
}

QString ai_project::formatPrice(double value) const
{
    if (value >= 1000.0)
        return "$" + QString::number(value, 'f', 0);

    if (value >= 1.0)
        return "$" + QString::number(value, 'f', 2);

    return "$" + QString::number(value, 'f', 4);
}

QString ai_project::formatVolume(double value) const
{
    if (value >= 10000000000.0)
        return QString::number(value / 10000000000.0, 'f', 2) + " млрд USD";

    if (value >= 1000000.0)
        return QString::number(value / 1000000.0, 'f', 2) + " млн USD";

    return QString::number(value, 'f', 2) + " USD";
}
```

Шаг 5. Что нажать в Qt Creator

После замены файлов:

1. Сохрани все файлы.
2. В Qt Creator сверху выбери:

Сборка → Запустить qmake

3. Потом:

Сборка → Пересобрать проект

4. Потом запускай зелёной кнопкой.

Что должно получиться

Откроется тёмное современное окно **CryptoTrack**:

- слева список 7 криптовалют;
- по центру цена и график;

- можно переключать монеты;
- можно переключать периоды;
- можно переключать `Линия / Свечи`;
- справа работает добавление оповещений;
- справа работает конвертор;
- кнопка `Обновить` немного меняет демонстрационные данные.

Это пока **не финальный проект**, а правильный первый слой. Следующий этап после проверки - подключение реального CoinGecko API.

Prompt:

24.05.2026, 15:57:34

C:\Users\bakki\Documents\project\ai_project.cpp:131: ошибка: aggregate 'QPainterPath path' has incomplete type and cannot be defined\ai_project.cpp: In member function 'void MiniChartWidget::drawLine(QPainter&, const QRect&, double, double)':\ai_project.cpp:131:22: error: aggregate 'QPainterPath path' has incomplete type and cannot be defined 131 | QPainterPath path; |
 ^~~~C:\Users\bakki\Documents\project\ai_project.cpp:141: ошибка: variable 'QPainterPath fillPath' has initializer but incomplete type\ai_project.cpp:141:22: error: variable 'QPainterPath fillPath' has initializer but incomplete type 141 | QPainterPath fillPath = path; |
 ^~~~~C:\Users\bakki\Documents\project\ai_project.cpp:235: ошибка: aggregate 'QPainterPath path' has incomplete type and cannot be defined\ai_project.cpp: In member function 'void MiniChartWidget::drawRsi(QPainter&, const QRect&):'\ai_project.cpp:235:22: error: aggregate 'QPainterPath path' has incomplete type and cannot be defined 235 | QPainterPath path; | ^~:-1:
 ошибка: [Makefile.Debug:149: debug/ai_project.o] Error 1 пока такие ошибки при сборке проекта

Response:

24.05.2026, 15:57:38

Ошибка простая: я использовал `QPainterPath`, но не подключил его заголовочный файл.

В `ai_project.cpp` в верхнем блоке `#include` добавь строку:

```
#include <QPainterPath>
```

Чтобы ты не правил вручную по одной строке, замени **весь блок include в начале ai_project.cpp** НА ЭТОТ:

```
#include "ai_project.h"

#include <QApplication>
#include <QButtonGroup>
#include <QComboBox>
```



```
#include <QDateTime>
#include <QFrame>
#include <QHBoxLayout>
#include <QLabel>
#include <QLineEdit>
#include <QListWidget>
#include <QMessageBox>
#include <QPainter>
#include <QPainterPath>
#include <QPushButton>
#include <QScrollBar>
#include <QVBoxLayout>
#include <QtMath>
```

После этого:

```
Сборка → Запустить qmake
Сборка → Пересобрать проект
```

Это не логическая ошибка проекта, просто не хватало подключения класса `QPainterPath`.
Дальше сборка должна пройти или покажет уже следующую конкретную ошибку.

Prompt:

24.05.2026, 15:59:02

```
C:\msys64\mingw64\lib\libQt6EntryPoint.a(qtentrypoint_win.cpp.obj):-1: ошибка: undefined reference to
qMain(int, char**) 'C:\msys64\mingw64\lib\libQt6EntryPoint.a(qtentrypoint_win.cpp.obj):-1:
ошибка: undefined reference to qMain(int, char**)':-1: ошибка: collect2.exe: error: ld returned 1 exit
status:-1: ошибка: [Makefile.Debug:72: debug/CryptoTrack.exe] Error 1 еще такие ошибки при сборке
```

Response:

24.05.2026, 15:59:06

Диагностика ошибки `undefined reference qMain`

Проблема скорее всего в том, что файл `main.cpp` не находится в списке `SOURCES` в `.pro`, либо путь к нему неверный. Qt Creator использует макрос для связывания `main` с `qMain`, и ошибка говорит, что компиляция не может найти функцию `main`. Нужно проверить файлы и пути.

Проверка проблемы с `qMain` и `main.cpp`

Необходимо проверить, что `main.cpp` включен в проект и правильно компилируется. Также стоит убедиться, что добавлен `CONFIG += console`, а путь к файлу в проекте

указан верно. Если `main` не компилируется, не будет и `qMain`. Рекомендуется пересобрать проект после изменений `.pro`. Цитировать файл, скорее всего, не нужно.

Думал на протяжении 8s

Это уже **не ошибка графики**, а ошибка линковки:

```
undefined reference to `qMain(int, char**)'
```

Значит Qt **не видит файл `main.cpp` в сборке** или в `main.cpp` нет правильной функции входа.

Скорее всего, у тебя `project.pro` не совпал с реальными файлами или Qt Creator не перечитал `.pro`.

1. Полностью замени `main.cpp`

Открой `main.cpp`, выдели всё и вставь:

```
#include "ai_project.h"

#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication app(argc, argv);

    ai_project window;
    window.show();

    return app.exec();
}
```

Важно: функция должна называться именно:

```
int main(int argc, char *argv[])
```

Не `qMain`, не `Main`, не `WinMain`.

2. Полностью замени `project.pro`

Открой `project.pro`, выдели всё и вставь:

```
QT += core gui widgets

CONFIG += c++17
```

```
greaterThan(QT_MAJOR_VERSION, 4): QT += widgets
```

```
TARGET = CryptoTrack
```

```
TEMPLATE = app
```

```
SOURCES += \
```

```
    main.cpp \
```

```
    ai_project.cpp
```

```
HEADERS += \
```

```
    ai_project.h
```

```
FORMS += \
```

```
    ai_project.ui
```

3. Обязательно пересоздай Makefile

После изменения `.pro` просто нажать “собрать” недостаточно.

Сделай строго так:

Сборка → Очистить проект

Сборка → Запустить qmake

Сборка → Пересобрать проект

4. Если ошибка останется

Тогда проблема почти точно в том, что Qt Creator **не видит** `main.cpp` как файл проекта.

Проверь в дереве проекта:

Исходники

└─ ai_project.cpp

└─ main.cpp

Если `main.cpp` там нет, сделай через Qt Creator:

Правый клик по проекту → Добавить существующие файлы...

И выбери:

main.cpp

После этого снова:

Сборка → Запустить qmake

Сборка → Пересобрать проект

Суть ошибки: `ai_project.cpp` уже компилируется, но программа не находит точку входа. То есть сейчас лечим не интерфейс, а подключение `main.cpp` к сборке.

Prompt:

24.05.2026, 16:01:48

запустил, вылезло такое, все собралось

Response:

24.05.2026, 16:01:52

Предлагаем следующий шаг в процессе разработки

Похоже, что текущий этап завершён, но есть некоторые мелкие визуальные нюансы, например, проблемы с выравниванием текста на правой панели и скрытым текстом на карточке объёмов. Можно решить эти вопросы и затем перейти к следующему этапу с подключением API. Считаю, это логичный следующий шаг.

Ищем актуальные данные из официальной документации

API CoinGecko вполне стабилен, но важно учитывать, что его параметры могут измениться. Поскольку API может обновляться, я должен проверить официальную документацию, чтобы быть уверенным в актуальности информации. Это важно, особенно перед тем как интегрировать его в код.

Поиск документации API CoinGecko

[Coins List with Market Data CoinGeckoApi — coin_gecko_api v0.1.0](#) [Crypto Data API: Most Comprehensive & Reliable ...](#) [CoinGecko API Client for Node.js](#) [CoinGecko Python API Docs](#) [CoinGecko](#) [CoinGecko API: The Cryptocurrency Data Powerhouse](#) [Coin Price by IDs - CoinGecko API Documentation](#) [coingecko_api - Dart API docs](#)

Планируем следующий этап и риски

На текущем этапе можно предложить план для версии v0.1 и перейти к следующему шагу. Однако перед тем как предложить код для API, стоит отметить возможные проблемы, связанные с SSL и MinGW. Нужно будет добавить поддержку сети и использовать `QNetworkAccessManager`. Следующий шаг — реализация получения реальных цен.

Думал на протяжении 17s

Отлично. **Версия 0.1 собрана и работает.** Визуал уже не выглядит как “прямоугольные кнопки из лабораторной”, это хороший старт.

По скрину вижу 3 мелких момента, но они не критичные:

1. Справа результат конвертора переносится некрасиво: `ETH` падает на новую строку.
2. Нижние графики визуально чуть тесные.
3. Пока данные демонстрационные, поэтому сейчас проект выглядит красиво, но ещё не выполняет главную задачу - загрузку реальных курсов.

Теперь логичный следующий этап - **версия 0.2: подключить CoinGecko API для текущих цен.**

CoinGecko endpoint `/coins/markets` подходит для этого, потому что он отдаёт текущую цену, объём торгов, рыночные данные и изменение цены. (docs.coingecko.com)

Историю графиков позже будем брать через `/coins/{id}/market_chart`, потому что он отдаёт исторические цены и объёмы. (docs.coingecko.com)

Что делаем сейчас

Добавляем:

- реальную загрузку цен через интернет;
- обновление левой панели;
- обновление цены выбранной монеты;
- кнопку `Обновить`, которая теперь реально обращается к CoinGecko;
- если API не сработал - приложение не падает, остаются демо-данные.

Шаг 1. Замени `project.pro`

Полностью замени `project.pro` на:

```
QT += core gui widgets network

CONFIG += c++17

greaterThan(QT_MAJOR_VERSION, 4): QT += widgets

TARGET = CryptoTrack
TEMPLATE = app

SOURCES += \
    main.cpp \
    ai_project.cpp
```

```
HEADERS += \  
    ai_project.h  
  
FORMS += \  
    ai_project.ui
```

Главное изменение здесь:

```
QT += network
```

Без этого `QNetworkAccessManager` не соберётся.

Шаг 2. Замени `ai_project.h`

Полностью замени `ai_project.h` на:

```
#ifndef AI_PROJECT_H  
#define AI_PROJECT_H  
  
#include <QMainWindow>  
#include <QVector>  
#include <QString>  
  
class QLabel;  
class QListWidget;  
class QPushButton;  
class QComboBox;  
class QLineEdit;  
class QFrame;  
class MiniChartWidget;  
class QNetworkAccessManager;  
class QNetworkReply;  
class QJsonObject;  
  
struct CoinUiData  
{  
    QString id;  
    QString symbol;  
    QString name;  
  
    double price;  
    double changePercent;  
    double volume;  
  
    QVector<double> history;  
    QVector<double> volumes;  
    QVector<double> rsi;  
};
```

```

class ai_project : public QMainWindow
{
    Q_OBJECT

public:
    explicit ai_project(QWidget *parent = nullptr);
    ~ai_project();

private:
    QVector<CoinUiData> coins;

    QNetworkAccessManager *networkManager;

    QListWidget *coinList;

    QLabel *coinNameLabel;
    QLabel *coinPriceLabel;
    QLabel *coinChangeLabel;
    QLabel *periodInfoLabel;
    QLabel *lastUpdateLabel;

    MiniChartWidget *priceChart;
    MiniChartWidget *volumeChart;
    MiniChartWidget *rsiChart;

    QPushButton *period24Button;
    QPushButton *period7Button;
    QPushButton *period30Button;
    QPushButton *period90Button;

    QPushButton *lineButton;
    QPushButton *candleButton;
    QPushButton *refreshButton;

    QComboBox *alertCoinCombo;
    QComboBox *alertConditionCombo;
    QLineEdit *alertPriceEdit;
    QListWidget *alertsList;

    QComboBox *fromCoinCombo;
    QComboBox *toCoinCombo;
    QLineEdit *amountEdit;
    QLabel *converterResultLabel;

    QString selectedCoinId;
    QString selectedPeriod;
    bool candleMode;

private:
    void setupInterface();
    void setupStyle();
    void fillDemoData();

```

```

void buildCoinList();

QFrame *createCard();
QPushButton *createSmallButton(const QString &text);

void selectCoin(int row);
void updateCenterPanel(const CoinUiData &coin);
void updateConverter();
void addAlert();
void refreshDemoData();

void fetchMarketData();
void handleMarketDataReply(QNetworkReply *reply);
void updateCoinFromJson(const QJsonObject &object);

int findCoinIndexById(const QString &id) const;
QString formatPrice(double value) const;
QString formatVolume(double value) const;
};

#endif

```

Шаг 3. В ai_project.cpp замени блок #include

В самом верху ai_project.cpp замени все #include на ЭТОТ блок:

```

#include "ai_project.h"

#include <QApplication>
#include <QButtonGroup>
#include <QComboBox>
#include <QDateTime>
#include <QFrame>
#include <QHBoxLayout>
#include <QJsonArray>
#include <QJsonDocument>
#include <QJsonObject>
#include <QLabel>
#include <QLineEdit>
#include <QListWidget>
#include <QMessageBox>
#include <QNetworkAccessManager>
#include <QNetworkReply>
#include <QNetworkRequest>
#include <QPainter>
#include <QPainterPath>
#include <QPushButton>
#include <QScrollBar>
#include <QUrl>

```



```
#include <QVBoxLayout>
#include <QtMath>
```

Шаг 4. В конструкторе добавь `networkManager`

Найди начало конструктора:

```
ai_project::ai_project(QWidget *parent)
    : QMainWindow(parent),
      coinList(nullptr),
```

Замени его на:

```
ai_project::ai_project(QWidget *parent)
    : QMainWindow(parent),
      networkManager(new QNetworkAccessManager(this)),
      coinList(nullptr),
```

Шаг 5. В конец конструктора добавь загрузку API

В конструкторе сейчас в конце есть такой кусок:

```
if (!coins.isEmpty())
{
    coinList->setCurrentRow(0);
    selectCoin(0);
}
```

Замени его на:

```
if (!coins.isEmpty())
{
    coinList->setCurrentRow(0);
    selectCoin(0);
}

fetchMarketData();
```

Шаг 6. Замени функцию `refreshDemoData()`

Найди старую функцию:

```
void ai_project::refreshDemoData()
{
    ...
}
```

Полностью замени её на:

```
void ai_project::refreshDemoData()
{
    fetchMarketData();
}
```

Название пока оставляем старое, чтобы не ломать уже подключённые сигналы. По факту теперь кнопка **Обновить** будет загружать реальные данные.

Шаг 7. Добавь новые функции в конец `ai_project.cpp`

В самый конец файла, после функции `formatVolume`, добавь:

```
void ai_project::fetchMarketData()
{
    refreshButton->setEnabled(false);
    refreshButton->setText("Загрузка...");

    QUrl url(
        "https://api.coingecko.com/api/v3/coins/markets"
        "?vs_currency=usd"
        "&ids=bitcoin,ethereum,solana,dogecoin,cardano,ripple,polkadot"
        "&order=market_cap_desc"
        "&per_page=7"
        "&page=1"
        "&sparkline=false"
        "&price_change_percentage=24h"
    );

    QNetworkRequest request(url);
    request.setHeader(QNetworkRequest::UserAgentHeader, "CryptoTrackQtCourseProject/1.0");

    QNetworkReply *reply = networkManager->get(request);

    connect(reply, &QNetworkReply::finished, this, [this, reply]()
    {
        handleMarketDataReply(reply);
        reply->deleteLater();
    });
}
```

```

});
}

void ai_project::handleMarketDataReply(QNetworkReply *reply)
{
    refreshButton->setEnabled(true);
    refreshButton->setText("Обновить");

    if (reply->error() != QNetworkReply::NoError)
    {
        lastUpdateLabel->setText("Ошибка загрузки: используются демо-данные");
        QMessageBox::warning(
            this,
            "Ошибка загрузки",
            "Не удалось получить данные CoinGecko.\n"
            "Проверьте интернет или попробуйте обновить позже.\n\n"
            "Приложение продолжит работу на демонстрационных данных."
        );
        return;
    }

    QByteArray responseData = reply->readAll();

    QJsonParseError parseError;
    QJsonDocument document = QJsonDocument::fromJson(responseData, &parseError);

    if (parseError.error != QJsonParseError::NoError || !document.isArray())
    {
        lastUpdateLabel->setText("Ошибка разбора данных API");
        QMessageBox::warning(
            this,
            "Ошибка данных",
            "CoinGecko вернул данные в неожиданном формате."
        );
        return;
    }

    QJsonArray array = document.array();

    for (const QJsonValue &value : array)
    {
        if (value.isObject())
            updateCoinFromJson(value.toObject());
    }

    QString oldSelectedId = selectedCoinId;

    buildCoinList();

    int selectedIndex = findCoinIndexById(oldSelectedId);

    if (selectedIndex < 0)
        selectedIndex = 0;

```

```

coinList->setCurrentRow(selectedIndex);
selectCoin(selectedIndex);

lastUpdateLabel->setText(
    "Последнее обновление: " +
    QDateTime::currentDateTime().toString("dd.MM.yyyy HH:mm:ss")
);
}

void ai_project::updateCoinFromJson(const QJsonObject &object)
{
    QString id = object.value("id").toString();

    int index = findCoinIndexById(id);

    if (index < 0)
        return;

    double newPrice = object.value("current_price").toDouble(coins[index].price);
    double newChange = object.value("price_change_percentage_24h").toDouble(coins[index].changePercent);
    double newVolume = object.value("total_volume").toDouble(coins[index].volume);

    coins[index].price = newPrice;
    coins[index].changePercent = newChange;
    coins[index].volume = newVolume;

    if (!coins[index].history.isEmpty())
    {
        coins[index].history.removeFirst();
        coins[index].history.append(newPrice);
    }

    if (!coins[index].volumes.isEmpty())
    {
        coins[index].volumes.removeFirst();
        coins[index].volumes.append(newVolume);
    }

    if (!coins[index].rsi.isEmpty())
    {
        double direction = coins[index].changePercent >= 0 ? 1.0 : -1.0;
        double newRsi = qBound(10.0, coins[index].rsi.last() + direction * 1.2, 90.0);

        coins[index].rsi.removeFirst();
        coins[index].rsi.append(newRsi);
    }
}

```

Шаг 8. Собери правильно

После изменений обязательно:

```
Сборка → Очистить проект
Сборка → Запустить qmake
Сборка → Пересобрать проект
```

Что должно измениться

После запуска:

- окно откроется сразу с демо-данными;
- затем приложение попытается загрузить реальные цены;
- кнопка станет `Загрузка...` ;
- если интернет есть, цены слева и в центре обновятся;
- если API не ответит, будет предупреждение, но приложение не упадёт.

Это будет **версия 0.2**: реальные текущие курсы уже подключены, но графики пока всё ещё демонстрационные. Следующим этапом подключим реальную историю цены для выбранной монеты.

Sources:

- [Coins List with Market Data](#)

Prompt:

24.05.2026, 16:08:09

вылезает ошибка не удалось получить данные CoinGecko Проверьте интернет и т.д., + текст почему то очень светло голубой, практически не читаемый, такой же текст в выпадающих барах, где допустим выбирать какую монету на какую поменять, можно ли поменять цвет шрифта

Response:

24.05.2026, 16:08:13

Решение проблемы с API и ошибками в интерфейсе

Кажется, ошибка может быть связана с отсутствием OpenSSL и проблемами с SSL на Windows/Qt. Пожалуй, нужно изменить код для диагностики ошибок и улучшения сообщений (например, показывать статус HTTP и текст ошибки). Также стоит настроить стиль для `QMessageBox` и других элементов интерфейса, чтобы исправить цвета.

Да, цвет поправить можно. Проблема возникла из-за этой строки в стиле:

```
QWidget {  
    color: #eef2ff;  
}
```

Она покрасила текст **во всех виджетах**, включая системное окно ошибки и выпадающие списки. На тёмном фоне это красиво, а на белом системном окне почти не видно.

Сейчас делаем две правки:

1. исправляем цвета текста в `QMessageBox` и выпадающих списках;
2. делаем нормальную диагностику ошибки CoinGecko, чтобы видеть не просто “не удалось”, а конкретную причину.

1. Замени блок `#include` В `ai_project.cpp`

Верхний блок `#include` замени на этот:

```
#include "ai_project.h"  
  
#include <QApplication>  
#include <QButtonGroup>  
#include <QComboBox>  
#include <QDateTime>  
#include <QFrame>  
#include <QHBoxLayout>  
#include <QJsonArray>  
#include <QJsonDocument>  
#include <QJsonObject>  
#include <QLabel>  
#include <QLineEdit>  
#include <QListWidget>  
#include <QMessageBox>  
#include <QNetworkAccessManager>  
#include <QNetworkReply>  
#include <QNetworkRequest>  
#include <QPainter>  
#include <QPainterPath>  
#include <QPushButton>  
#include <QScrollBar>  
#include <QSslSocket>  
#include <QUrl>  
#include <QVBoxLayout>  
#include <QtMath>
```

Добавилось:

```
#include <QSslSocket>
```

Это нужно, чтобы проверить SSL. CoinGecko работает через `https`, а на Windows с Qt/MinGW часто проблема именно в SSL-библиотеках.

2. Полностью замени функцию `setupStyle()`

Найди функцию:

```
void ai_project::setupStyle()
{
    ...
}
```

И полностью замени её на эту:

```
void ai_project::setupStyle()
{
    qApp->setStyleSheet(
        "QMainWindow {"
        "    background: #070b16;"
        "}"

        "QWidget {"
        "    font-family: Segoe UI, Arial;"
        "    color: #eef2ff;"
        "    font-size: 14px;"
        "}"

        "QFrame#Card {"
        "    background: #0f172a;"
        "    border: 1px solid #1e293b;"
        "    border-radius: 22px;"
        "}"

        "QLabel#AppTitle {"
        "    font-size: 28px;"
        "    font-weight: 800;"
        "    color: #ffffff;"
        "}"

        "QLabel#CoinName {"
        "    font-size: 24px;"
        "    font-weight: 700;"
        "    color: #ffffff;"
        "}"
```

```
"QLabel#CoinPrice {"  
  "    font-size: 42px;"  
  "    font-weight: 800;"  
  "    color: #ffffff;"  
  "}"
```

```
"QLabel#BlockTitle {"  
  "    font-size: 17px;"  
  "    font-weight: 700;"  
  "    color: #f8fafc;"  
  "}"
```

```
"QLabel#MutedText {"  
  "    color: #94a3b8;"  
  "    font-size: 13px;"  
  "}"
```

```
"QLabel#PositiveText {"  
  "    color: #22c55e;"  
  "    font-size: 16px;"  
  "    font-weight: 700;"  
  "}"
```

```
"QLabel#NegativeText {"  
  "    color: #ef4444;"  
  "    font-size: 16px;"  
  "    font-weight: 700;"  
  "}"
```

```
"QLabel#ConverterResult {"  
  "    background: #111c33;"  
  "    border: 1px solid #27344f;"  
  "    border-radius: 14px;"  
  "    padding: 12px;"  
  "    color: #dbeafe;"  
  "    font-weight: 700;"  
  "}"
```

```
"QPushButton {"  
  "    background: #1e293b;"  
  "    color: #e5e7eb;"  
  "    border: 1px solid #334155;"  
  "    border-radius: 13px;"  
  "    padding: 9px 14px;"  
  "    font-weight: 600;"  
  "}"
```

```
"QPushButton:hover {"  
  "    background: #273449;"  
  "    border: 1px solid #475569;"  
  "}"
```



```
"QPushButton:checked {"
    background: #2563eb;"
    color: white;"
    border: 1px solid #3b82f6;"
}"

"QListWidget#CoinList, QListWidget#SimpleList {"
    background: transparent;"
    border: none;"
    outline: none;"
}"

"QListWidget::item {"
    border-radius: 16px;"
    margin: 3px;"
    padding: 4px;"
}"

"QListWidget::item:selected {"
    background: #1d4ed8;"
}"

"QComboBox, QLineEdit {"
    background: #111827;"
    border: 1px solid #334155;"
    border-radius: 13px;"
    padding: 9px 11px;"
    color: #f8fafc;"
    selection-background-color: #2563eb;"
    selection-color: #ffffff;"
}"

"QComboBox:hover, QLineEdit:hover {"
    border: 1px solid #475569;"
}"

"QComboBox::drop-down {"
    border: none;"
    width: 28px;"
}"

"QComboBox QAbstractItemView {"
    background: #111827;"
    color: #f8fafc;"
    border: 1px solid #334155;"
    selection-background-color: #2563eb;"
    selection-color: #ffffff;"
    outline: none;"
}"

"QMessageBox {"
    background-color: #0f172a;"
}
```

```

"QMessageBox QLabel {"
    "    color: #f8fafc;"
    "    font-size: 14px;"
    "}"

"QMessageBox QPushButton {"
    "    min-width: 80px;"
    "    background: #1e293b;"
    "    color: #ffffff;"
    "    border: 1px solid #334155;"
    "    border-radius: 12px;"
    "    padding: 8px 14px;"
    "}"

"QMessageBox QPushButton:hover {"
    "    background: #2563eb;"
    "}"

"QScrollBar:vertical {"
    "    background: transparent;"
    "    width: 8px;"
    "}"

"QScrollBar::handle:vertical {"
    "    background: #334155;"
    "    border-radius: 4px;"
    "}"

"QScrollBar::add-line:vertical, QScrollBar::sub-line:vertical {"
    "    height: 0px;"
    "}"

);
}

```

Это исправит:

- светлый текст в окне ошибки;
- светлый текст в выпадающих списках;
- нечитаемый текст при выборе элементов.

3. Замени функцию `handleMarketDataReply`

Найди функцию:

```

void ai_project::handleMarketDataReply(QNetworkReply *reply)
{
    ...
}

```

И полностью замени её на эту:

```
void ai_project::handleMarketDataReply(QNetworkReply *reply)
{
    refreshButton->setEnabled(true);
    refreshButton->setText("Обновить");

    if (reply->error() != QNetworkReply::NoError)
    {
        int statusCode = reply->attribute(QNetworkRequest::HttpStatusCodeAttribute).toInt();

        QString sslInfo;

        if (QSslSocket::supportsSsl())
        {
            sslInfo =
                "SSL: поддерживается\n"
                "Версия SSL: " + QSslSocket::sslLibraryVersionString();
        }
        else
        {
            sslInfo =
                "SSL: НЕ поддерживается\n"
                "Вероятная причина: Qt не видит OpenSSL-библиотеки.";
        }

        QString errorText =
            "Не удалось получить данные CoinGecko.\n\n"
            "Техническая причина:\n" +
            reply->errorString() +
            "\n\nHTTP-код: " + QString::number(statusCode) +
            "\n\n" + sslInfo +
            "\n\nПриложение продолжит работу на демонстрационных данных.";

        lastUpdateLabel->setText("Ошибка загрузки: используются демо-данные");

        QMessageBox::warning(
            this,
            "Ошибка загрузки",
            errorText
        );

        return;
    }

    QByteArray responseData = reply->readAll();

    QJsonParseError parseError;
    QJsonDocument document = QJsonDocument::fromJson(responseData, &parseError);

    if (parseError.error != QJsonParseError::NoError || !document.isArray())
    {
        lastUpdateLabel->setText("Ошибка разбора данных API");
    }
}
```

```

        QMessageBox::warning(
            this,
            "Ошибка данных",
            "CoinGecko вернул данные в неожиданном формате.\n\n"
            "Ошибка JSON: " + parseError.errorString()
        );

        return;
    }

    QJsonArray array = document.array();

    for (const QJsonValue &value : array)
    {
        if (value.isObject())
            updateCoinFromJson(value.toObject());
    }

    QString oldSelectedId = selectedCoinId;

    buildCoinList();

    int selectedIndex = findCoinIndexById(oldSelectedId);

    if (selectedIndex < 0)
        selectedIndex = 0;

    coinList->setCurrentRow(selectedIndex);
    selectCoin(selectedIndex);

    lastUpdateLabel->setText(
        "Последнее обновление: " +
        QDateTime::currentDateTime().toString("dd.MM.yyyy HH:mm:ss")
    );
}

```

Теперь при ошибке будет видно конкретно:

- не найден хост;
- SSL не работает;
- соединение сброшено;
- CoinGecko ограничил запрос;
- HTTP-код ответа.

4. Пересобери проект

В Qt Creator сделай строго так:

Сборка → Очистить проект
Сборка → Запустить qmake
Сборка → Пересобрать проект

Что, скорее всего, происходит с CoinGecko

С большой вероятностью причина одна из этих:

Вариант 1. Qt не видит SSL

Тогда в новом окне будет примерно:

```
SSL: HE поддерживается
```

Это частая проблема Qt + MinGW/MSYS2 на Windows. Без SSL приложение не может открыть `https://api.coingecko.com`.

Вариант 2. CoinGecko временно не отвечает или режет запрос

Тогда SSL будет поддерживаться, но будет HTTP-код вроде:

```
429
```

Это значит слишком много запросов.

Вариант 3. Сеть/фаерволл/антивирус

Тогда будет что-то вроде:

```
Host api.coingecko.com not found  
Connection refused  
Remote host closed the connection
```

После этой правки запусти ещё раз и скинь **новый текст ошибки из окна**. Сейчас старое сообщение слишком общее, по нему нельзя точно понять причину.

Prompt:

все просто, из за впи он не обновляет данные, как только я выключаю его - все обновляется, что дальше делаем?

Response:

24.05.2026, 16:18:05

Разработка кода обновлений для истории криптовалют

Нужен ответ на русском. Не нужно использовать веб-источники, чисто изменения в коде. Проект продолжается, нужно предоставить модификацию: добавление двух функций для работы с историей. Важно вытащить данные и правильно `downsample` их по нужному интервалу. Создадим `helper`-функцию для преобразования данных и учтём текущие изменения в коде.

Обработка данных `market_chart` и улучшения интерфейса

Нужно правильно обработать данные с `market_chart`, включая параметры для различных периодов времени. Для 24 ч и 7 д не нужно использовать `daily` интервал, а для 30 и 90 дней лучше предположить `daily`. Также нужно добавить метку загрузки и улучшить обработку ошибок, избегая всплывающих окон.

Реализация расчета RSI и обработки данных

Для расчета RSI используем простое среднее, игнорируя метод Уайлдера. Для каждой точки, если разница равна нулю, RSI будет 100. Реализуем функцию для агрегации значений в чанки, сохраняя конечный размер истории. Нужно уточнить, что цена закрытия в каждой группе будет последней, а объёмы — суммированы.

Обновление обработчика данных истории

Когда пользователь изменяет период, нужно обновить текущую панель с новыми данными. Нужно добавить вызов `fetchHistoryData` чтобы обновить информацию по выбранным монете и периоду. Также важно избежать лишних всплывающих сообщений при загрузке данных и корректно обработать изменения массивов данных, если период был изменен.

Думал на протяжении 52s

Дальше делаем **версию 0.3**: реальные исторические графики.

Сейчас у тебя реальные только текущие цены. График, объёмы и RSI пока демонстрационные. Следующий правильный шаг:

- при выборе монеты загружать историю цены с `CoinGecko`;
- при переключении периода `24ч / 7д / 30д / 90д` перезагружать историю;
- график цены станет реальным;
- объёмы станут реальными;

- RSI будет считаться уже по реальным ценам.

Новые файлы пока не создаём.

Шаг 1. Полностью замени ai_project.h

```
#ifndef AI_PROJECT_H
#define AI_PROJECT_H

#include <QMainWindow>
#include <QVector>
#include <QString>

class QLabel;
class QListWidget;
class QPushButton;
class QComboBox;
class QLineEdit;
class QFrame;
class MiniChartWidget;
class QNetworkAccessManager;
class QNetworkReply;
class QJsonObject;

struct CoinUiData
{
    QString id;
    QString symbol;
    QString name;

    double price;
    double changePercent;
    double volume;

    QVector<double> history;
    QVector<double> volumes;
    QVector<double> rsi;
};

class ai_project : public QMainWindow
{
    Q_OBJECT

public:
    explicit ai_project(QWidget *parent = nullptr);
    ~ai_project();

private:
    QVector<CoinUiData> coins;
```

```

QNetworkAccessManager *networkManager;

QWidget *rootWidget;

QListWidget *coinList;

QLabel *coinNameLabel;
QLabel *coinPriceLabel;
QLabel *coinChangeLabel;
QLabel *periodInfoLabel;
QLabel *lastUpdateLabel;

MiniChartWidget *priceChart;
MiniChartWidget *volumeChart;
MiniChartWidget *rsiChart;

QPushButton *period24Button;
QPushButton *period7Button;
QPushButton *period30Button;
QPushButton *period90Button;

QPushButton *lineButton;
QPushButton *candleButton;
QPushButton *refreshButton;

QComboBox *alertCoinCombo;
QComboBox *alertConditionCombo;
QLineEdit *alertPriceEdit;
QListWidget *alertsList;

QComboBox *fromCoinCombo;
QComboBox *toCoinCombo;
QLineEdit *amountEdit;
QLabel *converterResultLabel;

QString selectedCoinId;
QString selectedPeriod;
bool candleMode;

private:
    void setupInterface();
    void setupStyle();
    void fillDemoData();
    void buildCoinList();

    QFrame *createCard();
    QPushButton *createSmallButton(const QString &text);

    void selectCoin(int row);
    void updateCenterPanel(const CoinUiData &coin);
    void updateConverter();
    void addAlert();
    void refreshDemoData();

    void fetchMarketData();

```



```
void handleMarketDataReply(QNetworkReply *reply);
void updateCoinFromJson(const QJsonObject &object);

void fetchHistoryData(const QString &coinId, const QString &period);
void handleHistoryDataReply(QNetworkReply *reply, const QString &coinId, const QString &period);

int daysForPeriod(const QString &period) const;
int targetPointCountForPeriod(const QString &period) const;

 QVector<double> reduceToClosingValues(const QVector<double> &source, int targetCount) const;
 QVector<double> reduceToAverageValues(const QVector<double> &source, int targetCount) const;
 QVector<double> calculateRsiValues(const QVector<double> &prices) const;

int findCoinIndexById(const QString &id) const;
QString formatPrice(double value) const;
QString formatVolume(double value) const;
};

#endif
```

Шаг 2. Замени функцию `selectCoin`

В `ai_project.cpp` найди:

```
void ai_project::selectCoin(int row)
```

И полностью замени функцию на:

```
void ai_project::selectCoin(int row)
{
    if (row < 0 || row >= coins.size())
        return;

    selectedCoinId = coins[row].id;

    updateCenterPanel(coins[row]);
    fetchHistoryData(selectedCoinId, selectedPeriod);
}
```

Теперь при выборе монеты будет не только меняться интерфейс, но и загружаться реальная история.

Шаг 3. В конструкторе убери двойной вызов `selectCoin`

В конструкторе найди блок:

```
if (!coins.isEmpty())
{
    coinList->setCurrentRow(0);
    selectCoin(0);
}

fetchMarketData();
```

Замени его на:

```
if (!coins.isEmpty())
{
    coinList->blockSignals(true);
    coinList->setCurrentRow(0);
    coinList->blockSignals(false);
    selectCoin(0);
}

fetchMarketData();
```

Это нужно, чтобы история не загружалась два раза подряд.

Шаг 4. В `handleMarketDataReply` тоже убери двойной вызов

В функции `handleMarketDataReply` найди в конце такой блок:

```
coinList->setCurrentRow(selectedIndex);
selectCoin(selectedIndex);

lastUpdateLabel->setText(
    "Последнее обновление: " +
    QDateTime::currentDateTime().toString("dd.MM.yyyy HH:mm:ss")
);
```

Замени на:

```
coinList->blockSignals(true);
coinList->setCurrentRow(selectedIndex);
coinList->blockSignals(false);

selectCoin(selectedIndex);

lastUpdateLabel->setText(
```

```
"Последнее обновление: " +  
    QDateTime::currentDateTime().toString("dd.MM.yyyy HH:mm:ss")  
);
```

Шаг 5. В конец ai_project.cpp добавь новые функции

В самый конец файла, после всех существующих функций, вставь:

```
void ai_project::fetchHistoryData(const QString &coinId, const QString &period)  
{  
    int days = daysForPeriod(period);  
  
    lastUpdateLabel->setText("Загрузка истории курса...");  
  
    QString urlText =  
        "https://api.coingecko.com/api/v3/coins/" + coinId +  
        "/market_chart?vs_currency=usd&days=" + QString::number(days);  
  
    QNetworkRequest request(QUrl(urlText));  
    request.setHeader(QNetworkRequest::UserAgentHeader, "CryptoTrackQtCourseProject/1.0");  
  
    QNetworkReply *reply = networkManager->get(request);  
  
    connect(reply, &QNetworkReply::finished, this, [this, reply, coinId, period]()  
    {  
        handleHistoryDataReply(reply, coinId, period);  
        reply->deleteLater();  
    });  
}  
  
void ai_project::handleHistoryDataReply(QNetworkReply *reply, const QString &coinId, const QS  
{  
    if (coinId != selectedCoinId || period != selectedPeriod)  
        return;  
  
    if (reply->error() != QNetworkReply::NoError)  
    {  
        lastUpdateLabel->setText("История курса не загружена, показаны сохранённые данные");  
        return;  
    }  
  
    QByteArray responseData = reply->readAll();  
  
    QJsonParseError parseError;  
    QJsonDocument document = QJsonDocument::fromJson(responseData, &parseError);  
  
    if (parseError.error != QJsonParseError::NoError || !document.isObject())
```

```

{
    lastUpdateLabel->setText("Ошибка разбора истории курса");
    return;
}

QJsonObject root = document.object();

QJsonArray pricesArray = root.value("prices").toArray();
QJsonArray volumesArray = root.value("total_volumes").toArray();

QVector<double> rawPrices;
QVector<double> rawVolumes;

for (const QJsonValue &value : pricesArray)
{
    QJsonArray pair = value.toArray();

    if (pair.size() >= 2)
        rawPrices.append(pair.at(1).toDouble());
}

for (const QJsonValue &value : volumesArray)
{
    QJsonArray pair = value.toArray();

    if (pair.size() >= 2)
        rawVolumes.append(pair.at(1).toDouble());
}

if (rawPrices.size() < 2)
{
    lastUpdateLabel->setText("Недостаточно исторических данных");
    return;
}

int coinIndex = findCoinIndexById(coinId);

if (coinIndex < 0)
    return;

int targetCount = targetPointCountForPeriod(period);

QVector<double> preparedPrices = reduceToClosingValues(rawPrices, targetCount);
QVector<double> preparedVolumes = reduceToAverageValues(rawVolumes, targetCount);

coins[coinIndex].history = preparedPrices;
coins[coinIndex].volumes = preparedVolumes;
coins[coinIndex].rsi = calculateRsiValues(preparedPrices);

if (!preparedPrices.isEmpty())
    coins[coinIndex].price = preparedPrices.last();

updateCenterPanel(coins[coinIndex]);

```

```

        lastUpdateLabel->setText(
            "История обновлена: " +
            QDateTime::currentDateTime().toString("dd.MM.yyyy HH:mm:ss")
        );
    }

int ai_project::daysForPeriod(const QString &period) const
{
    if (period == "24ч")
        return 1;

    if (period == "7д")
        return 7;

    if (period == "30д")
        return 30;

    return 90;
}

int ai_project::targetPointCountForPeriod(const QString &period) const
{
    if (period == "24ч")
        return 24;

    if (period == "7д")
        return 42;

    if (period == "30д")
        return 30;

    return 90;
}

 QVector<double> ai_project::reduceToClosingValues(const QVector<double> &source, int targetCo
{
    QVector<double> result;

    if (source.isEmpty() || targetCount <= 0)
        return result;

    if (source.size() <= targetCount)
        return source;

    for (int i = 0; i < targetCount; ++i)
    {
        int start = static_cast<int>(static_cast<double>(i) * source.size() / targetCount);
        int end = static_cast<int>(static_cast<double>(i + 1) * source.size() / targetCount);

        if (end <= start)
            end = start + 1;
    }
}

```

```

        if (end > source.size())
            end = source.size();

        result.append(source[end - 1]);
    }

    return result;
}

 QVector<double> ai_project::reduceToAverageValues(const QVector<double> &source, int targetCo
{
    QVector<double> result;

    if (source.isEmpty() || targetCount <= 0)
        return result;

    if (source.size() <= targetCount)
        return source;

    for (int i = 0; i < targetCount; ++i)
    {
        int start = static_cast<int>(static_cast<double>(i) * source.size() / targetCount);
        int end = static_cast<int>(static_cast<double>(i + 1) * source.size() / targetCount);

        if (end <= start)
            end = start + 1;

        if (end > source.size())
            end = source.size();

        double sum = 0.0;
        int count = 0;

        for (int j = start; j < end; ++j)
        {
            sum += source[j];
            ++count;
        }

        if (count > 0)
            result.append(sum / count);
    }

    return result;
}

 QVector<double> ai_project::calculateRsiValues(const QVector<double> &prices) const
{
    QVector<double> rsi;

    const int period = 14;

    if (prices.size() < 2)

```

```

return rsi;

for (int i = 0; i < prices.size(); ++i)
{
    if (i < period)
    {
        rsi.append(50.0);
        continue;
    }

    double gainSum = 0.0;
    double lossSum = 0.0;

    for (int j = i - period + 1; j <= i; ++j)
    {
        double change = prices[j] - prices[j - 1];

        if (change > 0)
            gainSum += change;
        else
            lossSum += qAbs(change);
    }

    double averageGain = gainSum / period;
    double averageLoss = lossSum / period;

    double value = 50.0;

    if (qFuzzyIsNull(averageLoss) && qFuzzyIsNull(averageGain))
    {
        value = 50.0;
    }
    else if (qFuzzyIsNull(averageLoss))
    {
        value = 100.0;
    }
    else
    {
        double rs = averageGain / averageLoss;
        value = 100.0 - 100.0 / (1.0 + rs);
    }

    rsi.append(qBound(0.0, value, 100.0));
}

return rsi;
}

```

Шаг 6. Пересобери

Сборка → Очистить проект
Сборка → Запустить qmake
Сборка → Пересобрать проект

Что должно получиться

После запуска:

- при выборе BTC/ETH/SOL и т.д. будет загружаться реальная история;
- при переключении 24ч / 7д / 30д / 90д график будет меняться по реальным данным;
- объёмы тоже будут приходить из CoinGecko;
- RSI будет считаться по реальным точкам цены;
- если VPN снова мешает, приложение просто оставит старые данные и не упадёт.

После этого следующий этап — **нормальные свечи и уровни поддержки/сопротивления**. Сейчас кнопка “Свечи” визуально рисует свечи из цен закрытия, но это ещё не полноценный OHLC-график.

Prompt:

24.05.2026, 16:21:36

```
C:\Users\bakki\Documents\project\ai_project.cpp:1231: предупреждение: parentheses were
disambiguated as a function declaration [-Wvexing-parse] ....\ai_project.cpp: In member function 'void
ai_project::fetchHistoryData(const QString&, const QString&): ....\ai_project.cpp:1231:28: warning:
parentheses were disambiguated as a function declaration [-Wvexing-parse] 1231 | QNetworkRequest
request(QUrl(urlText)); | ^~~~~~ ....\ai_project.cpp:1231:28: note: replace parentheses with
braces to declare a variable 1231 | QNetworkRequest request(QUrl(urlText)); | ^~~~~~ | - | {
- | }C:\Users\bakki\Documents\project\ai_project.cpp:1232: ошибка: request for member 'setHeader' in
'request', which is of non-class type 'QNetworkRequest(QUrl)' ....\ai_project.cpp:1232:13: error: request
for member 'setHeader' in 'request', which is of non-class type 'QNetworkRequest(QUrl)' 1232 |
request.setHeader(QNetworkRequest::UserAgentHeader, "CryptoTrackQtCourseProject/1.0"); |
^~~C:\Users\bakki\Documents\project\ai_project.cpp:1234: ошибка: no matching function for call to
'QNetworkAccessManager::get(QNetworkRequest (&)(QUrl))' ....\ai_project.cpp:1234:47: error: no
matching function for call to 'QNetworkAccessManager::get(QNetworkRequest (&)(QUrl))' 1234 |
QNetworkReply reply = networkManager->get(request); | ~~~~~^~~~~~
есть there are 3
candidatesC:\msys64\mingw64\include\qt6\QtNetwork\QNetworkAccessManager:1: In file included from
C:\msys64\mingw64\include\qt6\QtNetwork\QNetworkAccessManager:1, In file included from
C:\msys64\mingw64\include\qt6\QtNetwork\QNetworkAccessManager:1, from ....\ai_project.cpp:16:
есть candidate 1: 'QNetworkReply QNetworkAccessManager::get(const QNetworkRequest&)'
C:\msys64\mingw64\include\qt6\QtNetwork\qnetworkaccessmanager.h:83:20: 83 | QNetworkReply
*get(const QNetworkRequest &request); | ^ в есть no known conversion for argument 1 from
```


~~'QNetworkRequest(QUrl)' to 'const QNetworkRequest&'~~
C:/msys64/mingw64/include/qt6/QtNetwork/qnetworkaccessmanager.h:83:47: 83 | QNetworkReply
~~get(const QNetworkRequest &request);~~ | ~~~~~~~^ в Тъ candidate 2: 'QNetworkReply~~
~~QNetworkAccessManager::get(const QNetworkRequest&, QIODevice*)'~~
C:/msys64/mingw64/include/qt6/QtNetwork/qnetworkaccessmanager.h:84:20: 84 | QNetworkReply
~~*get(const QNetworkRequest &request, QIODevice *data);~~ | ~~^ в Тъ candidate expects 2 arguments, 1~~
~~provided в Тъ candidate 3: 'QNetworkReply* QNetworkAccessManager::get(const QNetworkRequest&,~~
~~const QByteArray&)' C:/msys64/mingw64/include/qt6/QtNetwork/qnetworkaccessmanager.h:85:20: 85 |~~
~~QNetworkReply *get(const QNetworkRequest &request, const QByteArray &data);~~ | ~~^ в Тъ candidate~~
~~expects 2 arguments, 1 provided~~такие ошибки

Response:

24.05.2026, 16:21:38

Это не страшная ошибка. C++ неправильно понял строку:

```
QNetworkRequest request(QUrl(urlText));
```

Он воспринял её не как создание объекта, а как объявление функции. Это классическая ошибка **most vexing parse**.

Замени в функции `fetchHistoryData()` **ВОТ ЭТОТ** кусок:

```
QNetworkRequest request(QUrl(urlText));  
request.setHeader(QNetworkRequest::UserAgentHeader, "CryptoTrackQtCourseProject/1.0");  
  
QNetworkReply *reply = networkManager->get(request);
```

на ЭТОТ:

```
QNetworkRequest request{QUrl(urlText)};  
request.setHeader(QNetworkRequest::UserAgentHeader, "CryptoTrackQtCourseProject/1.0");  
  
QNetworkReply *reply = networkManager->get(request);
```

То есть меняется только первая строка:

```
QNetworkRequest request{QUrl(urlText)};
```

После этого:

```
Сборка → Очистить проект  
Сборка → Запустить qmake  
Сборка → Пересобрать проект
```

Смысл ошибки: компилятор увидел `QNetworkRequest request(QUrl(urlText));` и решил, что `request` — это функция, а не объект. Поэтому дальше и пишет, что у `request` нет метода `setHeader`.

Prompt:

24.05.2026, 16:26:57

все пересобралось, пока скажу что вижу(может ты хотел это исправить позже: 1) начиная с solana и до последней монетки, независимо от какого периода графика я беру, свечи выглядят примерно одинаково, то есть например в интервале 24ч - большая свеча вниз красная, но и в 7д 30д 90д примерно такой же график, я думаю это не совсем правильно, у остальных такая же беда, однако у битка и эфира все в порядке, 2) я бы хотел чтобы при наводке на свечу показывалось на сколько процентов эта свеча упала, ну и в целом чтобы график был с циферками, а не просто свечи бары(опять же может ты хотел это реализовать потом)

Response:

24.05.2026, 16:27:01

Да, ты правильно заметил. Это не “потом само исправится”, тут реально есть два слабых места текущей версии.

Почему свечи у многих монет похожи

Сейчас свечи **не настоящие OHLC-свечи**. Я пока рисую их из обычных цен закрытия:

```
open = предыдущая цена
close = текущая цена
high/low = искусственно чуть выше/ниже
```

То есть это временная визуализация, а не полноценные биржевые свечи. Поэтому на SOL/DOGE/ADA/XRP/DOT графики могут выглядеть слишком похожими, особенно если движение цены за разные периоды однотипное.

Настоящие свечи надо делать отдельным этапом через OHLC-данные или через собственную агрегацию исторических точек. Но перед этим правильно добавить **оси, цифры и подсказки при наведении**, иначе даже настоящие свечи будут плохо читаемыми.

Сейчас сделаем промежуточный шаг: **улучшим графики**.

Что добавим сейчас

1. На графиках появятся **цифры по вертикальной оси**.
2. Внизу появятся номера точек/свечей.
3. При наведении мышкой на свечу будет показываться:
 - номер свечи;
 - open;
 - close;
 - изменение в процентах;
 - рост/падение.
4. Для линии будет показываться значение точки.
5. Для объёма будет показываться объём.
6. Для RSI будет показываться RSI.

Шаг 1. В `ai_project.cpp` замени блок `include`

Верхний блок `#include` замени на этот:

```
#include "ai_project.h"

#include <QApplication>
#include <QButtonGroup>
#include <QComboBox>
#include <QDateTime>
#include <QFrame>
#include <QHBoxLayout>
#include <QJsonArray>
#include <QJsonDocument>
#include <QJsonObject>
#include <QLabel>
#include <QLineEdit>
#include <QListWidget>
#include <QMessageBox>
#include <QMouseEvent>
#include <QNetworkAccessManager>
#include <QNetworkReply>
#include <QNetworkRequest>
#include <QPainter>
#include <QPainterPath>
#include <QPushButton>
#include <QScrollBar>
#include <QSslSocket>
#include <QToolTip>
#include <QUrl>
#include <QVBoxLayout>
#include <QtMath>
```

Шаг 2. Полностью замени класс

MiniChartWidget

В `ai_project.cpp` найди весь класс:

```
class MiniChartWidget : public QWidget
{
    ...
};
```

И полностью замени его на этот:

```
class MiniChartWidget : public QWidget
{
public:
    enum ChartType
    {
        LineChart,
        CandleChart,
        BarChart,
        RsiChart
    };

    explicit MiniChartWidget(QWidget *parent = nullptr)
        : QWidget(parent),
          chartType(LineChart)
    {
        setMouseTracking(true);
        setMinimumHeight(150);
        setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Expanding);
    }

    void setChartType(ChartType type)
    {
        chartType = type;
        update();
    }

    void setValues(const QVector<double> &newValues)
    {
        values = newValues;
        update();
    }

protected:
    void paintEvent(QPaintEvent *) override
    {
        QPainter painter(this);
        painter.setRenderHint(QPainter::Antialiasing, true);
```

```

QRect area = chartArea();

painter.setPen(Qt::NoPen);
painter.setBrush(QColor("#121a2e"));
painter.drawRoundedRect(rect(), 18, 18);

if (values.size() < 2)
{
    painter.setPen(QColor("#8f9bb3"));
    painter.drawText(area, Qt::AlignCenter, "Недостаточно данных");
    return;
}

double minValue = values[0];
double maxValue = values[0];

for (double value : values)
{
    minValue = qMin(minValue, value);
    maxValue = qMax(maxValue, value);
}

if (chartType == RsiChart)
{
    minValue = 0.0;
    maxValue = 100.0;
}

if (qFuzzyCompare(minValue, maxValue))
{
    maxValue += 1.0;
    minValue -= 1.0;
}

drawGrid(painter, area);
drawAxisLabels(painter, area, minValue, maxValue);

if (chartType == BarChart)
    drawBars(painter, area, minValue, maxValue);
else if (chartType == CandleChart)
    drawCandles(painter, area, minValue, maxValue);
else if (chartType == RsiChart)
    drawRsi(painter, area);
else
    drawLine(painter, area, minValue, maxValue);
}

void mouseMoveEvent(QMouseEvent *event) override
{
    if (values.size() < 2)
    {
        QToolTip::hideText();
    }
}

```

```

        return;
    }

    QRect area = chartArea();

    if (!area.contains(event->pos()))
    {
        QToolTip::hideText();
        return;
    }

    int index = indexByX(event->pos().x(), area);

    if (index < 0 || index >= values.size())
    {
        QToolTip::hideText();
        return;
    }

    QString text;

    if (chartType == CandleChart)
    {
        if (index == 0)
            index = 1;

        double open = values[index - 1];
        double close = values[index];

        if (qFuzzyIsNull(open))
            return;

        double percent = (close - open) / open * 100.0;

        text =
            "Свеча №" + QString::number(index) +
            "\nOpen: " + formatChartValue(open) +
            "\nClose: " + formatChartValue(close) +
            "\nИзменение: " + QString(percent >= 0 ? "+" : "") +
            QString::number(percent, 'f', 2) + "%";
    }
    else if (chartType == BarChart)
    {
        text =
            "Столбец №" + QString::number(index + 1) +
            "\nОбъём: " + formatChartValue(values[index]);
    }
    else if (chartType == RsiChart)
    {
        text =
            "Точка №" + QString::number(index + 1) +
            "\nRSI: " + QString::number(values[index], 'f', 2);
    }
}

```

```

else
{
    text =
        "Точка №" + QString::number(index + 1) +
        "\nЦена: " + formatChartValue(values[index]);
}

QToolTip::showText(event->globalPosition().toPoint(), text, this);
}

void leaveEvent(QEvent *) override
{
    QToolTip::hideText();
}

private:
    QVector<double> values;
    ChartType chartType;

    QRect chartArea() const
    {
        return rect().adjusted(62, 18, -18, -36);
    }

    int indexByX(int x, const QRect &area) const
    {
        if (values.isEmpty())
            return -1;

        double position = static_cast<double>(x - area.left()) / area.width();
        int index = qRound(position * (values.size() - 1));

        return qBound(0, index, values.size() - 1);
    }

    QString formatChartValue(double value) const
    {
        double absValue = qAbs(value);

        if (absValue >= 10000000000.0)
            return QString::number(value / 10000000000.0, 'f', 2) + "B";

        if (absValue >= 1000000.0)
            return QString::number(value / 1000000.0, 'f', 2) + "M";

        if (absValue >= 1000.0)
            return QString::number(value, 'f', 0);

        if (absValue >= 1.0)
            return QString::number(value, 'f', 2);

        return QString::number(value, 'f', 4);
    }
}

```

```

void drawGrid(QPainter &painter, const QRect &area)
{
    painter.setPen(QPen(QColor("#26314f"), 1));

    for (int i = 0; i <= 4; ++i)
    {
        int y = area.top() + area.height() * i / 4;
        painter.drawLine(area.left(), y, area.right(), y);
    }

    for (int i = 0; i <= 6; ++i)
    {
        int x = area.left() + area.width() * i / 6;
        painter.drawLine(x, area.top(), x, area.bottom());
    }

    painter.setPen(QPen(QColor("#3b4a6b"), 1));
    painter.drawLine(area.left(), area.top(), area.left(), area.bottom());
    painter.drawLine(area.left(), area.bottom(), area.right(), area.bottom());
}

void drawAxisLabels(QPainter &painter, const QRect &area, double minValue, double maxValue)
{
    painter.setPen(QColor("#8f9bb3"));
    painter.setFont(QFont("Segoe UI", 8));

    for (int i = 0; i <= 4; ++i)
    {
        double value = maxValue - (maxValue - minValue) * i / 4.0;
        int y = area.top() + area.height() * i / 4;

        QRect labelRect(6, y - 9, area.left() - 10, 18);
        painter.drawText(labelRect, Qt::AlignRight | Qt::AlignVCenter, formatChartValue(value));
    }

    int count = values.size();

    if (count > 1)
    {
        QVector<int> marks;
        marks.append(1);
        marks.append(qMax(1, count / 2));
        marks.append(count);

        for (int mark : marks)
        {
            int index = mark - 1;
            double x = area.left() + static_cast<double>(area.width()) * index / (count - 1);

            QRect labelRect(static_cast<int>(x) - 24, area.bottom() + 8, 48, 18);
            painter.drawText(labelRect, Qt::AlignCenter, QString::number(mark));
        }
    }
}

```



```

    }
}

QPointF pointAt(int index, const QRect &area, double minValue, double maxValue)
{
    double x = area.left() + static_cast<double>(area.width()) * index / (values.size() - 1);
    double normalized = (values[index] - minValue) / (maxValue - minValue);
    double y = area.bottom() - normalized * area.height();

    return QPointF(x, y);
}

void drawLine(QPainter &painter, const QRect &area, double minValue, double maxValue)
{
    QPainterPath path;
    path.moveTo(pointAt(0, area, minValue, maxValue));

    for (int i = 1; i < values.size(); ++i)
        path.lineTo(pointAt(i, area, minValue, maxValue));

    QLinearGradient gradient(area.topLeft(), area.bottomLeft());
    gradient.setColorAt(0.0, QColor(34, 197, 94, 90));
    gradient.setColorAt(1.0, QColor(34, 197, 94, 0));

    QPainterPath fillPath = path;
    fillPath.lineTo(area.right(), area.bottom());
    fillPath.lineTo(area.left(), area.bottom());
    fillPath.closeSubpath();

    painter.fillPath(fillPath, gradient);

    painter.setPen(QPen(QColor("#22c55e"), 3));
    painter.drawPath(path);

    painter.setBrush(QColor("#22c55e"));
    painter.setPen(Qt::NoPen);

    for (int i = 0; i < values.size(); ++i)
    {
        if (values.size() > 45 && i % 4 != 0 && i != values.size() - 1)
            continue;

        painter.drawEllipse(pointAt(i, area, minValue, maxValue), 3, 3);
    }
}

void drawBars(QPainter &painter, const QRect &area, double minValue, double maxValue)
{
    Q_UNUSED(minValue);

    int count = values.size();
    double step = static_cast<double>(area.width()) / count;
    double barWidth = qMax(3.0, step * 0.55);

```

```

painter.setPen(Qt::NoPen);
painter.setBrush(QColor("#38bdf8"));

for (int i = 0; i < count; ++i)
{
    double normalized = values[i] / maxValue;
    double barHeight = normalized * area.height();

    QRectF bar(
        area.left() + i * step + (step - barWidth) / 2.0,
        area.bottom() - barHeight,
        barWidth,
        barHeight
    );

    painter.drawRoundedRect(bar, 3, 3);
}
}

void drawCandles(QPainter &painter, const QRect &area, double minValue, double maxValue)
{
    int count = values.size();
    double step = static_cast<double>(area.width()) / count;
    double candleWidth = qMax(4.0, step * 0.48);

    for (int i = 1; i < count; ++i)
    {
        double open = values[i - 1];
        double close = values[i];

        double high = qMax(open, close) * 1.004;
        double low = qMin(open, close) * 0.996;

        auto mapY = [&](double value)
        {
            double normalized = (value - minValue) / (maxValue - minValue);
            return area.bottom() - normalized * area.height();
        };

        double x = area.left() + i * step;
        QColor color = close >= open ? QColor("#22c55e") : QColor("#ef4444");

        painter.setPen(QPen(color, 2));
        painter.drawLine(QPointF(x, mapY(high)), QPointF(x, mapY(low)));

        QRectF body(
            x - candleWidth / 2.0,
            qMin(mapY(open), mapY(close)),
            candleWidth,
            qAbs(mapY(close) - mapY(open))
        );
    }
}

```

```

        if (body.height() < 3)
            body.setHeight(3);

        painter.setPen(Qt::NoPen);
        painter.setBrush(color);
        painter.drawRoundedRect(body, 3, 3);
    }
}

void drawRsi(QPainter &painter, const QRect &area)
{
    painter.setPen(QPen(QColor("#ef4444"), 1, Qt::DashLine));
    int level70 = area.bottom() - area.height() * 70 / 100;
    painter.drawLine(area.left(), level70, area.right(), level70);

    painter.setPen(QPen(QColor("#22c55e"), 1, Qt::DashLine));
    int level30 = area.bottom() - area.height() * 30 / 100;
    painter.drawLine(area.left(), level30, area.right(), level30);

    painter.setPen(QColor("#8f9bb3"));
    painter.drawText(area.left() + 4, level70 - 4, "70");
    painter.drawText(area.left() + 4, level30 - 4, "30");

    QPainterPath path;

    auto rsiPoint = [&](int index)
    {
        double x = area.left() + static_cast<double>(area.width()) * index / (values.size() - 1);
        double y = area.bottom() - values[index] / 100.0 * area.height();
        return QPointF(x, y);
    };

    path.moveTo(rsiPoint(0));

    for (int i = 1; i < values.size(); ++i)
        path.lineTo(rsiPoint(i));

    painter.setPen(QPen(QColor("#a855f7"), 3));
    painter.drawPath(path);

    painter.setBrush(QColor("#a855f7"));
    painter.setPen(Qt::NoPen);

    for (int i = 0; i < values.size(); ++i)
    {
        if (values.size() > 45 && i % 4 != 0 && i != values.size() - 1)
            continue;

        painter.drawEllipse(rsiPoint(i), 3, 3);
    }
}
};

```

Шаг 3. Пересобери

```
Сборка → Очистить проект
Сборка → Запустить qmake
Сборка → Пересобрать проект
```

Что должно измениться

После запуска:

- на графиках появятся цифры слева;
- снизу появятся номера точек;
- при наведении на свечу появится всплывающая подсказка;
- для свечи будет показано изменение в процентах;
- график станет заметно понятнее.

По поводу одинаковых свечей

Это исправим следующим этапом, но уже правильно:

Следующий этап — настоящие свечи

Сейчас у нас:

```
псевдосвечи из цен закрытия
```

Нужно перейти к:

```
open / high / low / close
```

То есть каждая свеча должна хранить:

```
цена открытия
максимум
минимум
цена закрытия
изменение %
```

Тогда 24ч, 7д, 30д и 90д будут отличаться нормально, а свечи станут технически корректными.

Prompt:

24.05.2026, 16:31:19

давай дальше, пока все отлично, и напиши вначале, что еще по планам мы хотим сделать

Response:

24.05.2026, 16:31:21

Рассматриваю следующие шаги в коде

Пользователь хочет продолжить. Сейчас мы в проекте с кодом, и нужно продумать следующий шаг: реализуем ли реальные свечи OHLC? Ранее мы уже цитировали документацию, так что возможно, веб-запросы уже не нужны, если только детали API изменились. Мы не планируем изменять код с запросами, нужно подумать о компилируемых изменениях и предоставить патчи.

Планируем дальнейшие шаги по коду

Понимаю, что нужно предоставлять код по частям, а не всю программу сразу. Текущая структура такова: в `MiniChartWidget` используются `QVector<double>`, `chartType`, и `tooltips`. В `ai_project.h` есть данные `CoinUiData` (история, объемы, RSI). Мы можем агрегировать данные для создания свечей OHLC вместо использования `CoinGecko`, что устранил лимитации. Структура `CandleUiData` добавится в заголовок.

Думал на протяжении 30s

По плану дальше осталось сделать:

1. **Нормальные свечи**, а не временные псевдосвечи из линии.
2. **Уровни поддержки/сопротивления** на графике.
3. **Реальную проверку оповещений** при обновлении цены.
4. **Сохранение оповещений и настроек** после закрытия приложения.
5. **Автообновление данных** раз в заданный интервал.
6. **Финальная полировка интерфейса** и подготовка объяснения для защиты.

Сейчас делаем пункт 1: **свечи станут технически нормальнее**. Мы будем собирать свечи из исторических точек `CoinGecko`: для каждого интервала считаем `open`, `high`, `low`, `close`. Это уже не будет выглядеть как одинаковые псевдосвечи.

Шаг 1. Полностью замени `ai_project.h`

```

#ifndef AI_PROJECT_H
#define AI_PROJECT_H

#include <QMainWindow>
#include <QVector>
#include <QString>

class QLabel;
class QListWidget;
class QPushButton;
class QComboBox;
class QLineEdit;
class QFrame;
class MiniChartWidget;
class QNetworkAccessManager;
class QNetworkReply;
class QJsonObject;

struct CandleUiData
{
    double open;
    double high;
    double low;
    double close;
};

struct CoinUiData
{
    QString id;
    QString symbol;
    QString name;

    double price;
    double changePercent;
    double volume;

    QVector<double> history;
    QVector<double> volumes;
    QVector<double> rsi;
    QVector<CandleUiData> candles;
};

class ai_project : public QMainWindow
{
    Q_OBJECT

public:
    explicit ai_project(QWidget *parent = nullptr);
    ~ai_project();

private:
    QVector<CoinUiData> coins;

```

```

QNetworkAccessManager *networkManager;

QListWidget *coinList;

QLabel *coinNameLabel;
QLabel *coinPriceLabel;
QLabel *coinChangeLabel;
QLabel *periodInfoLabel;
QLabel *lastUpdateLabel;

MiniChartWidget *priceChart;
MiniChartWidget *volumeChart;
MiniChartWidget *rsiChart;

QPushButton *period24Button;
QPushButton *period7Button;
QPushButton *period30Button;
QPushButton *period90Button;

QPushButton *lineButton;
QPushButton *candleButton;
QPushButton *refreshButton;

QComboBox *alertCoinCombo;
QComboBox *alertConditionCombo;
QLineEdit *alertPriceEdit;
QListWidget *alertsList;

QComboBox *fromCoinCombo;
QComboBox *toCoinCombo;
QLineEdit *amountEdit;
QLabel *converterResultLabel;

QString selectedCoinId;
QString selectedPeriod;
bool candleMode;

private:
    void setupInterface();
    void setupStyle();
    void fillDemoData();
    void buildCoinList();

    QFrame *createCard();
    QPushButton *createSmallButton(const QString &text);

    void selectCoin(int row);
    void updateCenterPanel(const CoinUiData &coin);
    void updateConverter();
    void addAlert();
    void refreshDemoData();

    void fetchMarketData();

```

```

void handleMarketDataReply(QNetworkReply *reply);
void updateCoinFromJson(const QJsonObject &object);

void fetchHistoryData(const QString &coinId, const QString &period);
void handleHistoryDataReply(QNetworkReply *reply, const QString &coinId, const QString &p

int daysForPeriod(const QString &period) const;
int targetPointCountForPeriod(const QString &period) const;

 QVector<double> reduceToClosingValues(const QVector<double> &source, int targetCount) con
 QVector<double> reduceToAverageValues(const QVector<double> &source, int targetCount) con
 QVector<double> calculateRsiValues(const QVector<double> &prices) const;

 QVector<CandleUiData> buildCandlesFromPrices(const QVector<double> &source, int targetCou
 QVector<double> closeValuesFromCandles(const QVector<CandleUiData> &candles) const;

int findCoinIndexById(const QString &id) const;
QString formatPrice(double value) const;
QString formatVolume(double value) const;
};

#endif

```

Шаг 2. Полностью замени класс MiniChartWidget

В `ai_project.cpp` найди весь класс:

```

class MiniChartWidget : public QWidget
{
    ...
};

```

И замени его полностью на этот:

```

class MiniChartWidget : public QWidget
{
public:
    enum ChartType
    {
        LineChart,
        CandleChart,
        BarChart,
        RsiChart
    };

    explicit MiniChartWidget(QWidget *parent = nullptr)
        : QWidget(parent),

```



```

        chartType(LineChart)
    {
        setMouseTracking(true);
        setMinimumHeight(150);
        setSizePolicy(QSizePolicy::Expanding, QSizePolicy::Expanding);
    }

void setChartType(ChartType type)
{
    chartType = type;
    update();
}

void setValues(const QVector<double> &newValues)
{
    values = newValues;
    update();
}

void setCandles(const QVector<CandleUiData> &newCandles)
{
    candles = newCandles;
    update();
}

protected:
void paintEvent(QPaintEvent *) override
{
    QPainter painter(this);
    painter.setRenderHint(QPainter::Antialiasing, true);

    QRect area = chartArea();

    painter.setPen(Qt::NoPen);
    painter.setBrush(QColor("#121a2e"));
    painter.drawRoundedRect(rect(), 18, 18);

    int visibleCount = itemCount();

    if (visibleCount < 2)
    {
        painter.setPen(QColor("#8f9bb3"));
        painter.drawText(area, Qt::AlignCenter, "Недостаточно данных");
        return;
    }

    double minValue = 0.0;
    double maxValue = 0.0;

    findValueRange(minValue, maxValue);

    if (chartType == RsiChart)
    {

```

```

        minValue = 0.0;
        maxValue = 100.0;
    }

    if (qFuzzyCompare(minValue, maxValue))
    {
        maxValue += 1.0;
        minValue -= 1.0;
    }

    drawGrid(painter, area);
    drawAxisLabels(painter, area, minValue, maxValue);

    if (chartType == BarChart)
        drawBars(painter, area, minValue, maxValue);
    else if (chartType == CandleChart)
        drawCandles(painter, area, minValue, maxValue);
    else if (chartType == RsiChart)
        drawRsi(painter, area);
    else
        drawLine(painter, area, minValue, maxValue);
}

void mouseMoveEvent(QMouseEvent *event) override
{
    QRect area = chartArea();

    if (!area.contains(event->pos()) || itemCount() < 2)
    {
        QToolTip::hideText();
        return;
    }

    int index = indexByX(event->pos().x(), area, itemCount());

    if (index < 0 || index >= itemCount())
    {
        QToolTip::hideText();
        return;
    }

    QString text;

    if (chartType == CandleChart && !candles.isEmpty())
    {
        const CandleUiData &candle = candles[index];

        if (qFuzzyIsNull(candle.open))
            return;

        double percent = (candle.close - candle.open) / candle.open * 100.0;

        QString direction;

```

```

        if (percent > 0.0)
            direction = "рост";
        else if (percent < 0.0)
            direction = "падение";
        else
            direction = "без изменения";

        text =
            "Свеча №" + QString::number(index + 1) +
            "\nOpen: " + formatChartValue(candle.open) +
            "\nHigh: " + formatChartValue(candle.high) +
            "\nLow: " + formatChartValue(candle.low) +
            "\nClose: " + formatChartValue(candle.close) +
            "\nИзменение: " + QString(percent >= 0 ? "+" : "") +
            QString::number(percent, 'f', 2) + "% (" + direction + ")";
    }
    else if (chartType == BarChart)
    {
        text =
            "Столбец №" + QString::number(index + 1) +
            "\nОбъём: " + formatChartValue(values[index]);
    }
    else if (chartType == RsiChart)
    {
        text =
            "Точка №" + QString::number(index + 1) +
            "\nRSI: " + QString::number(values[index], 'f', 2);
    }
    else
    {
        text =
            "Точка №" + QString::number(index + 1) +
            "\nЦена: " + formatChartValue(values[index]);
    }

    QToolTip::showText(event->globalPosition().toPoint(), text, this);
}

void leaveEvent(QEvent *) override
{
    QToolTip::hideText();
}

private:
    QVector<double> values;
    QVector<CandleUiData> candles;
    ChartType chartType;

    QRect chartArea() const
    {
        return rect().adjusted(62, 18, -18, -36);
    }

```

```

int itemCount() const
{
    if (chartType == CandleChart && !candles.isEmpty())
        return candles.size();

    return values.size();
}

void findValueRange(double &minValue, double &maxValue) const
{
    if (chartType == CandleChart && !candles.isEmpty())
    {
        minValue = candles[0].low;
        maxValue = candles[0].high;

        for (const CandleUiData &candle : candles)
        {
            minValue = qMin(minValue, candle.low);
            maxValue = qMax(maxValue, candle.high);
        }

        return;
    }

    minValue = values[0];
    maxValue = values[0];

    for (double value : values)
    {
        minValue = qMin(minValue, value);
        maxValue = qMax(maxValue, value);
    }
}

int indexByX(int x, const QRect &area, int count) const
{
    if (count <= 0)
        return -1;

    double position = static_cast<double>(x - area.left()) / area.width();
    int index = qRound(position * (count - 1));

    return qBound(0, index, count - 1);
}

QString formatChartValue(double value) const
{
    double absValue = qAbs(value);

    if (absValue >= 1000000000.0)
        return QString::number(value / 1000000000.0, 'f', 2) + "B";
}

```

```

        if (absValue >= 1000000.0)
            return QString::number(value / 1000000.0, 'f', 2) + "M";

        if (absValue >= 1000.0)
            return QString::number(value, 'f', 0);

        if (absValue >= 1.0)
            return QString::number(value, 'f', 2);

        return QString::number(value, 'f', 4);
    }

void drawGrid(QPainter &painter, const QRect &area)
{
    painter.setPen(QPen(QColor("#26314f"), 1));

    for (int i = 0; i <= 4; ++i)
    {
        int y = area.top() + area.height() * i / 4;
        painter.drawLine(area.left(), y, area.right(), y);
    }

    for (int i = 0; i <= 6; ++i)
    {
        int x = area.left() + area.width() * i / 6;
        painter.drawLine(x, area.top(), x, area.bottom());
    }

    painter.setPen(QPen(QColor("#3b4a6b"), 1));
    painter.drawLine(area.left(), area.top(), area.left(), area.bottom());
    painter.drawLine(area.left(), area.bottom(), area.right(), area.bottom());
}

void drawAxisLabels(QPainter &painter, const QRect &area, double minValue, double maxValu
{
    painter.setPen(QColor("#8f9bb3"));
    painter.setFont(QFont("Segoe UI", 8));

    for (int i = 0; i <= 4; ++i)
    {
        double value = maxValue - (maxValue - minValue) * i / 4.0;
        int y = area.top() + area.height() * i / 4;

        QRect labelRect(6, y - 9, area.left() - 10, 18);
        painter.drawText(labelRect, Qt::AlignRight | Qt::AlignVCenter, formatChartValue(v
    }

    int count = itemCount();

    if (count > 1)
    {
        QVector<int> marks;
        marks.append(1);
    }
}

```

```

marks.append(qMax(1, count / 2));
marks.append(count);

for (int mark : marks)
{
    int index = mark - 1;
    double x = area.left() + static_cast<double>(area.width()) * index / (count -

    QRect labelRect(static_cast<int>(x) - 24, area.bottom() + 8, 48, 18);
    painter.drawText(labelRect, Qt::AlignCenter, QString::number(mark));
}
}

double mapY(double value, const QRect &area, double minValue, double maxValue) const
{
    double normalized = (value - minValue) / (maxValue - minValue);
    return area.bottom() - normalized * area.height();
}

QPointF pointAt(int index, const QRect &area, double minValue, double maxValue)
{
    double x = area.left() + static_cast<double>(area.width()) * index / (values.size() -
    double y = mapY(values[index], area, minValue, maxValue);

    return QPointF(x, y);
}

void drawLine(QPainter &painter, const QRect &area, double minValue, double maxValue)
{
    QPainterPath path;
    path.moveTo(pointAt(0, area, minValue, maxValue));

    for (int i = 1; i < values.size(); ++i)
        path.lineTo(pointAt(i, area, minValue, maxValue));

    QLinearGradient gradient(area.topLeft(), area.bottomLeft());
    gradient.setColorAt(0.0, QColor(34, 197, 94, 90));
    gradient.setColorAt(1.0, QColor(34, 197, 94, 0));

    QPainterPath fillPath = path;
    fillPath.lineTo(area.right(), area.bottom());
    fillPath.lineTo(area.left(), area.bottom());
    fillPath.closeSubpath();

    painter.fillPath(fillPath, gradient);

    painter.setPen(QPen(QColor("#22c55e"), 3));
    painter.drawPath(path);

    painter.setBrush(QColor("#22c55e"));
    painter.setPen(Qt::NoPen);

```

```

for (int i = 0; i < values.size(); ++i)
{
    if (values.size() > 45 && i % 4 != 0 && i != values.size() - 1)
        continue;

    painter.drawEllipse(pointAt(i, area, minValue, maxValue), 3, 3);
}
}

void drawBars(QPainter &painter, const QRect &area, double minValue, double maxValue)
{
    Q_UNUSED(minValue);

    int count = values.size();

    if (count <= 0 || qFuzzyIsNull(maxValue))
        return;

    double step = static_cast<double>(area.width()) / count;
    double barWidth = qMax(3.0, step * 0.55);

    painter.setPen(Qt::NoPen);
    painter.setBrush(QColor("#38bdf8"));

    for (int i = 0; i < count; ++i)
    {
        double normalized = values[i] / maxValue;
        double barHeight = normalized * area.height();

        QRectF bar(
            area.left() + i * step + (step - barWidth) / 2.0,
            area.bottom() - barHeight,
            barWidth,
            barHeight
        );

        painter.drawRoundedRect(bar, 3, 3);
    }
}

void drawCandles(QPainter &painter, const QRect &area, double minValue, double maxValue)
{
    if (candles.isEmpty())
        return;

    int count = candles.size();
    double step = static_cast<double>(area.width()) / count;
    double candleWidth = qMax(4.0, step * 0.52);

    for (int i = 0; i < count; ++i)
    {
        const CandleUiData &candle = candles[i];
    }
}

```

```

double x = area.left() + i * step + step / 2.0;

QColor color = candle.close >= candle.open
    ? QColor("#22c55e")
    : QColor("#ef4444");

double highY = mapY(candle.high, area, minValue, maxValue);
double lowY = mapY(candle.low, area, minValue, maxValue);
double openY = mapY(candle.open, area, minValue, maxValue);
double closeY = mapY(candle.close, area, minValue, maxValue);

painter.setPen(QPen(color, 2));
painter.drawLine(QPointF(x, highY), QPointF(x, lowY));

QRectF body(
    x - candleWidth / 2.0,
    qMin(openY, closeY),
    candleWidth,
    qAbs(closeY - openY)
);

if (body.height() < 3)
    body.setHeight(3);

painter.setPen(Qt::NoPen);
painter.setBrush(color);
painter.drawRoundedRect(body, 3, 3);
}
}

void drawRsi(QPainter &painter, const QRect &area)
{
    painter.setPen(QPen(QColor("#ef4444"), 1, Qt::DashLine));
    int level70 = area.bottom() - area.height() * 70 / 100;
    painter.drawLine(area.left(), level70, area.right(), level70);

    painter.setPen(QPen(QColor("#22c55e"), 1, Qt::DashLine));
    int level30 = area.bottom() - area.height() * 30 / 100;
    painter.drawLine(area.left(), level30, area.right(), level30);

    painter.setPen(QColor("#8f9bb3"));
    painter.drawText(area.left() + 4, level70 - 4, "70");
    painter.drawText(area.left() + 4, level30 - 4, "30");

    QPainterPath path;

    auto rsiPoint = [&](int index)
    {
        double x = area.left() + static_cast<double>(area.width()) * index / (values.size() - 1);
        double y = area.bottom() - values[index] / 100.0 * area.height();
        return QPointF(x, y);
    };
};

```



```

path.moveTo(rsiPoint(0));

for (int i = 1; i < values.size(); ++i)
    path.lineTo(rsiPoint(i));

painter.setPen(QPen(QColor("#a855f7"), 3));
painter.drawPath(path);

painter.setBrush(QColor("#a855f7"));
painter.setPen(Qt::NoPen);

for (int i = 0; i < values.size(); ++i)
{
    if (values.size() > 45 && i % 4 != 0 && i != values.size() - 1)
        continue;

    painter.drawEllipse(rsiPoint(i), 3, 3);
}
};

```

Шаг 3. В `fillDemoData()` добавь демо-свечи

В функции `fillDemoData()` найди конец цикла, где сейчас примерно так:

```

        coins.append(coin);
    }
}

```

Перед строкой:

```
coins.append(coin);
```

вставь:

```

for (int i = 0; i < coin.history.size(); ++i)
{
    double open = i == 0 ? coin.history[i] * 0.998 : coin.history[i - 1];
    double close = coin.history[i];

    CandleUiData candle;
    candle.open = open;
    candle.close = close;
    candle.high = qMax(open, close) * 1.004;
    candle.low = qMin(open, close) * 0.996;

    coin.candles.append(candle);
}

```

В итоге конец цикла должен выглядеть так:

```
for (int i = 0; i < coin.history.size(); ++i)
{
    double open = i == 0 ? coin.history[i] * 0.998 : coin.history[i - 1];
    double close = coin.history[i];

    CandleUiData candle;
    candle.open = open;
    candle.close = close;
    candle.high = qMax(open, close) * 1.004;
    candle.low = qMin(open, close) * 0.996;

    coin.candles.append(candle);
}

coins.append(coin);
}
```

Шаг 4. Замени функцию `updateCenterPanel`

Найди:

```
void ai_project::updateCenterPanel(const CoinUiData &coin)
```

И полностью замени функцию на:

```
void ai_project::updateCenterPanel(const CoinUiData &coin)
{
    coinNameLabel->setText(coin.name + " (" + coin.symbol + ")");
    coinPriceLabel->setText(formatPrice(coin.price));

    QString changeText = QString("%1%2% за 24 часа")
        .arg(coin.changePercent >= 0 ? "▲ +" : "▼ ")
        .arg(coin.changePercent, 0, 'f', 2);

    coinChangeLabel->setText(changeText);
    coinChangeLabel->setObjectName(coin.changePercent >= 0 ? "PositiveText" : "NegativeText");
    coinChangeLabel->style()->unpolish(coinChangeLabel);
    coinChangeLabel->style()->polish(coinChangeLabel);

    QVector<double> chartValues = coin.history;
    QVector<double> volumeValues = coin.volumes;
    QVector<double> rsiValues = coin.rsi;
    QVector<CandleUiData> candleValues = coin.candles;

    int targetCount = targetPointCountForPeriod(selectedPeriod);
```

```

if (chartValues.size() > targetCount)
    chartValues = chartValues.mid(chartValues.size() - targetCount);

if (volumeValues.size() > targetCount)
    volumeValues = volumeValues.mid(volumeValues.size() - targetCount);

if (rsiValues.size() > targetCount)
    rsiValues = rsiValues.mid(rsiValues.size() - targetCount);

if (candleValues.size() > targetCount)
    candleValues = candleValues.mid(candleValues.size() - targetCount);

priceChart->setValues(chartValues);
priceChart->setCandles(candleValues);
volumeChart->setValues(volumeValues);
rsiChart->setValues(rsiValues);

if (candleMode)
    priceChart->setChartType(MiniChartWidget::CandleChart);
else
    priceChart->setChartType(MiniChartWidget::LineChart);

lastUpdateLabel->setText("Последнее обновление: " +
                        QDateTime::currentDateTime().toString("dd.MM.yyyy HH:mm:ss"));
}

```

Шаг 5. В `updateCoinFromJson()` добавь обновление последней свечи

В функции `updateCoinFromJson()` найди этот блок:

```

if (!coins[index].history.isEmpty())
{
    coins[index].history.removeFirst();
    coins[index].history.append(newPrice);
}

```

Сразу после него вставь:

```

if (!coins[index].candles.isEmpty())
{
    double open = coins[index].candles.last().close;

    CandleUiData candle;
    candle.open = open;
    candle.close = newPrice;
    candle.high = qMax(open, newPrice) * 1.002;
    candle.low = qMin(open, newPrice) * 0.998;
}

```

```
coins[index].candles.removeFirst();  
coins[index].candles.append(candle);  
}
```

Шаг 6. Замени часть В handleHistoryDataReply

В функции `handleHistoryDataReply()` найди блок:

```
QVector<double> preparedPrices = reduceToClosingValues(rawPrices, targetCount);  
QVector<double> preparedVolumes = reduceToAverageValues(rawVolumes, targetCount);  
  
coins[coinIndex].history = preparedPrices;  
coins[coinIndex].volumes = preparedVolumes;  
coins[coinIndex].rsi = calculateRsiValues(preparedPrices);  
  
if (!preparedPrices.isEmpty())  
    coins[coinIndex].price = preparedPrices.last();
```

Замени его на:

```
QVector<CandleUiData> preparedCandles = buildCandlesFromPrices(rawPrices, targetCount);  
QVector<double> preparedPrices = closeValuesFromCandles(preparedCandles);  
QVector<double> preparedVolumes = reduceToAverageValues(rawVolumes, targetCount);  
  
if (preparedPrices.isEmpty())  
    preparedPrices = reduceToClosingValues(rawPrices, targetCount);  
  
coins[coinIndex].candles = preparedCandles;  
coins[coinIndex].history = preparedPrices;  
coins[coinIndex].volumes = preparedVolumes;  
coins[coinIndex].rsi = calculateRsiValues(preparedPrices);  
  
if (!preparedPrices.isEmpty())  
    coins[coinIndex].price = preparedPrices.last();
```

Шаг 7. В конец `ai_project.cpp` добавь две новые функции

В самый конец файла добавь:

```
QVector<CandleUiData> ai_project::buildCandlesFromPrices(const QVector<double> &source, int t  
{  
    QVector<CandleUiData> result;
```

```

if (source.isEmpty() || targetCount <= 0)
    return result;

if (source.size() < targetCount)
    targetCount = source.size();

for (int i = 0; i < targetCount; ++i)
{
    int start = static_cast<int>(static_cast<double>(i) * source.size() / targetCount);
    int end = static_cast<int>(static_cast<double>(i + 1) * source.size() / targetCount);

    if (end <= start)
        end = start + 1;

    if (end > source.size())
        end = source.size();

    CandleUiData candle;
    candle.open = source[start];
    candle.close = source[end - 1];
    candle.high = source[start];
    candle.low = source[start];

    for (int j = start; j < end; ++j)
    {
        candle.high = qMax(candle.high, source[j]);
        candle.low = qMin(candle.low, source[j]);
    }

    result.append(candle);
}

return result;
}

 QVector<double> ai_project::closeValuesFromCandles(const QVector<CandleUiData> &candles) const
{
    QVector<double> result;

    for (const CandleUiData &candle : candles)
        result.append(candle.close);

    return result;
}

```

Шаг 8. Пересобери

Сборка → Очистить проект

Сборка → Запустить qmake

Что должно измениться

Теперь свечи будут строиться не так:

предыдущая цена → текущая цена

А так:

open = первая цена внутри интервала
high = максимум внутри интервала
low = минимум внутри интервала
close = последняя цена внутри интервала

То есть для периода:

24ч → 24 свечи
7д → 42 свечи
30д → 30 свечей
90д → 90 свечей

Свечи должны отличаться заметнее, особенно на 7д / 30д / 90д. При наведении теперь будет показываться не только процент, но и Open / High / Low / Close.

Prompt:

24.05.2026, 16:40:52

все супер, давай дальше что осталось нам сделать

Response:

24.05.2026, 16:40:56

Продолжение работы над проектом и следующими шагами

Пользователь хочет знать следующие шаги. Похоже, не нужно использовать веб — речь идет о текущем проекте. Очень вероятно, предстоит добавить уровни поддержки/сопротивления и возможные оповещения. Чтобы это сделать, нужно будет изменить MiniChartWidget для работы с горизонтальными уровнями. Сложности могут возникнуть, но всё выполнимо.

Обновление панели и добавление уровней поддержки и сопротивления

Нужно обновить панель, чтобы она вычисляла уровни поддержки и сопротивления, используя функции `findSupportResistance`. В заголовке стоит объявить функции, такие как `calculateSupportLevels` и `calculateResistanceLevels`. Также нужно использовать `QVector` и, возможно, `std::sort` для сортировки. Уровни должны отображаться на графике с подписью "S1", "R1" и т.д.

Думал на протяжении 8s

Осталось довести проект до состояния нормальной курсовой:

1. **Уровни поддержки и сопротивления** на графике — делаем сейчас.
2. **Реальные оповещения**: не просто список, а проверка условия при обновлении цены.
3. **Автообновление** раз в 60–120 секунд.
4. **Сохранение оповещений и настроек** в файл.
5. **Финальная полировка интерфейса**: аккуратнее подписи, результат конвертора, мелкие отступы.
6. **Текст для защиты**: что делает приложение, как устроено API, как считаются RSI/свечи/уровни.

Сейчас делаем **уровни поддержки и сопротивления**. Это хороший следующий шаг, потому что преподаватель прямо предлагал “уровни поддержки, свечи и т.п.”.

Что добавим

На основном графике появятся горизонтальные линии:

S1, S2 — уровни поддержки
R1, R2 — уровни сопротивления

Логика будет такая:

- поддержка ищется по локальным минимумам;
- сопротивление ищется по локальным максимумам;
- берём последние значимые уровни;
- рисуем их поверх графика цены.

На защите это можно объяснить так:

Уровни поддержки и сопротивления рассчитываются по локальным минимумам и максимумам выбранного периода. Поддержка показывает области, где цена раньше разворачивалась вверх, сопротивление — области, где цена раньше разворачивалась вниз.

Шаг 1. В ai_project.cpp добавь include

В верхний блок `#include` добавь:

```
#include <algorithm>
```

Проще: если у тебя вверху сейчас блок `include`, просто поставь эту строку сразу после:

```
#include "ai_project.h"
```

Шаг 2. В ai_project.h добавь две функции

В `ai_project.h` найди ЭТОТ блок:

```
QVector<CandleUiData> buildCandlesFromPrices(const QVector<double> &source, int targetCount)
QVector<double> closeValuesFromCandles(const QVector<CandleUiData> &candles) const;
```

Сразу после него добавь:

```
QVector<double> calculateSupportLevels(const QVector<double> &prices) const;
QVector<double> calculateResistanceLevels(const QVector<double> &prices) const;
```

Должно получиться так:

```
QVector<CandleUiData> buildCandlesFromPrices(const QVector<double> &source, int targetCount)
QVector<double> closeValuesFromCandles(const QVector<CandleUiData> &candles) const;

QVector<double> calculateSupportLevels(const QVector<double> &prices) const;
QVector<double> calculateResistanceLevels(const QVector<double> &prices) const;
```

Шаг 3. В класс MiniChartWidget добавь поля уровней

В `ai_project.cpp` внутри класса `MiniChartWidget` найди:

```
private:
    QVector<double> values;
    QVector<CandleUiData> candles;
    ChartType chartType;
```


Замени на:

```
private:
    QVector<double> values;
    QVector<CandleUiData> candles;
    QVector<double> supportLevels;
    QVector<double> resistanceLevels;
    ChartType chartType;
```

Шаг 4. В класс `MiniChartWidget` добавь метод установки уровней

Внутри `MiniChartWidget`, рядом с методами:

```
void setValues(...)
void setCandles(...)
```

добавь этот метод:

```
void setSupportResistanceLevels(const QVector<double> &supports,
                                const QVector<double> &resistances)
{
    supportLevels = supports;
    resistanceLevels = resistances;
    update();
}
```

Лучше вставить сразу после:

```
void setCandles(const QVector<CandleUiData> &newCandles)
{
    candles = newCandles;
    update();
}
```

Шаг 5. В `paintEvent()` добавь рисование уровней

Внутри `MiniChartWidget::paintEvent()` найди этот блок:

```
if (chartType == BarChart)
    drawBars(painter, area, minValue, maxValue);
else if (chartType == CandleChart)
    drawCandles(painter, area, minValue, maxValue);
else if (chartType == RsiChart)
    drawRsi(painter, area);
else
    drawLine(painter, area, minValue, maxValue);
```

Замени его на:

```
if (chartType == BarChart)
{
    drawBars(painter, area, minValue, maxValue);
}
else if (chartType == CandleChart)
{
    drawCandles(painter, area, minValue, maxValue);
    drawSupportResistance(painter, area, minValue, maxValue);
}
else if (chartType == RsiChart)
{
    drawRsi(painter, area);
}
else
{
    drawLine(painter, area, minValue, maxValue);
    drawSupportResistance(painter, area, minValue, maxValue);
}
```

Шаг 6. В `MiniChartWidget` добавь функцию рисования уровней

Внутри класса `MiniChartWidget`, в приватную часть, например перед функцией:

```
void drawRsi(QPainter &painter, const QRect &area)
```

добавь:

```
void drawSupportResistance(QPainter &painter, const QRect &area, double minValue, double maxV
{
    if (supportLevels.isEmpty() && resistanceLevels.isEmpty())
        return;

    painter.setFont(QFont("Segoe UI", 8, QFont::Bold));
```

```

for (int i = 0; i < supportLevels.size(); ++i)
{
    double level = supportLevels[i];

    if (level < minValue || level > maxValue)
        continue;

    double y = mapY(level, area, minValue, maxValue);

    painter.setPen(QPen(QColor("#22c55e"), 1, Qt::DashLine));
    painter.drawLine(area.left(), y, area.right(), y);

    QRect labelRect(area.right() - 90, static_cast<int>(y) - 12, 84, 22);

    painter.setPen(Qt::NoPen);
    painter.setBrush(QColor(34, 197, 94, 40));
    painter.drawRoundedRect(labelRect, 7, 7);

    painter.setPen(QColor("#86efac"));
    painter.drawText(labelRect, Qt::AlignCenter,
        "S" + QString::number(i + 1) + " " + formatChartValue(level));
}

for (int i = 0; i < resistanceLevels.size(); ++i)
{
    double level = resistanceLevels[i];

    if (level < minValue || level > maxValue)
        continue;

    double y = mapY(level, area, minValue, maxValue);

    painter.setPen(QPen(QColor("#ef4444"), 1, Qt::DashLine));
    painter.drawLine(area.left(), y, area.right(), y);

    QRect labelRect(area.right() - 90, static_cast<int>(y) - 12, 84, 22);

    painter.setPen(Qt::NoPen);
    painter.setBrush(QColor(239, 68, 68, 40));
    painter.drawRoundedRect(labelRect, 7, 7);

    painter.setPen(QColor("#fecaca"));
    painter.drawText(labelRect, Qt::AlignCenter,
        "R" + QString::number(i + 1) + " " + formatChartValue(level));
}
}

```

Шаг 7. В `updateCenterPanel()` добавь расчёт уровней

В функции `updateCenterPanel()` найди блок:

```
priceChart->setValues(chartValues);  
priceChart->setCandles(candleValues);  
volumeChart->setValues(volumeValues);  
rsiChart->setValues(rsiValues);
```

Замени его на:

```
QVector<double> supportLevels = calculateSupportLevels(chartValues);  
QVector<double> resistanceLevels = calculateResistanceLevels(chartValues);  
  
priceChart->setValues(chartValues);  
priceChart->setCandles(candleValues);  
priceChart->setSupportResistanceLevels(supportLevels, resistanceLevels);  
  
volumeChart->setValues(volumeValues);  
rsiChart->setValues(rsiValues);
```

Шаг 8. В конец `ai_project.cpp` добавь расчёт уровней

В самый конец файла добавь эти две функции:

```
QVector<double> ai_project::calculateSupportLevels(const QVector<double> &prices) const  
{  
    QVector<double> levels;  
  
    if (prices.size() < 5)  
        return levels;  
  
    for (int i = 2; i < prices.size() - 2; ++i)  
    {  
        bool isLocalMinimum =  
            prices[i] < prices[i - 1] &&  
            prices[i] < prices[i - 2] &&  
            prices[i] <= prices[i + 1] &&  
            prices[i] <= prices[i + 2];  
  
        if (isLocalMinimum)  
            levels.append(prices[i]);  
    }  
  
    if (levels.isEmpty())  
        return levels;  
  
    std::sort(levels.begin(), levels.end());
```

```

QVector<double> compactLevels;

for (double level : levels)
{
    bool tooClose = false;

    for (double existing : compactLevels)
    {
        double differencePercent = qAbs(level - existing) / existing * 100.0;

        if (differencePercent < 0.8)
        {
            tooClose = true;
            break;
        }
    }

    if (!tooClose)
        compactLevels.append(level);
}

while (compactLevels.size() > 2)
    compactLevels.removeFirst();

return compactLevels;
}

QVector<double> ai_project::calculateResistanceLevels(const QVector<double> &prices) const
{
    QVector<double> levels;

    if (prices.size() < 5)
        return levels;

    for (int i = 2; i < prices.size() - 2; ++i)
    {
        bool isLocalMaximum =
            prices[i] > prices[i - 1] &&
            prices[i] > prices[i - 2] &&
            prices[i] >= prices[i + 1] &&
            prices[i] >= prices[i + 2];

        if (isLocalMaximum)
            levels.append(prices[i]);
    }

    if (levels.isEmpty())
        return levels;

    std::sort(levels.begin(), levels.end());

    QVector<double> compactLevels;

```

```
for (double level : levels)
{
    bool tooClose = false;

    for (double existing : compactLevels)
    {
        double differencePercent = qAbs(level - existing) / existing * 100.0;

        if (differencePercent < 0.8)
        {
            tooClose = true;
            break;
        }
    }

    if (!tooClose)
        compactLevels.append(level);
}

while (compactLevels.size() > 2)
    compactLevels.removeLast();

return compactLevels;
}
```

Шаг 9. Пересобери

Сборка → Очистить проект
Сборка → Запустить qmake
Сборка → Пересобрать проект

Что должно получиться

На графике цены появятся:

S1, S2 — зелёные уровни поддержки
R1, R2 — красные уровни сопротивления

Они будут меняться при:

- выборе другой монеты;
- переключении периода;
- переключении линии/свечей;
- обновлении исторических данных.

Это уже выглядит как аналитический инструмент, а не просто просмотр курса.

Prompt:

24.05.2026, 16:47:31

я бы только сделал, чтобы все эти вот s1 s2 r1 r2 по кнопке появлялись и пропадали, чтобы если вдруг хочется посмотреть на график чистым, чтобы без этого было, а так, если есть еще какие исправления, то давай

Response:

24.05.2026, 16:47:34

Оценка изменений кода по запросу

Нужно предоставить исправления в коде, добавив кнопку для уровней и поддержку/сопротивление. Убедимся, что мини-график правильно обновляется, добавив `levelsButton` в список и модификацию конструктора. Учитываем состояние текущего кода и добавляем соответствующие изменения для кнопок и отображения уровней.

Определение поведения кнопки "Уровни"

Пользователь предпочитает чистый интерфейс, но также хочет видеть кнопки, которые могут появляться и исчезать. Я думаю, что будет лучше по умолчанию установить "включено" для этого, чтобы кнопка была видимой. Кнопка будет активирована при нажатии и переключать отображение уровней в графике.

Думал на протяжении 8s

Да, это правильное исправление. Постоянные `s1/s2/r1/r2` перегружают график. Лучше сделать кнопку “Уровни”, которая включает/выключает поддержку и сопротивление.

Заодно это хорошо для защиты: можно показать, что аналитические слои графика управляются пользователем.

Что сейчас добавим

В центральной панели рядом с кнопками:

Линия | Свечи

появится кнопка:

Уровни

Если кнопка активна — на графике видны $S1/S2/R1/R2$.

Если выключить — график чистый.

Шаг 1. В `ai_project.h` добавь кнопку и флаг

Найди блок:

```
QPushButton *lineButton;
QPushButton *candleButton;
QPushButton *refreshButton;
```

Замени его на:

```
QPushButton *lineButton;
QPushButton *candleButton;
QPushButton *levelsButton;
QPushButton *refreshButton;
```

Теперь ниже найди:

```
QString selectedCoinId;
QString selectedPeriod;
bool candleMode;
```

Замени на:

```
QString selectedCoinId;
QString selectedPeriod;
bool candleMode;
bool showLevels;
```

Шаг 2. В конструкторе добавь инициализацию

В `ai_project.cpp` найди начало конструктора:

```
period90Button(nullptr),

lineButton(nullptr),
```



```
candleButton(nullptr),  
refreshButton(nullptr),
```

Замени на:

```
period90Button(nullptr),  
  
lineButton(nullptr),  
candleButton(nullptr),  
levelsButton(nullptr),  
refreshButton(nullptr),
```

Ниже в конце списка инициализации сейчас должно быть примерно так:

```
selectedPeriod("24ч"),  
candleMode(false)
```

Замени на:

```
selectedPeriod("24ч"),  
candleMode(false),  
showLevels(true)
```

Я оставляю уровни **включёнными по умолчанию**, чтобы преподаватель сразу увидел аналитику. Но пользователь сможет убрать их одной кнопкой.

Шаг 3. В `setupInterface()` создай кнопку уровней

В функции `setupInterface()` найди блок:

```
lineButton = createSmallButton("Линия");  
candleButton = createSmallButton("Свечи");  
  
lineButton->setCheckable(true);  
candleButton->setCheckable(true);  
lineButton->setChecked(true);
```

Замени его на:

```
lineButton = createSmallButton("Линия");  
candleButton = createSmallButton("Свечи");  
levelsButton = createSmallButton("Уровни");  
  
lineButton->setCheckable(true);
```

```
candleButton->setCheckable(true);  
levelsButton->setCheckable(true);  
  
lineButton->setChecked(true);  
levelsButton->setChecked(true);
```

Шаг 4. Добавь кнопку в панель управления

Найди блок:

```
controlLayout->addWidget(lineButton);  
controlLayout->addWidget(candleButton);  
controlLayout->addStretch();  
controlLayout->addWidget(periodInfoLabel);
```

Замени на:

```
controlLayout->addWidget(lineButton);  
controlLayout->addWidget(candleButton);  
controlLayout->addWidget(levelsButton);  
controlLayout->addStretch();  
controlLayout->addWidget(periodInfoLabel);
```

Шаг 5. Добавь обработчик кнопки

В `setupInterface()` найди обработчик кнопки `candleButton`:

```
connect(candleButton, &QPushButton::clicked, this, [this]()  
{  
    candleMode = true;  
    priceChart->setChartType(MiniChartWidget::CandleChart);  
});
```

Сразу после него вставь:

```
connect(levelsButton, &QPushButton::clicked, this, [this]()  
{  
    showLevels = levelsButton->isChecked();  
  
    int currentRow = coinList->currentRow();  
  
    if (currentRow >= 0)  
        selectCoin(currentRow);  
});
```

Шаг 6. Измени `updateCenterPanel()`

В функции `updateCenterPanel()` найди блок, который мы добавляли для уровней:

```
QVector<double> supportLevels = calculateSupportLevels(chartValues);
QVector<double> resistanceLevels = calculateResistanceLevels(chartValues);

priceChart->setValues(chartValues);
priceChart->setCandles(candleValues);
priceChart->setSupportResistanceLevels(supportLevels, resistanceLevels);
```

Замени на:

```
QVector<double> supportLevels;
QVector<double> resistanceLevels;

if (showLevels)
{
    supportLevels = calculateSupportLevels(chartValues);
    resistanceLevels = calculateResistanceLevels(chartValues);
}

priceChart->setValues(chartValues);
priceChart->setCandles(candleValues);
priceChart->setSupportResistanceLevels(supportLevels, resistanceLevels);
```

Теперь, если кнопка `Уровни` выключена, в график передаются пустые массивы, и линии не рисуются.

Шаг 7. Пересобери

```
Сборка → Очистить проект
Сборка → Запустить qmake
Сборка → Пересобрать проект
```

Что должно получиться

В верхней панели будет:

```
24ч  7д  30д  90д      Линия  Свечи  Уровни
```

Кнопка “Уровни”:

- синяя/активная — уровни поддержки и сопротивления видны;
- обычная/неактивная — график чистый.

Что делаем следующим этапом

Следующее самое логичное — **реальные оповещения**.

Сейчас оповещения только добавляются в список. Но они ещё не работают как механизм проверки цены.

Нужно сделать так:

```
Пользователь добавил:  
BTC выше 70000  
  
При каждом обновлении цены приложение проверяет:  
если BTC >= 70000 → показать окно уведомления и отключить это оповещение.
```

Это будет уже настоящая пользовательская функция, а не просто интерфейсная заготовка.

Prompt:

24.05.2026, 16:56:25

давай приступим к реальным оповещениям

Response:

24.05.2026, 16:56:32

Разработка патчей для системы уведомлений

Нужно добавить структуру Alert и интегрировать её в существующий код. Важно обновить существующие данные и добавить функции для обработки активных оповещений, хранения состояния и отображения сообщений через QMessageBox. Также стоит внедрить проверки на активность и условие триггеров для оповещений после обновления данных.

Обновление обработки оповещений цен

При обновлении данных нужно проверять, если цена превышает целевую или ниже, установить статус "неактивный" и собрать оповещения. Я обновлю структуру данных и функции для обработки оповещений в UI, добавлю контроль статуса активности, а

также обеспечу правильное обновление списков оповещений без удаления старых данных.

Думал на протяжении 6s

Приступаем. Сейчас сделаем так, чтобы оповещения стали реальными:

1. Пользователь выбирает монету.
2. Выбирает условие: цена выше / цена ниже.
3. Вводит целевую цену.
4. Оповещение попадает в список как активное.
5. При каждом обновлении данных CoinGecko приложение проверяет текущие цены.
6. Если условие выполнено — появляется окно уведомления.
7. Сработавшее оповещение становится неактивным.

После этого останется: автообновление, сохранение оповещений/настроек, финальная полировка.

Шаг 1. В `ai_project.h` добавь структуру оповещения

В `ai_project.h` после структуры `CoinUiData` добавь:

```
struct PriceAlertUiData
{
    QString coinId;
    QString coinSymbol;

    double targetPrice;
    bool aboveCondition;
    bool active;
};
```

То есть должно быть примерно так:

```
struct CoinUiData
{
    QString id;
    QString symbol;
    QString name;

    double price;
    double changePercent;
    double volume;

    QVector<double> history;
```

```
    QVector<double> volumes;  
    QVector<double> rsi;  
    QVector<CandleUiData> candles;  
};  
  
struct PriceAlertUiData  
{  
    QString coinId;  
    QString coinSymbol;  
  
    double targetPrice;  
    bool aboveCondition;  
    bool active;  
};
```

Шаг 2. В ai_project.h добавь массив оповещений

Найди:

```
QVector<CoinUiData> coins;
```

Сразу после него добавь:

```
QVector<PriceAlertUiData> priceAlerts;
```

Должно получиться:

```
QVector<CoinUiData> coins;  
QVector<PriceAlertUiData> priceAlerts;  
  
QNetworkAccessManager *networkManager;
```

Шаг 3. В ai_project.h добавь функции для оповещений

В приватной части найди:

```
void updateConverter();  
void addAlert();  
void refreshDemoData();
```

Замени на:

```
void updateConverter();
void addAlert();
void refreshAlertsList();
void checkPriceAlerts();
QString alertText(const PriceAlertUiData &alert) const;
void refreshDemoData();
```

Шаг 4. Замени функцию `addAlert()`

В `ai_project.cpp` найди старую функцию:

```
void ai_project::addAlert()
{
    ...
}
```

И полностью замени её на:

```
void ai_project::addAlert()
{
    QString coinId = alertCoinCombo->currentData().toString();
    QString coinSymbol = alertCoinCombo->currentText();

    bool ok = false;
    double targetPrice = alertPriceEdit->text().replace(",", ".").toDouble(&ok);

    if (!ok || targetPrice <= 0.0)
    {
        QMessageBox::warning(this, "Ошибка", "Введите корректную целевую цену.");
        return;
    }

    PriceAlertUiData alert;
    alert.coinId = coinId;
    alert.coinSymbol = coinSymbol;
    alert.targetPrice = targetPrice;
    alert.aboveCondition = alertConditionCombo->currentIndex() == 0;
    alert.active = true;

    priceAlerts.append(alert);

    alertPriceEdit->clear();
    refreshAlertsList();

    QMessageBox::information(
        this,
```

```
"Оповещение добавлено",
"Оповещение успешно добавлено:\n\n" + alertText(alert)

);
}
```

Шаг 5. В конец ai_project.cpp добавь функции оповещений

В самый конец файла добавь:

```
QString ai_project::alertText(const PriceAlertUiData &alert) const
{
    QString conditionText = alert.aboveCondition ? "выше" : "ниже";
    QString statusText = alert.active ? "активно" : "сработало";

    return QString("%1: цена %2 %3 USD [%4]")
        .arg(alert.coinSymbol)
        .arg(conditionText)
        .arg(alert.targetPrice, 0, 'f', 2)
        .arg(statusText);
}

void ai_project::refreshAlertsList()
{
    alertsList->clear();

    if (priceAlerts.isEmpty())
    {
        alertsList->addItem("Активных оповещений нет");
        return;
    }

    for (const PriceAlertUiData &alert : priceAlerts)
    {
        QListWidgetItem *item = new QListWidgetItem(alertText(alert));

        if (alert.active)
        {
            item->setForeground(QColor("#e5e7eb"));
        }
        else
        {
            item->setForeground(QColor("#64748b"));
        }

        alertsList->addItem(item);
    }
}
```



```
void ai_project::checkPriceAlerts()
{
    QStringList triggeredAlerts;

    for (PriceAlertUiData &alert : priceAlerts)
    {
        if (!alert.active)
            continue;

        int coinIndex = findCoinIndexById(alert.coinId);

        if (coinIndex < 0)
            continue;

        double currentPrice = coins[coinIndex].price;

        bool triggered = false;

        if (alert.aboveCondition && currentPrice >= alert.targetPrice)
            triggered = true;

        if (!alert.aboveCondition && currentPrice <= alert.targetPrice)
            triggered = true;

        if (triggered)
        {
            alert.active = false;

            QString conditionText = alert.aboveCondition ? "выше" : "ниже";

            triggeredAlerts.append(
                QString("%1: текущая цена %2 USD, условие: цена %3 %4 USD")
                    .arg(alert.coinSymbol)
                    .arg(currentPrice, 0, 'f', 4)
                    .arg(conditionText)
                    .arg(alert.targetPrice, 0, 'f', 4)
            );
        }
    }

    refreshAlertsList();

    if (!triggeredAlerts.isEmpty())
    {
        QMessageBox::information(
            this,
            "Сработало оповещение",
            "Достигнут заданный уровень цены:\n\n" +
            triggeredAlerts.join("\n")
        );
    }
}
```

Шаг 6. В `handleMarketDataReply()` добавь проверку оповещений

В функции `handleMarketDataReply()` найди конец успешной обработки:

```
lastUpdateLabel->setText(  
    "Последнее обновление: " +  
    QDateTime::currentDateTime().toString("dd.MM.yyyy HH:mm:ss")  
);
```

Сразу после этого добавь:

```
checkPriceAlerts();
```

Должно стать так:

```
lastUpdateLabel->setText(  
    "Последнее обновление: " +  
    QDateTime::currentDateTime().toString("dd.MM.yyyy HH:mm:ss")  
);  
  
checkPriceAlerts();
```

Шаг 7. После создания интерфейса сразу обнови список оповещений

В конструкторе найди блок:

```
buildCoinList();  
  
if (!coins.isEmpty())
```

Замени на:

```
buildCoinList();  
refreshAlertsList();  
  
if (!coins.isEmpty())
```

Шаг 8. Пересобери

```
Сборка → Очистить проект
Сборка → Запустить qmake
Сборка → Пересобрать проект
```

Как проверить

Проверка простая:

- 1. Посмотри текущую цену BTC.
- 2. Добавь оповещение чуть ниже текущей цены:

```
BTC цена выше 1000
```

- 3. Нажми `Обновить`.

Оповещение должно сразу сработать.

Для проверки нижнего условия:

```
BTC цена ниже 999999
```

Тоже должно сработать сразу, потому что текущая цена явно ниже.

Что изменится в приложении

Теперь список оповещений будет показывать статус:

```
BTC: цена выше 70000.00 USD [активно]
ETH: цена ниже 3000.00 USD [сработало]
```

Сработавшие оповещения не удаляются, а становятся неактивными. Это лучше для курсовой: видно историю срабатываний, а не просто исчезновение записи.

Prompt:

24.05.2026, 17:00:37

что еще нужно реализовать? или мы полностью завершили проект?

Response:

24.05.2026, 17:00:41

Размышляю о завершении разработки

Похоже, что ядро функционально готово, но ещё не завершено на 100%. Нужно подумать про автоперенос обновлений, сохранение настроек альбертов и возможность удаления. В UI ещё есть доработка. Если сказать пользователю, что приложение не завершено, это может быть честно — но стоит ли дать рекомендации по финишному этапу или показать код?

Думал на протяжении 7s

Полностью завершённым я бы проект **ещё не считал**. Сейчас у нас уже есть рабочее ядро курсовой, но до “законченного приложения” нужно доделать несколько вещей.

Что уже реализовано

Сейчас проект уже выглядит достойно:

- современный тёмный интерфейс;
- левая панель с 7 криптовалютами;
- загрузка реальных текущих цен через CoinGecko;
- выбор монеты;
- периоды 24ч / 7д / 30д / 90д;
- реальная история цены;
- линейный график;
- свечной график;
- объёмы торгов;
- RSI;
- уровни поддержки и сопротивления;
- кнопка включения/выключения уровней;
- конвертор криптовалют;
- реальные оповещения по цене;
- обработка ошибки API/VPN/сети.

То есть основная идея, которую ты согласовывал с преподавателем, уже покрыта: анализ криптовалют, графики, RSI, свечи, уровни, оповещения и конвертор.

Что ещё желательно реализовать

1. Автообновление данных

Сейчас пользователь нажимает `Обновить` вручную. Лучше добавить автоматическое обновление, например раз в 60 секунд.

Это важно, потому что оповещения должны срабатывать не только при ручном обновлении, а сами.

На защите это звучит сильнее:

Приложение периодически обновляет рыночные данные и автоматически проверяет условия пользовательских оповещений.

2. Сохранение оповещений

Сейчас оповещения живут только пока приложение открыто. Закрыл программу — список потерялся.

Нужно сохранять их в файл:

```
alerts.json
```

Тогда после перезапуска приложения активные и сработавшие оповещения будут восстановлены.

3. Сохранение настроек

Желательно сохранять:

- последнюю выбранную монету;
- выбранный период;
- тип графика: линия/свечи;
- включены ли уровни;
- возможно, последнюю пару в конверторе.

Файл:

```
settings.json
```

Это мелочь, но она делает приложение завершённым.

4. Управление оповещениями

Сейчас оповещение можно добавить, но нельзя удалить.

Нужно добавить хотя бы одну кнопку:

```
Удалить выбранное
```

И, возможно:

Очистить сработавшие

Иначе список со временем превратится в мусор.

5. Финальная полировка интерфейса

Не критично, но желательно:

- сделать результат конвертора компактнее;
- добавить подпись, что цены в USD;
- добавить статус загрузки истории отдельно от статуса обновления цены;
- убрать слишком частые всплывающие окна при массовом срабатывании оповещений;
- сделать нормальное сообщение “нет активных оповещений”.

6. Защитная часть

Это не код, но для курсовой обязательно нужно подготовить:

- описание назначения приложения;
- описание интерфейса;
- описание используемого API;
- объяснение структуры программы;
- объяснение расчёта RSI;
- объяснение свечей `open/high/low/close` ;
- объяснение поддержки/сопротивления;
- объяснение оповещений;
- возможные ограничения: зависимость от интернета, лимиты CoinGecko, VPN может мешать запросам.

Моя оценка текущего состояния

Сейчас проект примерно на уровне:

75–80% готовности

Для демонстрации преподавателю — уже можно показывать.

Для финальной сдачи — лучше доделать:

1. автообновление;
2. сохранение оповещений;
3. удаление оповещений;
4. сохранение настроек.

Самый правильный следующий шаг — **автообновление + проверка оповещений по таймеру**. Это маленькая доработка, но она сразу делает приложение живым, а не просто ручным просмотрщиком курсов.

Prompt:

24.05.2026, 17:01:30

давай тогда дальше продолжим

Response:

24.05.2026, 17:01:35

Оценка изменений в коде для автообновления

Мне нужно продолжить работать над кодом с автообновлением, возможно, без использования веба. Нужно добавить функциональность с QTimer, установить кнопку для включения/выключения и выбрать фиксированный интервал. Лучше постепенно внедрять изменения и не перегружать код. Начну с автообновления и легко добавлю дополнительные детали по мере необходимости.

Разработка интерфейса с кнопкой автообновления и защитой от загрузок

Мне нужно добавить в интерфейс кнопку «Автообновление», которая будет переключаемая. Самое главное — избежать превышения лимитов запросов, например, CoinGecko. Возможно, для простоты размещу кнопку рядом с кнопкой обновления. Timer будет управлять запросами и их частотой.

Думал на протяжении 7s

Продолжаем с **автообновления**.

Сейчас приложение обновляет цены только по кнопке. После этой доработки будет:

1. Кнопка "Автообновление" в верхней части окна.
2. Если включить — приложение само обновляет данные раз в 60 секунд.
3. При каждом автообновлении проверяются реальные оповещения.
4. Если цена достигла заданного уровня — появляется уведомление.
5. Если выключить — остаётся только ручная кнопка "Обновить".

Это логичный следующий шаг после реальных оповещений.

Шаг 1. В ai_project.h добавь QTimer

В верхней части ai_project.h , где идут объявления классов:

```
class QNetworkAccessManager;  
class QNetworkReply;  
class QJsonObject;
```

Добавь ниже:

```
class QTimer;
```

Должно стать так:

```
class QNetworkAccessManager;  
class QNetworkReply;  
class QJsonObject;  
class QTimer;
```

Шаг 2. В ai_project.h добавь поле таймера

Найди:

```
QNetworkAccessManager *networkManager;
```

Сразу после него добавь:

```
QTimer *autoUpdateTimer;
```

Должно быть так:

```
QNetworkAccessManager *networkManager;  
QTimer *autoUpdateTimer;
```

Шаг 3. В ai_project.h добавь кнопку автообновления

Найди блок:


```
QPushButton *lineButton;  
QPushButton *candleButton;  
QPushButton *levelsButton;  
QPushButton *refreshButton;
```

Замени на:

```
QPushButton *lineButton;  
QPushButton *candleButton;  
QPushButton *levelsButton;  
QPushButton *refreshButton;  
QPushButton *autoUpdateButton;
```

Шаг 4. В `ai_project.h` добавь функцию переключения автообновления

Найди блок:

```
void checkPriceAlerts();  
QString alertText(const PriceAlertUiData &alert) const;  
void refreshDemoData();
```

Замени на:

```
void checkPriceAlerts();  
QString alertText(const PriceAlertUiData &alert) const;  
  
void toggleAutoUpdate();  
void refreshDemoData();
```

Шаг 5. В `ai_project.cpp` добавь `include`

В верхний блок `#include` добавь:

```
#include <QTimer>
```

Удобнее поставить рядом с остальными Qt include, например после:

```
#include <QToolTip>
```

Шаг 6. В конструкторе добавь инициализацию таймера и кнопки

В начале конструктора найди:

```
networkManager(new QNetworkAccessManager(this)),  
coinList(nullptr),
```

Замени на:

```
networkManager(new QNetworkAccessManager(this)),  
autoUpdateTimer(new QTimer(this)),  
coinList(nullptr),
```

Теперь ниже найди блок инициализации кнопок:

```
levelsButton(nullptr),  
refreshButton(nullptr),
```

Замени на:

```
levelsButton(nullptr),  
refreshButton(nullptr),  
autoUpdateButton(nullptr),
```

Шаг 7. В конструкторе настрой таймер

В конструкторе после строки:

```
setupStyle();
```

Добавь:

```
autoUpdateTimer->setInterval(60000);  
  
connect(autoUpdateTimer, &QTimer::timeout, this, [this]()  
{  
    fetchMarketData();  
});
```

Должно получиться примерно так:

```
fillDemoData();
setupInterface();
setupStyle();

autoUpdateTimer->setInterval(60000);

connect(autoUpdateTimer, &QTimer::timeout, this, [this]()
{
    fetchMarketData();
});

buildCoinList();
refreshAlertsList();
```

Шаг 8. В `setupInterface()` создай кнопку автообновления

Найди блок:

```
lastUpdateLabel = new QLabel("Последнее обновление: —");
lastUpdateLabel->setObjectName("MutedText");

refreshButton = createSmallButton("Обновить");
```

Замени на:

```
lastUpdateLabel = new QLabel("Последнее обновление: —");
lastUpdateLabel->setObjectName("MutedText");

refreshButton = createSmallButton("Обновить");

autoUpdateButton = createSmallButton("Автообновление: выкл");
autoUpdateButton->setCheckable(true);
```

Шаг 9. Добавь кнопку в верхнюю правую область

Чуть ниже найди:

```
updateLayout->addWidget(lastUpdateLabel);
updateLayout->addWidget(refreshButton, 0, Qt::AlignRight);
```

Замени на:

```
updateLayout->addWidget(lastUpdateLabel);  
updateLayout->addWidget(refreshButton, 0, Qt::AlignRight);  
updateLayout->addWidget(autoUpdateButton, 0, Qt::AlignRight);
```

Шаг 10. Подключи кнопку автообновления

В `setupInterface()` найди:

```
connect(refreshButton, &QPushButton::clicked, this, &ai_project::refreshDemoData);
```

Сразу после неё добавь:

```
connect(autoUpdateButton, &QPushButton::clicked, this, &ai_project::toggleAutoUpdate);
```

Шаг 11. Защити `fetchMarketData()` от двойных запросов

Найди функцию:

```
void ai_project::fetchMarketData()  
{  
    refreshButton->setEnabled(false);  
    refreshButton->setText("Загрузка...");  
}
```

Замени начало функции на:

```
void ai_project::fetchMarketData()  
{  
    if (!refreshButton->isEnabled())  
        return;  
  
    refreshButton->setEnabled(false);  
    refreshButton->setText("Загрузка...");  
}
```

Это нужно, чтобы таймер не запускал новый запрос, пока старый ещё не завершился.

Шаг 12. В конец `ai_project.cpp` добавь функцию `toggleAutoUpdate`

В самый конец файла добавь:

```
void ai_project::toggleAutoUpdate()
{
    if (autoUpdateButton->isChecked())
    {
        autoUpdateTimer->start();
        autoUpdateButton->setText("Автообновление: вкл");

        lastUpdateLabel->setText(
            "Автообновление включено: каждые 60 секунд"
        );

        fetchMarketData();
    }
    else
    {
        autoUpdateTimer->stop();
        autoUpdateButton->setText("Автообновление: выкл");

        lastUpdateLabel->setText(
            "Автообновление выключено"
        );
    }
}
```

Шаг 13. Пересобери

Сборка → Очистить проект
Сборка → Запустить `qmake`
Сборка → Пересобрать проект

Как проверить

1. Запусти приложение.
2. Нажми кнопку:

Автообновление: выкл

Она должна стать:

```
Автообновление:  вкл
```

- 3. Данные сразу обновятся.
- 4. Потом приложение будет обновляться само раз в 60 секунд.
- 5. Добавь тестовое оповещение:

```
BTC цена выше 1000
```

- 6. При автообновлении оно должно сработать.

Что останется после этого

После автообновления останутся 3 важные доработки:

```
1. Удаление выбранного оповещения.  
2. Сохранение оповещений в alerts.json.  
3. Сохранение настроек приложения.
```

Следующим лучше сделать **удаление и очистку оповещений**, потому что сейчас список можно только пополнять.

Prompt:

24.05.2026, 17:07:00

давай дальше удаление и очистку оповещений

Response:

24.05.2026, 17:07:12

Проверка кода и интерфейса

Необходимо продолжить код на русском, без использования внешних ресурсов. Вопрос касается модификации интерфейса и методов. Нужно корректно обновить список предупреждений, добавив кнопки и правильно обрабатывая пустые или неактивные данные. Удаление выбранных элементов должно не исчезать, если список пуст.

Думал на протяжении 7s

Делаем управление оповещениями:

1. Удалить выбранное оповещение.
2. Очистить все сработавшие оповещения.
3. Не трогать активные оповещения при очистке сработавших.
4. Защититься от удаления пустой строки "Активных оповещений нет".

Шаг 1. В `ai_project.h` добавь функции

Найди блок:

```
void addAlert();
void refreshAlertsList();
void checkPriceAlerts();
QString alertText(const PriceAlertUiData &alert) const;
```

Замени на:

```
void addAlert();
void deleteSelectedAlert();
void clearTriggeredAlerts();
void refreshAlertsList();
void checkPriceAlerts();
QString alertText(const PriceAlertUiData &alert) const;
```

Шаг 2. В `setupInterface()` добавь кнопки под списком оповещений

В `ai_project.cpp` найди этот блок:

```
alertsList = new QListWidget();
alertsList->setObjectName("SimpleList");
alertsList->setMinimumHeight(125);
```

Сразу после него вставь:

```
QPushButton *deleteAlertButton = createSmallButton("Удалить выбранное");
QPushButton *clearTriggeredAlertsButton = createSmallButton("Очистить сработавшие");

QHBoxLayout *alertButtonsLayout = new QHBoxLayout();
alertButtonsLayout->setSpacing(8);
alertButtonsLayout->addWidget(deleteAlertButton);
alertButtonsLayout->addWidget(clearTriggeredAlertsButton);
```

Шаг 3. Добавь кнопки в правую панель

Найди блок:

```
rightLayout->addWidget(alertsTitle);
rightLayout->addWidget(alertCoinCombo);
rightLayout->addWidget(alertConditionCombo);
rightLayout->addWidget(alertPriceEdit);
rightLayout->addWidget(addAlertButton);
rightLayout->addWidget(alertsList);
rightLayout->addSpacing(14);
```

Замени его на:

```
rightLayout->addWidget(alertsTitle);
rightLayout->addWidget(alertCoinCombo);
rightLayout->addWidget(alertConditionCombo);
rightLayout->addWidget(alertPriceEdit);
rightLayout->addWidget(addAlertButton);
rightLayout->addWidget(alertsList);
rightLayout->addLayout(alertButtonsLayout);
rightLayout->addSpacing(14);
```

Шаг 4. Подключи кнопки к функциям

В `setupInterface()` найди:

```
connect(addAlertButton, &QPushButton::clicked, this, &ai_project::addAlert);
```

Сразу после неё вставь:

```
connect(deleteAlertButton, &QPushButton::clicked, this, &ai_project::deleteSelectedAlert);
connect(clearTriggeredAlertsButton, &QPushButton::clicked, this, &ai_project::clearTriggeredA
```

Должно получиться так:

```
connect(addAlertButton, &QPushButton::clicked, this, &ai_project::addAlert);
connect(deleteAlertButton, &QPushButton::clicked, this, &ai_project::deleteSelectedAlert);
connect(clearTriggeredAlertsButton, &QPushButton::clicked, this, &ai_project::clearTriggeredA

connect(calculateButton, &QPushButton::clicked, this, &ai_project::updateConverter);
```


Шаг 5. Замени функцию refreshAlertsList()

Найди старую функцию:

```
void ai_project::refreshAlertsList()
{
    ...
}
```

И полностью замени на:

```
void ai_project::refreshAlertsList()
{
    alertsList->clear();

    if (priceAlerts.isEmpty())
    {
        QListWidgetItem *emptyItem = new QListWidgetItem("Оповещений пока нет");
        emptyItem->setForeground(QColor("#64748b"));
        emptyItem->setFlags(Qt::NoItemFlags);

        alertsList->addItem(emptyItem);
        return;
    }

    for (int i = 0; i < priceAlerts.size(); ++i)
    {
        const PriceAlertUiData &alert = priceAlerts[i];

        QListWidgetItem *item = new QListWidgetItem(
            QString::number(i + 1) + ". " + alertText(alert)
        );

        item->setData(Qt::UserRole, i);

        if (alert.active)
        {
            item->setForeground(QColor("#e5e7eb"));
        }
        else
        {
            item->setForeground(QColor("#64748b"));
        }

        alertsList->addItem(item);
    }
}
```

Шаг 6. В конец ai_project.cpp добавь функции удаления

В самый конец файла добавь:

```
void ai_project::deleteSelectedAlert()
{
    if (priceAlerts.isEmpty())
    {
        QMessageBox::information(
            this,
            "Удаление оповещения",
            "Список оповещений пуст."
        );
        return;
    }

    int row = alertsList->currentRow();

    if (row < 0 || row >= priceAlerts.size())
    {
        QMessageBox::information(
            this,
            "Удаление оповещения",
            "Выберите оповещение в списке."
        );
        return;
    }

    PriceAlertUiData removedAlert = priceAlerts[row];

    QMessageBox::StandardButton answer = QMessageBox::question(
        this,
        "Удаление оповещения",
        "Удалить выбранное оповещение?\n\n" + alertText(removedAlert),
        QMessageBox::Yes | QMessageBox::No
    );

    if (answer != QMessageBox::Yes)
        return;

    priceAlerts.removeAt(row);
    refreshAlertsList();
}

void ai_project::clearTriggeredAlerts()
{
    if (priceAlerts.isEmpty())
    {
        QMessageBox::information(
```

```

        this,
        "Очистка оповещений",
        "Список оповещений пуст."
    );
    return;
}

int triggeredCount = 0;

for (const PriceAlertUiData &alert : priceAlerts)
{
    if (!alert.active)
        ++triggeredCount;
}

if (triggeredCount == 0)
{
    QMessageBox::information(
        this,
        "Очистка оповещений",
        "Сработавших оповещений нет."
    );
    return;
}

QMessageBox::StandardButton answer = QMessageBox::question(
    this,
    "Очистка оповещений",
    "Удалить все сработавшие оповещения?\n\n"
    "Будет удалено: " + QString::number(triggeredCount),
    QMessageBox::Yes | QMessageBox::No
);

if (answer != QMessageBox::Yes)
    return;

for (int i = priceAlerts.size() - 1; i >= 0; --i)
{
    if (!priceAlerts[i].active)
        priceAlerts.removeAt(i);
}

refreshAlertsList();
}

```

Шаг 7. Пересобери проект

Сборка → Очистить проект
 Сборка → Запустить qmake
 Сборка → Пересобрать проект

Как проверить

Проверка удаления выбранного

1. Добавь 2–3 оповещения.
2. Кликни по одному из них.
3. Нажми:

Удалить выбранное

4. Подтверди удаление.

Должно удалиться только выбранное оповещение.

Проверка очистки сработавших

1. Добавь тестовое оповещение, которое сразу выполнится:

BTC цена выше 1000

2. Нажми Обновить .
3. Оповещение должно стать [сработало] .
4. Нажми:

Очистить сработавшие

5. Активные оповещения должны остаться, сработавшие — исчезнуть.

После этого останутся две финальные крупные вещи:

1. Сохранение оповещений в `alerts.json`.
2. Сохранение настроек приложения.

Лучше следующим этапом сделать `alerts.json` , потому что без сохранения список оповещений всё ещё пропадает после закрытия программы.

Prompt:

давай к сохранению оповещений, только, в кнопках удалить выбранное и очистить сработавшее, текст не влезает в кнопку и не видно текста, либо кнопку больше сделать либо текст поменьше

Response:

24.05.2026, 17:11:44

Разработка функций для работы с уведомлениями

Нужно ответить на вопрос на русском. Пользователь просил код — используем патчи вместо полного кода и придерживаемся требований. Нужно сохранять уведомления в JSON и использовать методы для сохранения и загрузки. Для этого воспользуемся функциями, такими как `QStandardPaths` и `QFile`, перед вызовом `refreshAlertsList` добавим загрузку уведомлений.

Думал на протяжении 7s

Делаем сразу две вещи:

1. **Исправляем кнопки оповещений**, чтобы текст нормально помещался.
2. **Добавляем сохранение оповещений в `alerts.json`**, чтобы после закрытия приложения они не пропадали.

Файл будет создаваться рядом с `.exe` приложения:

```
alerts.json
```

1. Исправляем кнопки “Удалить” и “Очистить”

Сейчас две длинные кнопки стоят в одну строку, поэтому текст не помещается. Лучше сделать их одна под другой.

В `setupInterface()` найди блок:

```
QPushButton *deleteAlertButton = createSmallButton("Удалить выбранное");
QPushButton *clearTriggeredAlertsButton = createSmallButton("Очистить сработавшие");

QHBoxLayout *alertButtonsLayout = new QHBoxLayout();
alertButtonsLayout->setSpacing(8);
alertButtonsLayout->addWidget(deleteAlertButton);
alertButtonsLayout->addWidget(clearTriggeredAlertsButton);
```

Замени на:

```
QPushButton *deleteAlertButton = createSmallButton("Удалить выбранное");
QPushButton *clearTriggeredAlertsButton = createSmallButton("Очистить сработавшие");

deleteAlertButton->setMinimumHeight(36);
clearTriggeredAlertsButton->setMinimumHeight(36);

QVBoxLayout *alertButtonsLayout = new QVBoxLayout();
alertButtonsLayout->setSpacing(8);
alertButtonsLayout->addWidget(deleteAlertButton);
alertButtonsLayout->addWidget(clearTriggeredAlertsButton);
```

Теперь кнопки будут во всю ширину правой панели, текст должен нормально влезать.

2. В `ai_project.h` добавь функции сохранения

Найди блок:

```
void addAlert();
void deleteSelectedAlert();
void clearTriggeredAlerts();
void refreshAlertsList();
void checkPriceAlerts();
QString alertText(const PriceAlertUiData &alert) const;
```

Замени на:

```
void addAlert();
void deleteSelectedAlert();
void clearTriggeredAlerts();
void refreshAlertsList();
void checkPriceAlerts();
QString alertText(const PriceAlertUiData &alert) const;

void saveAlertsToFile() const;
void loadAlertsFromFile();
QString alertsFilePath() const;
```

3. В `ai_project.cpp` добавь include

В верхний блок `#include` добавь:

```
#include <QFile>
```

Если хочешь проще, рядом с остальными include должно быть примерно так:

```
#include <QDateTime>
#include <QFile>
#include <QFrame>
```

4. В деструкторе добавь сохранение

Найди:

```
ai_project::~ai_project()
{
}
```

Замени на:

```
ai_project::~ai_project()
{
    saveAlertsToFile();
}
```

5. В конструкторе загрузи оповещения из файла

В конструкторе найди блок:

```
buildCoinList();
refreshAlertsList();

if (!coins.isEmpty())
```

Замени на:

```
buildCoinList();
loadAlertsFromFile();
refreshAlertsList();

if (!coins.isEmpty())
```

Теперь при запуске приложение сначала загрузит сохранённые оповещения, потом покажет их в списке.

6. В addAlert() добавь сохранение

В функции addAlert() найди конец:

```
alertPriceEdit->clear();
refreshAlertsList();

QMessageBox::information(
    this,
    "Оповещение добавлено",
    "Оповещение успешно добавлено:\n\n" + alertText(alert)
);
```

Замени на:

```
alertPriceEdit->clear();
refreshAlertsList();
saveAlertsToFile();

QMessageBox::information(
    this,
    "Оповещение добавлено",
    "Оповещение успешно добавлено:\n\n" + alertText(alert)
);
```

7. В deleteSelectedAlert() добавь сохранение

В функции deleteSelectedAlert() найди конец:

```
priceAlerts.removeAt(row);
refreshAlertsList();
```

Замени на:

```
priceAlerts.removeAt(row);
refreshAlertsList();
saveAlertsToFile();
```

8. В clearTriggeredAlerts() добавь сохранение

В функции `clearTriggeredAlerts()` найди конец:

```
refreshAlertsList();
```

Замени на:

```
refreshAlertsList();  
saveAlertsToFile();
```

Важно: если у тебя в функции `clearTriggeredAlerts()` несколько `refreshAlertsList();`, менять нужно именно самый последний, после удаления сработавших.

9. В `checkPriceAlerts()` добавь сохранение после срабатывания

В функции `checkPriceAlerts()` найди блок:

```
if (!triggeredAlerts.isEmpty())  
{  
    QMessageBox::information(  
        this,  
        "Сработало оповещение",  
        "Достигнут заданный уровень цены:\n\n" +  
        triggeredAlerts.join("\n")  
    );  
}
```

Замени на:

```
if (!triggeredAlerts.isEmpty())  
{  
    saveAlertsToFile();  
  
    QMessageBox::information(  
        this,  
        "Сработало оповещение",  
        "Достигнут заданный уровень цены:\n\n" +  
        triggeredAlerts.join("\n")  
    );  
}
```

Теперь, если оповещение сработало и стало неактивным, это состояние тоже сохранится.

10. В конец ai_project.cpp добавь функции сохранения

В самый конец файла добавь:

```
QString ai_project::alertsFilePath() const
{
    return QApplication::applicationDirPath() + "/alerts.json";
}

void ai_project::saveAlertsToFile() const
{
    QJsonArray alertsArray;

    for (const PriceAlertUiData &alert : priceAlerts)
    {
        QJsonObject object;

        object["coinId"] = alert.coinId;
        object["coinSymbol"] = alert.coinSymbol;
        object["targetPrice"] = alert.targetPrice;
        object["aboveCondition"] = alert.aboveCondition;
        object["active"] = alert.active;

        alertsArray.append(object);
    }

    QJsonDocument document(alertsArray);

    QFile file(alertsFilePath());

    if (!file.open(QIODevice::WriteOnly | QIODevice::Truncate))
        return;

    file.write(document.toJson(QJsonDocument::Indented));
    file.close();
}

void ai_project::loadAlertsFromFile()
{
    QFile file(alertsFilePath());

    if (!file.exists())
        return;

    if (!file.open(QIODevice::ReadOnly))
        return;

    QByteArray data = file.readAll();
    file.close();
}
```

```
QJsonParseError parseError;  
QJsonDocument document = QJsonDocument::fromJson(data, &parseError);  
  
if (parseError.error != QJsonParseError::NoError || !document.isArray())  
    return;  
  
QJsonArray alertsArray = document.array();  
  
priceAlerts.clear();  
  
for (const QJsonValue &value : alertsArray)  
{  
    if (!value.isObject())  
        continue;  
  
    QJsonObject object = value.toObject();  
  
    PriceAlertUiData alert;  
    alert.coinId = object.value("coinId").toString();  
    alert.coinSymbol = object.value("coinSymbol").toString();  
    alert.targetPrice = object.value("targetPrice").toDouble();  
    alert.aboveCondition = object.value("aboveCondition").toBool();  
    alert.active = object.value("active").toBool();  
  
    if (alert.coinId.isEmpty() || alert.coinSymbol.isEmpty() || alert.targetPrice <= 0.0)  
        continue;  
  
    priceAlerts.append(alert);  
}  
}
```

11. Пересобери

```
Сборка → Очистить проект  
Сборка → Запустить qmake  
Сборка → Пересобрать проект
```

Как проверить сохранение

1. Запусти приложение.
2. Добавь 2–3 оповещения.
3. Закрой приложение.
4. Запусти снова.
5. Оповещения должны остаться в списке.

Потом проверь файл рядом с .exe . Он должен называться:

```
alerts.json
```

Внутри будет примерно так:

```
[
  {
    "aboveCondition": true,
    "active": true,
    "coinId": "bitcoin",
    "coinSymbol": "BTC",
    "targetPrice": 70000
  }
]
```

После этого останется последний крупный блок: **сохранение настроек интерфейса** — выбранная монета, период, тип графика, включены ли уровни и автообновление.

Prompt:

24.05.2026, 17:18:15

все топ - давай завершающее

Response:

24.05.2026, 17:18:22

Заключение и сохранение настроек

Пользователь, похоже, хочет завершающий шаг, связанный с сохранением настроек и возможной доработкой. Не нужно снова ссылаться на файлы, если только не используются их конкретные данные. На этом этапе достаточно представить код, избегая излишних ссылок.

Думал на протяжении 16s

Завершающий этап: **сохранение настроек приложения.**

После этого проект уже можно считать функционально завершённым: при закрытии и повторном запуске будут сохраняться:

```
последняя выбранная монета;
выбранный период: 24ч / 7д / 30д / 90д;
тип графика: линия или свечи;
```

включены ли уровни поддержки/сопротивления;
состояние автообновления.

Файл будет создаваться рядом с `.exe` :

```
settings.json
```

1. В `ai_project.h` добавь функции настроек

Найди блок:

```
void saveAlertsToFile() const;  
void loadAlertsFromFile();  
QString alertsFilePath() const;
```

Сразу после него добавь:

```
void saveSettingsToFile() const;  
void loadSettingsFromFile();  
QString settingsFilePath() const;  
void applyPeriodButtonState();
```

Должно получиться так:

```
void saveAlertsToFile() const;  
void loadAlertsFromFile();  
QString alertsFilePath() const;  
  
void saveSettingsToFile() const;  
void loadSettingsFromFile();  
QString settingsFilePath() const;  
void applyPeriodButtonState();
```

2. В `ai_project.cpp` добавь include

В верхний блок `#include` добавь:

```
#include <QCoreApplication>
```

Если уже есть `QApplication` , всё равно добавь — так будет чище.

3. Замени деструктор

Найди:

```
ai_project::~~ai_project()
{
    saveAlertsToFile();
}
```

Замени на:

```
ai_project::~~ai_project()
{
    saveAlertsToFile();
    saveSettingsToFile();
}
```

4. В конструкторе загрузи настройки

В конструкторе найди этот блок:

```
buildCoinList();
loadAlertsFromFile();
refreshAlertsList();

if (!coins.isEmpty())
{
    coinList->blockSignals(true);
    coinList->setCurrentRow(0);
    coinList->blockSignals(false);
    selectCoin(0);
}

fetchMarketData();
```

Замени на:

```
buildCoinList();
loadAlertsFromFile();
loadSettingsFromFile();
refreshAlertsList();

if (!coins.isEmpty())
{
    int selectedIndex = findCoinIndexById(selectedCoinId);
```

```

if (selectedIndex < 0)
    selectedIndex = 0;

coinList->blockSignals(true);
coinList->setCurrentRow(selectedIndex);
coinList->blockSignals(false);

selectCoin(selectedIndex);
}

if (autoUpdateButton->isChecked())
{
    autoUpdateTimer->start();
    autoUpdateButton->setText("Автообновление: вкл");
}

fetchMarketData();

```

5. В переключение периода добавь сохранение

В `setupInterface()` найди обработчик `period24Button`:

```

connect(period24Button, &QPushButton::clicked, this, [this]()
{
    selectedPeriod = "24ч";
    periodInfoLabel->setText("Период: 24 часа, шаг: 1 час");
    selectCoin(coinList->currentRow());
});

```

Замени на:

```

connect(period24Button, &QPushButton::clicked, this, [this]()
{
    selectedPeriod = "24ч";
    periodInfoLabel->setText("Период: 24 часа, шаг: 1 час");
    selectCoin(coinList->currentRow());
    saveSettingsToFile();
});

```

Теперь аналогично добавь `saveSettingsToFile()`; в обработчики `7д`, `30д`, `90д`.

Например `7д` должен стать:

```

connect(period7Button, &QPushButton::clicked, this, [this]()
{
    selectedPeriod = "7д";

```

```
periodInfoLabel->setText("Период: 7 дней, шаг: 4 часа");  
selectCoin(coinList->currentRow());  
saveSettingsToFile();  
});
```

6. В переключение графика добавь сохранение

Найди:

```
connect(lineButton, &QPushButton::clicked, this, [this]()  
{  
    candleMode = false;  
    priceChart->setChartType(MiniChartWidget::LineChart);  
});
```

Замени на:

```
connect(lineButton, &QPushButton::clicked, this, [this]()  
{  
    candleMode = false;  
    priceChart->setChartType(MiniChartWidget::LineChart);  
    saveSettingsToFile();  
});
```

Найди:

```
connect(candleButton, &QPushButton::clicked, this, [this]()  
{  
    candleMode = true;  
    priceChart->setChartType(MiniChartWidget::CandleChart);  
});
```

Замени на:

```
connect(candleButton, &QPushButton::clicked, this, [this]()  
{  
    candleMode = true;  
    priceChart->setChartType(MiniChartWidget::CandleChart);  
    saveSettingsToFile();  
});
```

7. В кнопку уровней добавь сохранение

Найди:

```
connect(levelsButton, &QPushButton::clicked, this, [this]()
{
    showLevels = levelsButton->isChecked();

    int currentRow = coinList->currentRow();

    if (currentRow >= 0)
        selectCoin(currentRow);
});
```

Замени на:

```
connect(levelsButton, &QPushButton::clicked, this, [this]()
{
    showLevels = levelsButton->isChecked();

    int currentRow = coinList->currentRow();

    if (currentRow >= 0)
        selectCoin(currentRow);

    saveSettingsToFile();
});
```

8. В selectCoin() добавь сохранение выбранной монеты

Найди:

```
void ai_project::selectCoin(int row)
{
    if (row < 0 || row >= coins.size())
        return;

    selectedCoinId = coins[row].id;

    updateCenterPanel(coins[row]);
    fetchHistoryData(selectedCoinId, selectedPeriod);
}
```

Замени на:

```
void ai_project::selectCoin(int row)
{
```

```
if (row < 0 || row >= coins.size())  
    return;  
  
selectedCoinId = coins[row].id;  
  
updateCenterPanel(coins[row]);  
fetchHistoryData(selectedCoinId, selectedPeriod);  
saveSettingsToFile();  
}
```

9. В toggleAutoUpdate() добавь сохранение

Найди функцию:

```
void ai_project::toggleAutoUpdate()  
{  
    if (autoUpdateButton->isChecked())  
    {  
        autoUpdateTimer->start();  
        autoUpdateButton->setText("Автообновление: вкл");  
  
        lastUpdateLabel->setText(  
            "Автообновление включено: каждые 60 секунд"  
        );  
  
        fetchMarketData();  
    }  
    else  
    {  
        autoUpdateTimer->stop();  
        autoUpdateButton->setText("Автообновление: выкл");  
  
        lastUpdateLabel->setText(  
            "Автообновление выключено"  
        );  
    }  
}
```

Замени на:

```
void ai_project::toggleAutoUpdate()  
{  
    if (autoUpdateButton->isChecked())  
    {  
        autoUpdateTimer->start();  
        autoUpdateButton->setText("Автообновление: вкл");  
  
        lastUpdateLabel->setText(  
            "Автообновление включено: каждые 60 секунд"
```

```

);

fetchMarketData();
}
else
{
    autoUpdateTimer->stop();
    autoUpdateButton->setText("Автообновление: выкл");

    lastUpdateLabel->setText(
        "Автообновление выключено"
    );
}

saveSettingsToFile();
}

```

10. В конец ai_project.cpp добавь функции настроек

В самый конец файла добавь:

```

QString ai_project::settingsFilePath() const
{
    return QApplication::applicationDirPath() + "/settings.json";
}

void ai_project::saveSettingsToFile() const
{
    QJsonObject object;

    object["selectedCoinId"] = selectedCoinId;
    object["selectedPeriod"] = selectedPeriod;
    object["candleMode"] = candleMode;
    object["showLevels"] = showLevels;
    object["autoUpdate"] = autoUpdateButton && autoUpdateButton->isChecked();

    if (fromCoinCombo)
        object["converterFrom"] = fromCoinCombo->currentData().toString();

    if (toCoinCombo)
        object["converterTo"] = toCoinCombo->currentData().toString();

    QJsonDocument document(object);

    QFile file(settingsFilePath());

    if (!file.open(QIODevice::WriteOnly | QIODevice::Truncate))

```

```

        return;

        file.write(document.toJson(QJsonDocument::Indented));
        file.close();
    }

void ai_project::loadSettingsFromFile()
{
    QFile file(settingsFilePath());

    if (!file.exists())
        return;

    if (!file.open(QIODevice::ReadOnly))
        return;

    QByteArray data = file.readAll();
    file.close();

    QJsonParseError parseError;
    QJsonDocument document = QJsonDocument::fromJson(data, &parseError);

    if (parseError.error != QJsonParseError::NoError || !document.isObject())
        return;

    QJsonObject object = document.object();

    QString loadedCoinId = object.value("selectedCoinId").toString();

    if (findCoinIndexById(loadedCoinId) >= 0)
        selectedCoinId = loadedCoinId;

    QString loadedPeriod = object.value("selectedPeriod").toString();

    if (loadedPeriod == "24ч" || loadedPeriod == "7д" ||
        loadedPeriod == "30д" || loadedPeriod == "90д")
    {
        selectedPeriod = loadedPeriod;
    }

    candleMode = object.value("candleMode").toBool(false);
    showLevels = object.value("showLevels").toBool(true);

    lineButton->setChecked(!candleMode);
    candleButton->setChecked(candleMode);
    levelsButton->setChecked(showLevels);

    autoUpdateButton->setChecked(object.value("autoUpdate").toBool(false));
    autoUpdateButton->setText(autoUpdateButton->isChecked()
        ? "Автообновление: вкл"
        : "Автообновление: выкл");

    applyPeriodButtonState();

```

```
QString converterFrom = object.value("converterFrom").toString();
QString converterTo = object.value("converterTo").toString();

int fromIndex = fromCoinCombo->findData(converterFrom);
int toIndex = toCoinCombo->findData(converterTo);

if (fromIndex >= 0)
    fromCoinCombo->setCurrentIndex(fromIndex);

if (toIndex >= 0)
    toCoinCombo->setCurrentIndex(toIndex);
}

void ai_project::applyPeriodButtonState()
{
    period24Button->setChecked(selectedPeriod == "24ч");
    period7Button->setChecked(selectedPeriod == "7д");
    period30Button->setChecked(selectedPeriod == "30д");
    period90Button->setChecked(selectedPeriod == "90д");

    if (selectedPeriod == "24ч")
        periodInfoLabel->setText("Период: 24 часа, шаг: 1 час");
    else if (selectedPeriod == "7д")
        periodInfoLabel->setText("Период: 7 дней, шаг: 4 часа");
    else if (selectedPeriod == "30д")
        periodInfoLabel->setText("Период: 30 дней, шаг: 1 день");
    else
        periodInfoLabel->setText("Период: 90 дней, шаг: 1 день");
}
```

11. Пересобери проект

Сборка → Очистить проект
Сборка → Запустить qmake
Сборка → Пересобрать проект

12. Как проверить

1. Запусти приложение.
2. Выбери, например, Ethereum.
3. Поставь период 30д.
4. Включи Свечи.
5. Выключи Уровни.
6. Включи автообновление.

7. Закрой приложение.

8. Запусти снова.

Должно восстановиться:

```
Ethereum;  
30д;  
свечной график;  
уровни выключены;  
автообновление включено.
```

Итоговое состояние проекта

Теперь проект можно считать завершённым по функционалу.

Реализовано:

```
реальные текущие цены CoinGecko;  
история цен;  
линейный график;  
свечной график open/high/low/close;  
объёмы торгов;  
RSI;  
уровни поддержки и сопротивления;  
включение/выключение уровней;  
конвертор криптовалют;  
добавление оповещений;  
срабатывание оповещений;  
удаление выбранного оповещения;  
очистка сработавших оповещений;  
сохранение оповещений;  
автообновление;  
сохранение настроек интерфейса;  
обработка сетевых ошибок.
```

Дальше уже не код, а подготовка к сдаче: описание проекта, структура программы, речь для защиты и ответы на возможные вопросы преподавателя.

Prompt:

24.05.2026, 17:27:20

вот при кнопке обновить, биток эфир и солана обновляются хорошо, все правильно, но потом доге коин и кардано с плохими графиками - обновить кнопка уже не работает вот такое сообщение выходит, хотя в начале первые два раза кнопка обновить работает хорошо без проблем - то есть впринципе она работоспособна

Response:

24.05.2026, 17:27:25

Решение проблемы с ограничением запросов (429)

Пользователю нужно объяснить, что ошибка 429 возникает из-за слишком большого числа запросов. Проблему можно решить с помощью кэширования или ограничение частоты запросов. Важно внести изменения в код, чтобы избежать "штормов запросов". Нужно добавить проверку и флаги для отслеживания процессов запросов.

Ограничение запросов и уменьшение частоты обновлений

Проблема с ошибкой 429 возникает, когда слишком часто обновляются данные рынка и истории. Нужно сократить частоту автоматических обновлений и использовать кэширование для истории. Также можно увеличить интервал обновлений, чтобы уменьшить нагрузку на API. Простой фикс — увеличить интервал автообновления до 2-3 минут.

Думал на протяжении 14s

Это не поломка кнопки. На скрине важная строка:

HTTP-код: 429

429 означает: **слишком много запросов к CoinGecko за короткое время**. То есть кнопка работает, интернет работает, SSL работает, но CoinGecko временно режет запросы.

Проблема у нас в логике: после обновления текущих цен приложение снова вызывает `selectCoin()`, а `selectCoin()` дополнительно тянет историю графика. В итоге один "Обновить" может превращаться не в один запрос, а в цепочку запросов. CoinGecko это быстро ограничивает.

Сейчас исправляем правильно:

1. Кнопка `Обновить` будет обновлять **текущие цены**, но не будет лишней раз перезагружать историю.
2. История будет загружаться при выборе монеты или периода.
3. При ошибке 429 больше не будет огромного всплывающего окна, а будет нормальный статус: "лимит запросов, подождите".
4. Добавим защиту от слишком частого ручного обновления.

Шаг 1. В `ai_project.h` добавь поля защиты от частых запросов

Найди блок:

```
QString selectedCoinId;
QString selectedPeriod;
bool candleMode;
bool showLevels;
```

Замени на:

```
QString selectedCoinId;
QString selectedPeriod;
bool candleMode;
bool showLevels;

bool marketRequestInProgress;
qint64 lastMarketRequestMs;
```

Шаг 2. В конструкторе добавь инициализацию

В `ai_project.cpp` найди конец списка инициализации:

```
selectedPeriod("24ч"),
candleMode(false),
showLevels(true)
```

Замени на:

```
selectedPeriod("24ч"),
candleMode(false),
showLevels(true),
marketRequestInProgress(false),
lastMarketRequestMs(0)
```

Если у тебя после `showLevels(true)` уже есть запятая или ещё поля — просто аккуратно добавь эти два значения в конец.

Шаг 3. Замени функцию `fetchMarketData()`

Найди функцию:


```
void ai_project::fetchMarketData()
{
    ...
}
```

Полностью замени её на:

```
void ai_project::fetchMarketData()
{
    if (marketRequestInProgress)
        return;

    qint64 nowMs = QDateTime::currentMSecsSinceEpoch();

    if (lastMarketRequestMs > 0 && nowMs - lastMarketRequestMs < 15000)
    {
        lastUpdateLabel->setText(
            "Слишком частое обновление. Подождите несколько секунд."
        );
        return;
    }

    marketRequestInProgress = true;
    lastMarketRequestMs = nowMs;

    refreshButton->setEnabled(false);
    refreshButton->setText("Загрузка...");

    QUrl url(
        "https://api.coingecko.com/api/v3/coins/markets"
        "?vs_currency=usd"
        "&ids=bitcoin,ethereum,solana,dogecoin,cardano,ripple,polkadot"
        "&order=market_cap_desc"
        "&per_page=7"
        "&page=1"
        "&sparkline=false"
        "&price_change_percentage=24h"
    );

    QNetworkRequest request(url);
    request.setHeader(QNetworkRequest::UserAgentHeader, "CryptoTrackQtCourseProject/1.0");

    QNetworkReply *reply = networkManager->get(request);

    connect(reply, &QNetworkReply::finished, this, [this, reply]()
    {
        handleMarketDataReply(reply);
        reply->deleteLater();
    });
}
```

Шаг 4. В `handleMarketDataReply()` в начале сбрасывай флаг запроса

Найди начало функции:

```
void ai_project::handleMarketDataReply(QNetworkReply *reply)
{
    refreshButton->setEnabled(true);
    refreshButton->setText("Обновить");
}
```

Замени на:

```
void ai_project::handleMarketDataReply(QNetworkReply *reply)
{
    marketRequestInProgress = false;

    refreshButton->setEnabled(true);
    refreshButton->setText("Обновить");
}
```

Шаг 5. В `handleMarketDataReply()` отдельно обработай ошибку 429

В этой же функции найди блок:

```
if (reply->error() != QNetworkReply::NoError)
{
    int statusCode = reply->attribute(QNetworkRequest::HttpStatusCodeAttribute).toInt();

    QString sslInfo;

    if (QSslSocket::supportsSsl())
    {
        sslInfo =
            "SSL: поддерживается\n"
            "Версия SSL: " + QSslSocket::sslLibraryVersionString();
    }
    else
    {
        sslInfo =
            "SSL: НЕ поддерживается\n"
            "Вероятная причина: Qt не видит OpenSSL-библиотеки.";
    }

    QString errorText =
```

```

        "Не удалось получить данные CoinGecko.\n\n"
        "Техническая причина:\n" +
        reply->errorString() +
        "\n\nHTTP-код: " + QString::number(statusCode) +
        "\n\n" + sslInfo +
        "\n\nПриложение продолжит работу на демонстрационных данных.";

lastUpdateLabel->setText("Ошибка загрузки: используются демо-данные");

QMessageBox::warning(
    this,
    "Ошибка загрузки",
    errorText
);

return;
}

```

Замени его на:

```

if (reply->error() != QNetworkReply::NoError)
{
    int statusCode = reply->attribute(QNetworkRequest::HttpStatusCodeAttribute).toInt();

    if (statusCode == 429)
    {
        lastUpdateLabel->setText(
            "CoinGecko временно ограничил запросы. Подождите 1-2 минуты."
        );

        QMessageBox::information(
            this,
            "Лимит запросов",
            "CoinGecko временно ограничил количество запросов.\n\n"
            "Это не ошибка программы. Подождите 1-2 минуты и нажмите обновить снова.\n"
            "Чтобы не ловить лимит, не нажимайте обновление слишком часто."
        );

        return;
    }

    int currentRow = coinList->currentRow();

    if (currentRow >= 0 && currentRow < coins.size())
        updateCenterPanel(coins[currentRow]);

    lastUpdateLabel->setText(
        "Не удалось обновить данные. Используются последние загруженные значения."
    );

    return;
}

```

Теперь приложение не будет каждый раз показывать огромную техническую ошибку.

Шаг 6. Исправь главную причину лишних запросов

В `handleMarketDataReply()` найди успешный конец обработки. Сейчас там должен быть примерно такой блок:

```
QString oldSelectedId = selectedCoinId;

buildCoinList();

int selectedIndex = findCoinIndexById(oldSelectedId);

if (selectedIndex < 0)
    selectedIndex = 0;

coinList->blockSignals(true);
coinList->setCurrentRow(selectedIndex);
coinList->blockSignals(false);

selectCoin(selectedIndex);

lastUpdateLabel->setText(
    "Последнее обновление: " +
    QDateTime::currentDateTime().toString("dd.MM.yyyy HH:mm:ss")
);

checkPriceAlerts();
```

Замени его на:

```
QString oldSelectedId = selectedCoinId;

buildCoinList();

int selectedIndex = findCoinIndexById(oldSelectedId);

if (selectedIndex < 0)
    selectedIndex = 0;

coinList->blockSignals(true);
coinList->setCurrentRow(selectedIndex);
coinList->blockSignals(false);

selectedCoinId = coins[selectedIndex].id;
updateCenterPanel(coins[selectedIndex]);
```

```
updateConverter();

lastUpdateLabel->setText(
    "Последнее обновление цен: " +
    QDateTime::currentDateTime().toString("dd.MM.yyyy HH:mm:ss")
);

checkPriceAlerts();
```

Ключевая замена:

```
selectCoin(selectedIndex);
```

мы убрали.

Почему это важно: `selectCoin()` вызывает `fetchHistoryData()`. А при обычном обновлении цен нам не надо каждый раз грузить историю графика. Именно из-за этого у тебя шёл лишний поток запросов.

Шаг 7. Увеличь интервал автообновления

В конструкторе найди:

```
autoUpdateTimer->setInterval(60000);
```

Замени на:

```
autoUpdateTimer->setInterval(120000);
```

То есть автообновление будет раз в 2 минуты. Для CoinGecko это безопаснее.

Шаг 8. Исправь текст автообновления

В `toggleAutoUpdate()` найди:

```
"Автообновление включено: каждые 60 секунд"
```

Замени на:

```
"Автообновление включено: каждые 120 секунд"
```

Шаг 9. Пересобери

```
Сборка → Очистить проект
Сборка → Запустить qmake
Сборка → Пересобрать проект
```

Что изменится после правки

Теперь:

```
Обновить
```

делает только то, что должен делать:

```
обновляет текущие цены 7 монет;
проверяет оповещения;
обновляет левую панель и текущую цену.
```

А история графика будет грузиться только когда реально нужно:

```
выбрал другую монету;
переключил 24ч / 7д / 30д / 90д.
```

Это должно убрать ситуацию, где первые 2–3 обновления работают, а потом CoinGesco выдаёт 429 .

Если после правки 429 всё равно иногда появляется — это уже нормально для бесплатного API. Но приложение будет вести себя корректно: не ломаться, не сбивать графики и не спамить техническими ошибками.